

A MERGED RESEARCH AND IMPLEMENTATION REPORT

Surreal and Surcomplex Numbers

in Symbolic Computer Algebra

Exact Representation Tiers, Effective Cores, Certified Supports,
Analytic and Contour Layers, Branch and Phase Conventions,
and Three Delivered Wolfram Prototypes

September 21, 2026

Abstract

A symbolic computer algebra system can support large and useful parts of the surreal field **No** and its complexification **SC** = **No**[i] without attempting to store an arbitrary surreal number or to decide an arbitrary identity. The right product is not one universal numeric type but a layered family of exact domains with separately advertised, independently queryable capability contracts: an aggressively decidable algebraic core surrounded by explicitly partial extensions. Throughout we separate exact denotation, canonical representation, effective coefficient access, decidable equality and order, certified approximation, and proof-carrying evidence; these are six different promises, and a system that conflates them will overstate what it can do.

We fix an effective rational-monomial core and prove that it is an effective ordered field with terminating arithmetic, equality, valuation, leading coefficient and real sign, and we record the three different specializations of that core that were actually implemented. We prove three negative results — no total zero test for arbitrary computable coefficient programs (by two genuinely different reductions, whose morals differ), no total support-admissibility validator, and no universal leading-term or sign algorithm for high-rank program-described grid series even under a nonzero promise — together with one positive counterweight beyond the algebraic core, the 2026 D-algebraic transseries zero-test, carried with its hypotheses intact. A constructive finite-grid dependency lemma, given here in two independent constructions of its auxiliary positive functional, separates effective coefficient access from finite truncation at a requested valuation; a positive rank-one criterion and three counterexamples of two different kinds delimit that separation.

On the analytic side we develop exact functions in four separate categories: formal germs, ordinary holomorphic lifts, Hahn-coherent data on a fixed common ordinary domain, and the strictly weaker radius-free Hahn-germ algebra, in which coefficients share no ordinary radius. Polynomial algebra receives initial polynomials, Newton profiles, Hensel cluster lifting and two separate sharp root-perturbation theorems. Finite quotient algebras receive residue and trace pairings, multiplication matrices, a perturbed residue formula and a worked two-scale coupled system. Contour operations receive the coefficientwise Hahn functional, algebraic rational periods, microscopic rescaling, a separating torus and a certificate-based Stokes rewrite. Surcomplex angles receive the finite-angle circle, an exact cut-free phase, the Cayley coordinate, the canonical strips, two logarithm branches, and infinite circular phases under two distinct and separately attributed accounts of nonuniqueness.

Wolfram Language facilities are assessed facility by facility against official documentation consulted on September 21, 2026, and a set of host-integration prohibitions is stated. A proposed object architecture is given in proposal voice only; nothing named in it exists in any delivered file. **Three** separately delimited Wolfram Language prototypes are delivered: **RationalHahn** over $\mathbb{Q}$, **SurrealCASCore** over $\mathbb{Q}(i)$ with rational exponents, and **HahnRational** over $\mathbb{Q}(i)$ with integer exponents. Their envelopes differ on every axis that matters, two of them export six identical public symbol names with incompatible signatures, and no sentence in this report asserts the union of their capabilities.

Status and provenance. This is an implementation-oriented mathematical exposition merged from three independently written source articles. Six distinct supplied research manuscripts stand behind it, one of them in two different revisions; they are treated as sources, not as independently refereed or machine-verified publications. Established theorems, deductions proved in the sources, results imported from the supplied manuscripts, proposed interfaces, and executed example tests are distinguished throughout. The three original source packages supplied finite checks, with no Lean files, formalization blueprints, proof-assistant audits, or pinned Lean dependencies. Current repository formalization coverage is recorded separately in `docs/FORMALIZATION.md`; it is not supplied by these prototype tests. No universal surreal simplifier, complete surcomplex analytic system, or complete surreal CAS is claimed.

Contents

1	The answer: a layered exact system with queryable capabilities	1
1.1	What should be supported first	1
1.2	Six promises, and the four-way version two of the sources state	1
1.3	The running illustration	2
1.4	An initial capability map	2
1.5	Provenance: six supplied manuscripts, one of them in two revisions	3
2	Objects the software must distinguish	4
2.1	Normal forms, workspaces, and set-sized parents	4
2.2	Conway monomials are not generic powers	5
2.3	Ordinal arithmetic, field arithmetic, and the host's infinity	5
2.4	Order, valuation, standard part, and the finite part	6
2.5	Four incompatible uses of “infinitesimal”	6
2.6	A type is more than a coefficient container	6
3	Limits of finite representation and universal simplification	7
3.1	There is no finite notation for every surreal	7
3.2	The zero-test obstruction: one theorem, two reductions	7
3.3	Support admissibility is not a universal automatic test	8
3.4	Known nonzero does not make the leading term computable	9
3.5	Positive counterweights	9
3.6	Richardson's undecidability and its restrictions	10
4	A taxonomy of exact representations	10
4.1	Cuts, sign expansions, and the simplest-number semantics	10
4.2	Finite normal forms	11
4.3	Rational expressions	11
4.4	Algebraic specifications and branch selectors	11
4.5	Puiseux data and ramification	12
4.6	Grids, recursive towers, and unrestricted Hahn data	12
4.7	Expression graphs, transseries, and hyperseries	13
4.8	A representation and closure map	14
5	The effective core	14
5.1	One core theorem at the generality all three sources support	14
5.2	The three specializations, and which kernel implements which	15
5.3	The cancellation trap	16
5.4	Effective real closure and complexification	16
5.5	The non-density trap: why ordinary interval isolation is insufficient	17
5.6	What the core deliberately omits	18
5.7	What transfer does and does not buy	18
6	Certified supports and finite dependency	19
6.1	The two conditions, and four standard non-examples	19
6.2	Compositional certificates and the five support services	19
6.3	A certificate is not an evaluation algorithm	20
6.4	The finite-grid dependency lemma, with both constructions	20
6.5	The small-scale theorem and its effectivity gap	21
6.6	Inverses, positive recursions, and the reciprocal recurrence	22
7	Precision is a mathematical type, not a display setting	22
7.1	The $O_v(\cdot)$ convention	22
7.2	Certified precision propagation	23
7.3	The positive finite-prefix criterion, and three counterexamples	23
7.4	Three truncation interfaces, and two worked compressions	24
7.5	Nested descriptions and the ω th position	25
7.6	Coefficient precision is a second, independent axis	25
7.7	These are not fine-topological limits	25

7.8	Jets, asymptotics, and exact tails	26
7.9	Rounding and the $\Pi \oplus \mathbb{Z}$ floor	26
8	Polynomial algebra and root-cluster algorithms	26
8.1	Finite algebra over a declared coefficient domain	26
8.2	The initial polynomial and its exact root counts	27
8.3	Newton profiles without a real-valued plot	27
8.4	Hensel lifting, clusters, and algebraic fallback	29
8.5	Root-perturbation thresholds: two different sharp theorems	29
8.6	Implicit analytic roots	30
8.7	Near-tangency conditioning is algebraic information	31
9	Exact functions: formal germs, ordinary lifts, coherent data, radius-free germs	31
9.1	Formal evaluation versus ordinary lift	31
9.2	The Taylor lift, and what it does not confer	31
9.3	Which familiar functions can be included	33
9.4	Singular centres and the summability boundary	33
9.5	Formal, analytic, and asymptotic descriptors are different	33
9.6	Hahn-coherent data on a fixed common ordinary domain	34
9.7	The radius-free Hahn-germ algebra: a strictly weaker certificate	34
9.8	Composition contracts and the blind-substitution failure	35
9.9	Evaluation, translation, and finite jets	36
9.10	Preparation and division as operator algorithms	36
9.11	A nonpolynomial algebraic cluster: two developments of $\sin^2 z = t$	37
10	Finite quotient algebras and residues	38
10.1	Remainders, traces, and norms without choosing roots	38
10.2	Residue pairing versus trace pairing	38
10.3	An analytic observable in a polynomial quotient, by two routes	38
10.4	Normal forms, multiplication matrices, and the perturbed residue	39
10.5	Length preservation and the length-drop counterexample	39
10.6	The coupled system at two independent scales	40
11	Integration and contour operations	41
11.1	Primitives by category	41
11.2	The coefficientwise contour functional, and what does not justify it	42
11.3	Algebraic rational periods and winding by certified crossings	42
11.4	Microscopic rescaling	42
11.5	The separating torus, and why the determinant matters	43
11.6	Stokes as a certificate-based rewrite	44
11.7	Three circles that a single real precision parameter would confuse	44
11.8	Coherent primitives, fine-local constants, and a category warning	44
12	Exponentials, logarithms, and transserial scale semantics	44
12.1	Gonshor exp and log as exact constructors	44
12.2	The exp and log splits, and why the naive series fails	45
12.3	An exact exp-log expression language	45
12.4	Lambert W as an exact implicit constructor	46
12.5	Transseries backends and their published antecedents	46
12.6	Asymptotic data do not determine exact values	47
12.7	The factorial series and its exponentially small homogeneous solution	47
12.8	All-scale rigidity as a category boundary	48
12.9	Oscillation is a separate layer	48
13	Surcomplex arithmetic, angles, branches, and infinite phases	48
13.1	Pairs and squared modulus	48
13.2	All directions admit finite surreal angles	49
13.3	The Cayley coordinate	49
13.4	Canonical strips for exponential and trigonometry	49
13.5	Inverse trigonometry, and why an infinite argument needs no infinite phase	50

13.6	Two logarithm branches must not share one cache key	50
13.7	General complex powers and the strip obligation	51
13.8	Infinite phases I: the character classification (from TR)	51
13.9	Infinite phases II: the E_α family (from SA)	52
13.10	Finite trigonometric polynomials and the $2n$ root bound	53
13.11	Positivity must see infinitesimal angles	53
14	Differentiation: three operators	54
14.1	The three operators and their worked contrasts	54
14.2	Three proposed name sets, printed as three proposals	55
14.3	Differential equations must name their category	55
15	Host integration: what Wolfram Language supplies and what an adapter must add	55
15.1	A documentation-based assessment	55
15.2	Do not understate the current scope of <code>Root</code>	56
15.3	<code>SeriesData</code> is a view, not the universal storage format	57
15.4	<code>PossibleZeroQ</code> is heuristic	57
15.5	<code>NumericQ</code> , <code>N</code> , and the Archimedean impossibility	57
15.6	Infinity	58
15.7	Conservative dispatch, explicit operations before upvalues	58
15.8	Cache keys	59
15.9	Serialization and the code-execution risk	59
15.10	Lossless coercion before convenient notation	59
15.11	Precedents in other systems	59
16	A proposed architecture — explicitly not delivered	60
16.1	Architecture requirements all three sources agree on	60
16.2	Three concrete syntax sketches, side by side	61
16.3	Three result protocols, side by side	62
16.4	Component responsibilities	62
16.5	Proof-carrying expressions without a proof assistant	63
17	The three delivered prototypes	63
17.1	<code>RationalHahn</code> — the capability-tiers kernel, over $\mathbb{Q}$	64
17.2	<code>SurrealCASCore</code> — the layered-representations kernel, over $\mathbb{Q}(i)$ with $\mathbb{Q}^r$ exponents	65
17.3	<code>HahnRational</code> — the Gaussian-prototype kernel, over $\mathbb{Q}(i)$ with $\mathbb{Z}^d$ exponents	67
17.4	Comparison, symbol collisions, and four cross-attribution warnings	68
17.5	Recovered on audit	71
17.6	Shared contract boundaries of all three prototypes	71
18	What was actually executed	72
18.1	What the three archives recorded	72
18.2	What was independently reproduced for this report	74
18.3	The one real defect: <code>RationalHahn</code> cannot be loaded as distributed	74
18.4	What the executed checks do and do not establish	75
19	Implementation priorities, costs, and regression boundaries	75
19.1	A union staged plan	75
19.2	Complexity must include representation and output size	77
19.3	A union regression and acceptance list	77
20	Capability and operation matrix	78
21	Conclusions	80
21.1	The representable subsets and their operations	80
21.2	Four independent sources of limitation	81
21.3	Attach the necessary information to the object	81
A	Source map and boundaries of reliance	82
A.1	A note on what is not claimed	84

B	Minimal backend contracts and component responsibilities	85
C	Notation and semantic conventions	85
D	Reproducibility and archive contents	86
D.1	Building this report	86
D.2	Archive contents	86
D.3	Three separate run instructions	87
D.4	Combined statement	87
	References	88

1 The answer: a layered exact system with queryable capabilities

1.1 What should be supported first

A useful implementation should begin with an exact ordered field of rational expressions in explicitly ordered surreal monomials, extend it by algebraic root objects, and attach certified series representations when expansions are wanted. This already permits genuine infinite and infinitesimal quantities rather than a symbolic placeholder for a limit. Complexifying the real field gives an exact surcomplex coefficient domain, after which polynomial division, greatest common divisors, resultants, finite linear algebra and polynomial root specifications become natural operations.

The following calculations should all be straightforward in an initial system:

$$\begin{aligned}(\omega + 1)^2 - \omega^2 - 2\omega &= 1, \\ (\omega + 1)^{-1} &= \frac{t}{1+t}, \quad t = \omega^{-1}, \\ \sqrt{\omega^2 + 1} &= \omega\sqrt{1+t^2}, \\ \arctan \omega &= \frac{\pi}{2} - \arctan t.\end{aligned}$$

The second equality is already an exact answer; expanding it into infinitely many monomials is optional. The third requires an algebraic extension, and the fourth requires the canonical finite-angle analytic layer. A system should be able to explain these changes of domain rather than perform them silently.

A second layer should represent exact infinite series by finite rules with proved support conditions: evaluate Taylor series at infinitesimals, compute requested coefficients, and, where justified, return finite truncations with valuation error bounds. A third layer can add selected exponential–logarithmic and transserial constructors, and ordinary holomorphic functions lifted to finite surcomplex arguments. These are extensions with additional semantics, not consequences of adding one arithmetic head.

The design has a deliberately asymmetric policy. Exact algebra is aggressive inside a declared decidable domain; more general analytic expressions remain symbolic until their domain, support and branch obligations have been discharged. An unevaluated but well-defined expression is a legitimate exact result. An expression with an unproved summability obligation is not yet an admitted exact number. The system should never silently replace an unsupported value by zero, by an ordinary infinity, by a large real approximation, or by a convention-dependent phase.

Three questions guide every interface below. What mathematical object does the expression denote? What operations can the representation carry out with guaranteed termination? What information is retained when the system returns a finite approximation? Answering the first does not settle the other two.

1.2 Six promises, and the four-way version two of the sources state

The organizing distinction of this report is that “exact” names several independent capabilities that must be queried separately. We use the six-way list, which is a strict refinement of the four-way lists given independently by two of the three source articles. The refinement splits out *canonicity* — equal values have identical normalized data — and *proof-carrying representation*, neither of which the four-way lists name. Canonicity is not a cosmetic addition: it is exactly the property that

distinguishes the canonical sparse layer of one delivered prototype from that same prototype's deliberately unreduced fraction wrapper ([section 17](#)).

Promise	What it promises	What it does not promise
Exact denotation	A finite description singles out one mathematical value, without rounding.	A normal form, a numerical approximation, or a terminating equality test.
Canonical representation	Equal values have identical normalized data.	That the normalization is cheap, or that it exists for every admitted representation.
Effective coefficient access	A requested coefficient can be calculated by a terminating procedure.	That the first nonzero coefficient can be found, or that all coefficients below a cutoff can be listed.
Decidable algebra	Equality and the required field operations terminate on all valid inputs in a declared class.	Closure under every elementary or special function; acceptable complexity.
Certified approximation	The answer includes a proved error specification in a named valuation or topology.	Equality to the truncated expression, or approximation in the full surreal fine topology.
Proof-carrying representation	Checkable evidence is supplied for the conditions that make the expression meaningful.	A machine-checked formalization; the evidence is checked against trusted constructors.

Two of the three source articles present a four-way version of this taxonomy: denotationally exact / coefficient-effective relative to an indexed support description / decision-effective for a named operation set / certified truncation in one case, and exact denotation / effective coefficient access / decidable algebra / certified approximation in the other. Both insist, as we do, that the levels are independent and must be queried independently. We do not print “there are four levels of exactness” as the merged claim, because that would silently drop two capabilities.

1.3 The running illustration

Consider three outputs:

$$A = \frac{1}{1-t}, \quad B = 1 + t + \cdots + t^9 + O_v(10), \quad C = \text{StrongSum}(n \mapsto t^n).$$

Here $v(t) = 1$ and $O_v(10)$ means an error of valuation at least 10, as in [section 7.1](#). A names one exact element; it is a finite exact field expression whose Hahn normal form has infinitely many terms. B names a *precision class* unless it carries an additional exact-tail specification; the displayed polynomial $1 + t + \cdots + t^9$ is not the same exact number as A . And C names the same element as A only after the system has certified strong summability and the intended monomial interpretation. Printing an infinite summation symbol, or leaving a host expression inactive, supplies no certificate. Conversely, the program description of a series can be exact while equality of two such programs' values is undecidable ([theorem 3.5](#)). These are structural distinctions, not implementation inconveniences.

1.4 An initial capability map

The following map is the union of the two presentation taxonomies given by the sources. Its rows are not a claim that every object in an abstract Hahn field or every analytic germ is finitely describable;

they identify workable *presentation families* inside the mathematical objects.

Representation	Exact content available	Principal computational boundary
Hereditarily finite cuts and sign data	Dyadic rationals; birthday and simplicity data	Not closed under division; recursive trees grow; $1/3$ is already outside.
Finite Hahn sums / normal forms	Finite sums of admitted monomials; nested finite scales	Closed under $+$ and $\times$; reciprocals generally need another representation.
Ordered rational monomial expressions	A non-Archimedean fraction field	Total arithmetic, equality and real order for effective parents; algebraic independence and the monomial order must be fixed; not algebraically closed.
Ordered algebraic data	Real closure, complex polynomial roots	Needs non-Archimedean root selectors, not floating-point isolation.
Certified series grammars / lazy Hahn series	Exact infinite sums, local Taylor values, positive grids	Support validity, effective zero and leading-term access are separate services.
Exp-log and transserial graphs	Infinite exponential scales, asymptotic composition	Partial normalization; the input class of an actual transseries algorithm is narrower than “all transseries”.
Analytic germs and Hahn families	Local functions, finite jets, coherent periods	Domain, support, branch and ordinary coefficient algorithms must all be supplied.
Finitely described Hahn data, possibly relative to a fixed oracle	A countable collection of exact values for each fixed oracle	No universal equality, support validation or finite cutoff materialization.

1.5 Provenance: six supplied manuscripts, one of them in two revisions

This report merges three independently written source articles. They were given different supplied research manuscripts, so the phrase “the supplied manuscript” is ambiguous across the group and is never used in the singular here. The six distinct manuscripts are:

- *Surcomplex Analysis: Infinitesimal Calculus, Hahn-Coherent Holomorphy, and Contour Theory*, in **two revisions**. Revision (2) [1] was supplied to the capability-tiers article; revision (3) [2] was supplied to the other two. We write **SA** when the distinction does not matter and name the revision when it does.
- *Trigonometry on the Surcomplex Plane: Canonical Angles, Infinite Triangles, Infinitesimal Contact, and Hahn-Analytic Oscillation* [3], supplied only to the capability-tiers article. We write **TR**.
- *Surcomplex Polynomial Algebra: Order Geometry, Newton Profiles, and Scale-Resolved Stability* [4], supplied only to the layered-representations article. We write **PA**.
- *Hartogs Coherence, Finite Maps, and Residue Duality over the Surcomplex Numbers* [5], supplied only to the layered-representations article. We write **HC**.
- *Infinitesimal Analytic Geometry over the Surcomplex Numbers: Radius-Free Hahn Germs, a Nullstellensatz, and Conservation of Singular Zeros* [6], supplied only to the Gaussian-rational-prototype article. We write **AG**.

- *Jordan Separation, Cauchy Theory, and Stokes Formulae on the Surcomplex Plane: A Category-Sensitive Contour Calculus* [7], supplied only to the Gaussian-rational-prototype article. We write **CT**.

Every result imported below is attributed to the specific manuscript, and where relevant to the specific revision, that supplies it.

Only one of the three delivered archives bundled its sources: the capability-tiers archive shipped unchanged copies of SA (2) and TR inside its own delivery, so only its provenance was independently inspectable from the delivery itself. No supplied manuscript is redistributed with this report; which manuscript, and which revision, supplies each imported result is recorded throughout the text. Those same two manuscripts were also supplied to two other merged reports in this collection: TR is the canonical-angles-and-oscillation manuscript behind

Trigonometry on the Surcomplex Plane (docs/surcomplex/trigonometry/), and SA (2) is, after CRLF normalization, the infinitesimal-calculus-and-contour-theory manuscript behind *Surcomplex Analysis: Holomorphic Functions on No[i]* (docs/surcomplex/analysis/).

The other two archives cite their manuscripts rather than bundling them.

Two distinct source-gap disclaimers survive from the sources and are both honest limitations. First, two further archives mentioned *inside* TR were not supplied, are not included, are not treated as independently reviewed sources, and their own referenced verification scripts were not rerun. Second, a fourth, global-divisor manuscript cited *inside* PA is not among the three files supplied to that article and is not used here as an independently inspected source.

None of the three original source packages supplied a Lean formalization or pinned proof-assistant dependencies. Their finite symbolic checks, recorded in section 18, do not establish general theorems in Lean. The containing repository now maintains a separate implementation and coverage ledger in docs/FORMALIZATION.md; that ledger, rather than the historical package inventory, determines which source claims have checked counterparts.

2 Objects the software must distinguish

2.1 Normal forms, workspaces, and set-sized parents

Following all three sources and all six supplied manuscripts we write

$$t^\gamma := \omega^{-\gamma}, \quad A = \sum_{\gamma \in S} a_\gamma t^\gamma, \quad S \subseteq \Gamma \text{ well ordered}, \quad (2.1)$$

where the displayed power of ω is the *Conway monomial map* and Γ is a set-sized ordered additive subgroup of **No**. Put $F_\Gamma = \mathbb{R}((t^\Gamma))$ and $K_\Gamma = \mathbb{C}((t^\Gamma))$. For nonzero A ,

$$v(A) = \min \text{supp}(A), \quad \text{lc}(A) = a_{v(A)}, \quad v(0) = +\infty,$$

so $v(AB) = v(A) + v(B)$ and $v(A + B) \geq \min\{v(A), v(B)\}$. For a real series the sign is the sign of the leading coefficient. When Γ is divisible the real Hahn field is real closed and the complex one is algebraically closed; this algebraic-closure input is a theorem about the complete mathematical field, not a terminating algorithm for arbitrary coded series [11, Corollary 4].

This is a semantic workspace, not a finite data structure. Even $\mathbb{Q}((t))$ contains uncountably many arbitrary rational coefficient sequences. A practical implementation replaces $\mathbb{R}$ or $\mathbb{C}$ by a declared coefficient representation, replaces arbitrary exponents by an effective ordered-group representation,

and admits only selected finitely described supports and coefficient rules. Such a replacement preserves some closure properties and loses others; every closure claim must refer to the replacement, not to the surrounding Hahn field. Every collection of coefficients, every support and every family summed in this report is a set: **No** is a proper class, but every individual expression is finite data interpreted in suitable set-sized workspaces. A class-sized theory may justify those parents without ever allocating a class-sized list, ring or support.

2.2 Conway monomials are not generic powers

The symbol ω^a in a Conway normal form denotes the omega-map. It satisfies $\omega^{a+b} = \omega^a \omega^b$, but it is *not* defined by the generic prescription $\text{Exp}_{\mathbf{No}}(a \text{Log}_{\mathbf{No}} \omega)$: the canonical real surreal exponential and logarithm are an additional structure [9, 10]. The distinction matters as soon as the exponent is surreal but not an ordinary real. Concretely,

$\omega^{1/\omega}$ (Conway notation) is *infinite*, whereas

$$\text{Exp}_{\mathbf{No}}\left(\frac{\text{Log}_{\mathbf{No}} \omega}{\omega}\right) = 1 + \text{a positive infinitesimal}.$$

The first has positive exponent, hence an Archimedean class strictly above that of 1; in the second, $\text{Log}_{\mathbf{No}} \omega / \omega$ is infinitesimal, by the growth properties of the surreal exponential. Rational powers of ω cause no such conflict: both constructions give the unique appropriate positive algebraic root. The dangerous step is extending that rational-power simplification to arbitrary surreal exponents.

An interface should therefore distinguish at least three constructors,

$$\text{ConwayMonomial}[a], \quad \text{SurrealPower}[x,y], \quad \text{OrdinalPower}[a,b],$$

where for positive surreal x the second may mean $\text{Exp}_{\mathbf{No}}(y \text{Log}_{\mathbf{No}} x)$ and the third is ordinal exponentiation. Identities relating them must be proved on their particular domains, and the two constructions need an actual compatibility theorem on the chosen argument class rather than an inherited notation. A printed superscript is not sufficient semantic metadata. General complex powers need still more: a chosen logarithm or a specified algebraic-root branch (section 13.7).

2.3 Ordinal arithmetic, field arithmetic, and the host's infinity

Ordinary ordinal addition is not surreal field addition: $1 +_{\text{ord}} \omega = \omega$, whereas $1 + \omega = \omega + 1 > \omega$ in the field. On embedded ordinals, surreal field arithmetic agrees with natural (Hessenberg) arithmetic, not with an unrestricted replacement of ordinal operations by field operations [8]. A package supporting both needs distinct constructors, and birthday or simplicity queries should be a separate capability from field equality.

A genuine infinite field element is also not a projective point at infinity and not the host's **Infinity**:

$$\omega - \omega = 0, \quad \omega / \omega = 1, \quad \omega^{-1} > 0.$$

A head used for a pole or an extended limit cannot supply those field laws; **Infinity** participates in extended-limit arithmetic and is not a named element of a non-Archimedean ordered field [37]. In all three delivered prototypes **Infinity** appears only as the conventional metadata value $v(0) = +\infty$, never as a stored field element.

Nor is a surreal infinitesimal a nilpotent: $t^n \neq 0$ for every ordinary n . See section 2.5.

2.4 Order, valuation, standard part, and the finite part

With ordinary Archimedean coefficients, $v(z) \geq 0$ means z is finite and $v(z) > 0$ means it is infinitesimal. Let $\mathcal{O}$ denote the finite elements and $\mathfrak{m}$ the infinitesimals. Every $z \in \mathcal{O}$ decomposes uniquely as $z = c + \eta$ with c ordinary and $\eta \in \mathfrak{m}$; the standard part $\text{st}(z) = c$ is a ring homomorphism *on this finite subring*. The zero coefficient of an infinite element can also be extracted, but calling it a standard part would misstate the domain of the residue homomorphism.

The *finite part* fin used by TR is a different projection: it removes all negative-exponent terms from any normal form. It is additive but not multiplicative, since

$$\text{fin}(\omega \cdot \omega^{-1}) = 1 \neq \text{fin}(\omega) \text{fin}(\omega^{-1}) = 0.$$

These operations require different names and different rewrite rules. Even when a standard part exists mathematically, extracting it from an arbitrary program-defined series may not be computable.

For a surcomplex $z = x + iy$, conjugation and modulus are $\bar{z} = x - iy$ and $|z| = \sqrt{x^2 + y^2}$. The modulus satisfies the ordinary triangle inequality while the valuation satisfies an ultrametric inequality: an order disk $|z - a| < r$ is not a valuation ball $v(z - a) > v(r)$ [4].

Software must distinguish an exact valuation from a lower bound. If the leading displayed coefficient is not known to be nonzero, the valuation has not been established. Cancellation in a sum is not cosmetic: it can change sign, scale, and the number of roots in a selected ball. The delivered kernels make this concrete; see the cancellation trap in [section 5.3](#).

2.5 Four incompatible uses of “infinitesimal”

A nonzero surreal infinitesimal has an inverse and is not nilpotent. A dual number $a + b\epsilon$ with $\epsilon^2 = 0$ belongs to a different algebra. A truncation modulo t^N belongs to a quotient ring with nilpotents. These rings are useful for jets and automatic differentiation, but neither is a subfield of the surreals: storing a surreal t in a dual-number object and applying its multiplication law asserts the false equality $t^2 = 0$.

An *asymptotic variable tending to zero* is a fourth kind of object. It is a variable in a function or germ, not yet a particular surreal constant. Converting its formal expansion into an exact Hahn value requires a selected interpretation and a summability proof. We retain all four distinctions even where one displayed expression is convenient for all four purposes.

2.6 A type is more than a coefficient container

An exact object should carry or reference its coefficient domain, its exponent domain and order, its embedding into the surreal numbers, its analytic interpretation, and any branch or derivation convention. A formal generator called s is not enough: $s = t^2$ and $s \prec t^n$ for every ordinary $n > 0$ give different fields and different equality tests. Domain identifiers should be structural and serializable rather than dependent on session-local assumptions that can later change. Ordinary substitution can destroy the intended relationships: replacing $u = t^\omega$ by t^{100} collapses a rank, and **a different variable order is a different parent even when the printed rational expression is identical**. That last point is not hypothetical here — two of the three delivered prototypes use opposite coordinate significance conventions ([section 17.4.4](#)).

3 Limits of finite representation and universal simplification

3.1 There is no finite notation for every surreal

Proposition 3.1 (The finite-description bound). *For any fixed finite or countable alphabet, each symbol itself specified by finite data, a language of finite strings can denote at most countably many individually specified numbers. It cannot denote every real number, and therefore cannot denote every surreal number.*

Proof. There are countably many strings of each finite length and countably many lengths, so countably many strings in all. The image of this countable set under a partial denotation map cannot have larger cardinality. The reals are uncountable and are contained in **No**. $\square$

All three sources prove or state this independently, and draw the same three consequences, which we record as remarks rather than as three propositions.

Remark 3.2 (Oracles move the information, they do not encode it). A symbol for an arbitrary unspecified real parameter gives a symbolic family, not a finite encoding of every possible value of that parameter. An oracle or an opaque atom can stand for an arbitrary real or surreal, but the missing information has then been moved outside the finite input rather than encoded in ordinary finite memory; the resulting object is exact *relative to the oracle*. A symbol c carrying assumptions names a parameter or a family until a particular value is specified.

Remark 3.3 (Set-sized support is not a computability statement). “Every normal form has set-sized support” is a statement about cardinality and order type, not about computation. A set may be uncountable, noncomputable, or lack any finite structural description. Likewise, a proof assistant can quantify over all surreals without making them machine-representable data, and a computer algebra system can state universal identities in abstract variables without materializing their values.

Remark 3.4 (No largest representable subset). There is no uniquely largest useful finitely representable subset independent of a chosen language. Adding one new constant, one new support notation, or one new certified constructor may strictly extend an existing language, and each enlargement changes the available decision procedures. Even two representations of the *same* mathematical subset can have very different computational properties — which is exactly the content of [theorem 3.5](#). The practical question is which representations offer a useful and honest collection of operations.

3.2 The zero-test obstruction: one theorem, two reductions

Theorem 3.5 (Total coefficient access does not decide zero). *There is no algorithm which, for every total program specifying rational coefficients a_n of a valid Hahn series $A = \sum_{n \geq 0} a_n t^n$, decides whether $A = 0$. Consequently no universal terminating equality or three-way sign procedure exists for this unrestricted representation class.*

Two of the three sources reduce from halting in one way and the third in another. The two reductions have different punchlines, both load-bearing, so we print both.

Reduction A: values in a decidable rational field. For a machine M and its input, let $a_n = 1$ if M has halted *within* n steps, and $a_n = 0$ otherwise. Computing a_n needs only bounded simulation, so the sequence is total computable, and its support lies in $\mathbb{N}$, making the displayed series a valid Hahn sum at $t = \omega^{-1}$. It is zero exactly when M never halts. A total zero test would therefore decide

halting. All coefficients are nonnegative, so a sign procedure distinguishing zero from positive would do the same; equality reduces to zero testing by subtraction. $\square$

Remark 3.6 (Moral of Reduction A). In this reduction every individual value is either 0 or the rational function $t^N/(1-t)$ for some halting time N . Thus even values lying in a *decidable rational field* become undecidable when presented by unrestricted coefficient programs. *Representation, not merely membership in an abstract field, determines the algorithmic guarantee.* The whole capability-contract thesis of this report rests on that sentence.

Reduction B: a promise of at most one nonzero coefficient. For a machine M and its input, let $b_M(n) = 1$ if M first halts at step n (for $n \geq 1$) and $b_M(n) = 0$ otherwise; this is computable by simulating exactly n steps. Then $a_M = \sum_{n \geq 1} b_M(n)t^n$ has support either empty or a singleton, and is zero precisely when M never halts. $\square$

Remark 3.7 (Moral of Reduction B). Here every input in the reduction actually denotes either zero or a single monomial — indeed a polynomial. The obstruction therefore survives even when *every represented series is promised to have at most one nonzero coefficient*. This is the sharper form of the same point: an explicit finite polynomial list is different information from a program that might reveal one nonzero coefficient arbitrarily late. There is no tension with [theorem 5.1](#): opaque programs promised merely to denote finite sparse polynomials need not have a computable support bound, and can describe exactly the same set of values with a much weaker computational contract.

Neither reduction is “the” proof; both survive.

Corollary 3.8 (No universal three-way sign; equality reduces to zero testing). *There is no universal terminating three-way sign procedure for this representation class, and equality of two such presentations reduces to zero testing of their difference.*

Corollary 3.9 (No universal leading-term algorithm). *A leading-term algorithm that always either reports zero or returns the least nonzero exponent would also decide zero. No such algorithm exists for arbitrary total coefficient programs.*

All three sources insist independently on the same operational consequence. **Returning False after a fixed unsuccessful search is unsound.** The correct result is an explicit unknown status or a resource-limited inconclusive result. A lazy series implementation remains useful, but must admit nontermination, resource-bounded **Unknown**, or restricted input grammars carrying extra zero certificates. Numerical sampling does not repair the problem. Sage’s lazy-series documentation makes the related practical distinction between coefficient generation and equality decidable only in special cases [\[39\]](#).

The obstruction requires neither arbitrary real coefficients nor complicated surreal exponents: a certified support container $\mathbb{N}$ is available from the outset, and the difficulty is locating the first nonzero coefficient or proving none exists. It also does not prevent a zero test for rational generating functions, algebraic series with finite defining data, or other specifically certified families ([section 3.5](#)). The promise must be attached to the representation class rather than to the word “exact”.

3.3 Support admissibility is not a universal automatic test

Proposition 3.10 (Unrestricted support validation is undecidable). *No algorithm decides Hahn admissibility for every program which outputs a candidate support using total bounded-time coefficient tests.*

Proof. For a machine M , assign at exponent $-n$ (for $n \geq 1$) the coefficient 1 if M has halted by step n and 0 otherwise. Every individual test terminates. If M never halts the exact support is empty and admissible. If M halts, the support contains an infinite strictly decreasing tail $-n, -(n+1), \dots$ and is not well ordered. Deciding admissibility would decide halting. $\square$

The remedy is not to omit validation. It is to use constructors whose support properties follow from their syntax or from checked certificates, and to treat unrestricted user generators as untrusted until obligations are supplied. A validator can be total and complete for a restricted grammar; it cannot be both for all programs of the preceding proposition.

3.4 Known nonzero does not make the leading term computable

Proposition 3.11 (A high-rank leading-term and sign obstruction). *For total program-described two-scale grid series, even a promise that the value is nonzero does not give a universal algorithm for its exact valuation or its sign.*

Proof. Use exponents in $\mathbb{Q}_{\text{lex}}^2$ and let a_n be the first-halt coefficients $b_M(n)$ of Reduction B, with halting steps indexed from 1; their support is empty or a singleton, which is what makes the object below finite. (Reduction A's coefficients are cumulative — $a_n = 1$ once M has halted *within* n steps — so they would give infinitely many terms. The valuation and sign conclusions would survive, but the count below would not.) Put

$$B_M = t^{(1,0)} - \sum_{n \geq 1} a_n t^{(0,n)}.$$

This is *always nonzero* and actually has one or two terms. If M never halts it is positive with valuation $(1, 0)$; if M halts at step n it is negative with valuation $(0, n)$. Either a sign algorithm or an exact-valuation algorithm would decide halting. $\square$

Remark 3.12 (The rank-one contrast, which must not be generalized). There is a genuine contrast with a one-variable power series carrying a known integer lower exponent bound. Under a nonzero promise, sequentially computing its coefficients eventually finds the first nonzero one, because only finitely many exponents precede any candidate. In higher rank, infinitely many potential smaller exponents can precede a known nonzero coefficient. An implementation must not generalize the rank-one search argument without checking this.

3.5 Positive counterweights

The negative results above delimit *unrestricted* representations. Rich structured languages escape them, and three positive routes should be recorded explicitly so that the picture is not read as uniformly negative.

First, rational and algebraic representations already avoid the obstruction: equality in the effective core is decided by the zero test on a single polynomial ([theorem 5.1](#)), and algebraic elements with finite defining data and valid root selectors have decidable equality and order ([theorem 5.3](#)).

Second, for an ordinary-point solution of a scalar linear differential equation with analytic coefficients and nonvanishing highest-order coefficient, finitely many initial derivatives determine the solution. With effective coefficient arithmetic and the equation recorded, vanishing of the requisite initial data is a finite zero certificate. Within a representation that can construct an annihilating

equation for a difference, identity reduces to a finite initial-data test. Products and derivatives are handled in the usual D-finite setting; reciprocals and arbitrary composition need not remain D-finite. Singular points require a certified recurrence start and treatment of exceptional indices: the phrase “satisfies a differential equation” alone does not guarantee uniqueness. Wolfram’s `DifferentialRoot` is a relevant ordinary-function precedent, not an automatic Hahn-series implementation [32].

Third, and most significant, is a published positive decidability result beyond the algebraic core. Chen, Fang and van der Hoeven’s *A Zero-Test for D-Algebraic Transseries* appeared in the ISSAC 2026 proceedings, pp. 105–113, published July 12, 2026 [20]. Their framework uses effective differential ambient fields and structured grid-based transseries; their Theorem 18 extends the ambient field, and its zero-test, by a distinguished solution of an appropriate quasi-linear differential equation, with Ritt reduction and separation bounds central to the argument.

Remark 3.13 (Scope of the 2026 zero-test). This is a concrete route beyond elementary expressions for structured differential-algebraic extensions. It is **not** a zero-test for arbitrary series-producing programs. A CAS implementing it must preserve its effective-field, transbasis and solution-selection assumptions rather than replacing them with an unrestricted `DifferentialRoot` label.

3.6 Richardson’s undecidability and its restrictions

Richardson supplies further undecidability results for sufficiently rich expression languages of real functions [13]. The restrictions of that particular theorem matter, and it must not be inflated into a blanket claim that every restricted exponential-constant identity problem is known to be undecidable. The elementary coefficient-program reductions of section 3.2 are enough for the unrestricted Hahn-generator interface considered here, and they are independent of difficult identities among elementary constants.

4 A taxonomy of exact representations

4.1 Cuts, sign expansions, and the simplest-number semantics

The original cut presentation and the sign-sequence presentation retain simplicity and birthday information. Hereditarily finite legal cut trees represent exactly the numbers with finite sign expansions, namely the dyadic rationals. Different cut trees can represent the same dyadic value. They are excellent for teaching the recursive construction and for checking small identities, but a poor universal arithmetic storage format: $1/3$ has a short rational expression and is not a finite-birthday surreal, so division immediately leaves this representation.

A legal number cut has every left option smaller than every right option, and its value is the *simplest* number between the options, not an arbitrary point in an interval. For instance $\{0 \mid 4\} = 1$, not the midpoint 2. A cut constructor therefore cannot be implemented by choosing a convenient solution of the inequalities [9].

Compressed infinite cuts and ordinal-indexed sign blocks can be exact — the familiar description of ω by all natural numbers on the left is the standard example — and can name real numbers with effective digit descriptions and much more. Their effectiveness depends on the language used for the options, not on the braces. The compression language must say how a transfinite block is indexed, how its legality is certified, and which comparisons terminate; an ordinal notation appearing in an index is not a universal algorithm for all ordinal or surreal questions. Computing the simplest number between arbitrary program-defined classes of options is not a generic finite operation. A cut

constructor should record an ordinal or well-foundedness certificate and an inequality certificate rather than accept an arbitrary predicate and promise a canonical normal form.

Neither birthday nor simplicity is determined by the field structure alone. A Hahn-field backend that knows addition and order should not pretend to supply shortest sign expansions or the simplest number between arbitrary cuts. Birthdays and simplicity comparisons belong in an optional capability, not among the prerequisites for ordinary field arithmetic.

4.2 Finite normal forms

A sparse finite sum

$$A = c_1 t^{\gamma_1} + \cdots + c_m t^{\gamma_m}, \quad \gamma_1 < \cdots < \gamma_m,$$

is a good exact representation once coefficient and exponent equality are decidable. Addition merges supports; multiplication forms pairwise exponent sums and collects equal terms; conjugation acts coefficientwise; for real coefficients, ordering is a leading-term calculation. Recursive finite normal forms for the exponents extend the hierarchy: a coefficient/exponent tree can describe nested scales without materializing a sign sequence.

This representation is generally a ring, not a field: $(1 - t)^{-1} = \sum_{n \geq 0} t^n$ is not a finite-support normal form. An implementation must either move to rational-expression storage, create a lazy infinite sum with a support certificate, or return an explicit unsupported-domain result. Declaring the inverse to have a finite normal form would be incorrect. If the coefficient zero test is only partial, a sparse map may contain unresolved terms and cannot honestly advertise its first stored key as the valuation.

4.3 Rational expressions

Fractions of finite monomial polynomials are especially effective: their arithmetic can often be performed with no series expansion at all. If numerator and denominator are sparse polynomials in independent monomial generators, equality is decided by cross multiplication, and the result is an exact field element even when no finite normal form exists. A common-denominator normal form and polynomial gcd cancellation can improve performance but are not necessary for the correctness of that equality test. This is the representation the effective core of [section 5](#) formalizes and all three delivered prototypes implement, in three different specializations.

4.4 Algebraic specifications and branch selectors

An algebraic object should contain a polynomial over the declared coefficient field together with enough data to select one root. For nonzero P , a bare equation $P(z) = 0$ denotes a finite root set, not one selected number; the zero polynomial imposes no condition. Valid selectors include an ordered real root index, a Thom encoding, a certified simple residual root after a scale change, a certified infinitesimal expansion distinguishing a branch, or a tower of ordered algebraic extensions. A valuation ball containing a multiple cluster is not a unique-root selector, and a numeric approximation to an ordinary real root is not a sufficient selector for roots separated only by infinitesimals. An arbitrary integer root index does not inherit the meaning of an ordinary complex root ordering at non-Archimedean coefficients [\[23\]](#).

An exact algebraic number need not have a finite radical expression, and the same is true over surreal coefficient fields. Implicit root objects are therefore a central representation, not an

unfortunate fallback: they give finite descriptions of infinitely supported roots and algebraic closure of a well-chosen effective core, at the cost of root-branch management and potentially large algebraic degrees. Thus $\sqrt{1+t}$ is most naturally stored as the positive root of $X^2 - 1 - t$, with its binomial expansion an auxiliary view rather than its definition. The output of an algebraic operation need not belong to the input's finite-support grammar, yet can still have a finite exact implicit representation.

For a simple binomial root the construction is explicit. If $a = ct^\gamma(1 + \epsilon)$ with $v(\epsilon) > 0$, a chosen n th root is

$$c^{1/n} t^{\gamma/n} \sum_{j \geq 0} \binom{1/n}{j} \epsilon^j, \quad (4.1)$$

which requires a coefficient-root choice and the exponent γ/n ; the series is strongly summable, and multiplication by the n th roots of unity supplies the other roots. Formula (4.1) is an exact lazy description, usually not a finite normal form. Adjoining γ/n may enlarge the exponent group and adjoining $c^{1/n}$ may enlarge the coefficient field; neither enlargement should be concealed. For complex a a branch selector remains necessary even though algebraic closedness supplies all roots, and formulas such as $(ab)^{1/q} = a^{1/q}b^{1/q}$ must not be used with unrelated complex root choices.

4.5 Puiseux data and ramification

In rank one and characteristic zero, algebraic branches can be described by Puiseux expansions after ramification: the union of the fields $k((t^{1/n}))$ over an algebraically closed coefficient field is algebraically closed [11, Corollary 7]. For polynomial equations over an effective rational-function base, Newton–Puiseux expansion with residual factorization and branch selection is an appropriate implementation strategy.

The *field of all Puiseux series* still permits arbitrary infinite coefficient sequences, including noncomputable ones; its algebraic closedness does not encode those sequences, and it should not be identified with the finitely described algebraic functions. Finite defining equations plus branch data give an effective class; an arbitrary coefficient stream leads back to [theorem 3.5](#). Higher-rank algebraic algorithms should use nested valued fields, valuation extensions and explicit selected-root objects rather than flattening every scale into one real exponent. Merely passing a custom Hahn head into a general-purpose `Root` or `Solve` call does not install those algorithms.

4.6 Grids, recursive towers, and unrestricted Hahn data

We use the more general of the two grid definitions given by the sources.

Definition 4.1 (Effective positive grid). Fix vectors $\gamma_0, e_1, \dots, e_m \in \mathbb{Q}^d$, each e_j lexicographically positive. A grid presentation has support contained in

$$\gamma_0 + \mathbb{N}e_1 + \dots + \mathbb{N}e_m, \quad (4.2)$$

and supplies its coefficients either by a direct exponent procedure or by a coefficient procedure on the multi-indices, with collisions added. **A finite union of such presentations is permitted.**

Two of the three sources use only the single-translate form; the finite-union form is strictly more general and costs nothing. The coefficient rule on the grid is still part of the input. Finite grids are useful because the support condition follows from positivity and the Hahn–Neumann lemma, and because coefficient extraction can be made effective under explicit assumptions ([theorem 6.1](#)). Different tuples may represent the same exponent, so coefficients must be combined by finite

convolution rather than treated as distinct monomials. Grid-based methods have a substantial existing computational theory [14] and should not be presented as newly invented. They do not make every coefficient sequence decidable, nor do they imply that every valuation cutoff contains only finitely many terms (section 7.3).

Recursive exponent towers can represent finitely many nested Conway monomials whose exponents themselves have finite normal forms; a well-founded finite dependency graph makes arithmetic and comparison recursive. A monomial with exponent ω is finitely describable even though it is not in a rank-one rational-exponent lattice. Such a language captures far more than rational powers, but every extension must specify normalization of exponents, cross-level coercions, and which operations preserve the syntax. A recursive exponent backend should export its own equality, order and normalization procedures with a well-founded construction rank; an arbitrary reference back to the same object is not a legitimate normal form. A general expression in `SurrealExp`, `SurrealLog` and `ConwayMonomial` need not possess a known complete normalizer merely because its expression graph is finite.

General Hahn supports can be well ordered without lying in a finite grid. Even a countable support of order type $\omega + 1$ cannot be traversed completely by a monotonically increasing ordinary stream: after its first ω terms, no finite step reaches the final term. Such supports require nested blocks, symbolic support sets, indexed access, or exact structural expressions. “Lazy” must not be used as a synonym for “an ordinary iterator suffices”. A useful support language should include finite supports, arithmetic progressions, positive finitely generated grids, nested lexicographic blocks and selected well-ordered families with supplied dependency routines; more exotic computable ordinal-indexed supports may be added as optional extensions.

At the broadest level one can store an abstract Hahn datum with a proof of well-ordered support and a coefficient specification. This supports theorem statements and some symbolic transformations. It should advertise only the computational capabilities actually supplied by its certificates, rather than pretending that a proof of existence is an enumeration algorithm.

4.7 Expression graphs, transseries, and hyperseries

A directed acyclic graph built from exact constants, field operations, algebraic roots, selected exponentials and logarithms, and certified sums is a flexible denotational language. Its finite size does not imply a finite normal form. Common-subexpression sharing is particularly valuable for algebraic towers and repeated local expansions. Descriptions using a rational generating function, a total coefficient recurrence with specified initial data, or a formal implicit equation with a uniqueness certificate are likewise finite and exact; these mechanisms have different equality algorithms, and arbitrary coefficient programs have none.

Transseries offer a structured enlargement for exponential and logarithmic scales and for composition at infinity, with a substantial algebraic and differential apparatus [14]; the 2026 D-algebraic zero test of theorem 3.13 is an important additional algorithmic direction [20]. Published work interprets LE-, EL- and omega-series as functions on positive infinite surreals, with composition and differentiation in specified classes [16]. For implementation, select finite or effectively generated fragments with explicit coefficient and support algorithms; the full mathematical classes are not all finitely described.

The hyperseries representation theorem of Bagayoko and van der Hoeven provides a much broader mathematical correspondence between surreal values and growth-rate expressions [18]. It is cited here for the mathematical result only. It is not a finite-string encoding of every surreal; the

countability argument of [theorem 3.1](#) still applies, and a representation theorem with infinitary data does not by itself supply normalization, equality, or a storage format.

4.8 A representation and closure map

Representation	Exact values included	Main closure or decision boundary
Finite cut trees / sign data	Dyadic rationals; finite-birthday and simplicity data	Not closed under division; simplicity retained but not supplied by a Hahn backend.
Compressed cuts and ordinal-indexed blocks	ω , effectively described reals, selected transfinite objects	Effectiveness lies in the option language; simplest-number queries are not generic.
Finite sparse normal forms	Finite sums of admitted monomials	Closed under addition and multiplication; inverse usually needs another representation.
Monomial rational functions	Fractions of those sums	Exact field algebra and equality with effective independent generators; not algebraically closed.
Effective real closure and its complexification	All algebraic elements over the chosen ordered monomial field	Real order and algebraic equality decidable; complexity may be severe.
Rational or algebraic series	Infinite supports defined by finite algebraic data	Strong exact structure; root selection remains necessary.
Certified recurrence or D-finite series	Many special-function expansions, including divergent formal series	Coefficients effective with adequate initial data; closure and identity are representation-specific.
Certified positive grids (finite unions)	Multiscale series with finite generators	Fixed coefficients can be effective; a bounded valuation prefix can still be infinite.
Selected transserial syntax	Exponential and logarithmic scale hierarchies	Requires a specified ordered differential and composition framework.
Analytic germs and Hahn families	Local functions, finite jets, coherent periods	Domain, support, branch and ordinary coefficient algorithms must be supplied.
General program- or proof-described Hahn data	A broad countable collection of exact values	No universal equality, support validation or truncation algorithm.

5 The effective core

5.1 One core theorem at the generality all three sources support

All three source articles prove the same effective-core theorem, over three different coefficient fields and with three different treatments of rational exponents and of the real closure. Those are differences of scope, not of mathematics. We state the theorem once at the generality all three support, and then print the three specializations.

Theorem 5.1 (Effective rational monomial core). *Let k be an effective field, with an effective order when real comparisons are requested. Let Γ be an effectively ordered additive group and let $X_1, \dots, X_d$ be monomials whose exponents $\beta_1, \dots, \beta_d \in \Gamma$ are independent over $\mathbb{Z}$ (no nonzero integer exponent vector sums to zero). Then $k(X_1, \dots, X_d)$ embeds in $k((t^\Gamma))$, and finite rational-function descriptions give an effective field: arithmetic terminates, and equality of P/Q and P'/Q' is decided by the zero*

test on the single polynomial $PQ' - P'Q$. For nonzero $f = P/Q$, writing $\alpha(P)$ for the least exponent of P in the induced order and $c(P)$ for that coefficient,

$$v(f) = \alpha(P) - \alpha(Q), \quad \text{lc}(f) = \frac{c(P)}{c(Q)}, \quad \text{sgn}(f) = \text{sgn}(c(P)c(Q)), \quad (5.1)$$

the last when k is ordered, where sgn is the sign in k . On the Hahn valuation ring $v(f) \geq 0$, the residue in k is 0 if $v(f) > 0$ and $\text{lc}(f)$ if $v(f) = 0$. If, in addition, k is embedded in the ordinary reals or complexes and the exponent group is interpreted by Conway monomials, this residue is the ordinary standard part: $v(f) < 0$ then means f is infinite and has no finite standard part. For arbitrary k the residue assertion makes no claim about ordinary finiteness. The zero case has residue and standard part 0 when applicable, with $v(0) = +\infty$ as a distinguished marker.

Proof. Distinct integer exponent vectors give distinct values of $\sum n_j \beta_j$ by integer independence, so a nonzero polynomial in the X_j maps to a nonempty finite normal form and cannot vanish; the generators are algebraically independent over k and the embedding extends to fractions. Concretely, write a nonzero denominator as $Q = ct^\beta(1 + \epsilon)$ with all exponents of the finite ϵ strictly positive; the strong geometric series $\sum_{n \geq 0} (-\epsilon)^n$ is a Hahn inverse, justified coefficientwise by the positive-support lemma of [section 6.1](#).

Polynomial arithmetic over k is effective because coefficient arithmetic and equality are. Equality of fractions reduces to the zero test on $PQ' - P'Q$, a finite coefficient computation. The leading term of a product cannot cancel because the least exponent of each factor is unique, which gives (5.1); the Hahn ordering gives the sign statement. For $v(f) \geq 0$, extracting the zero-exponent coefficient is the residue map: it vanishes at positive valuation and is the leading coefficient at valuation zero. With ordinary coefficients the positive-valuation tail is infinitesimal, which gives the stated standard-part interpretation. No infinite expansion is required by any of these algorithms. $\square$

Polynomial gcd reduction, when supplied by the coefficient backend, yields reduced fractions, and normalizing the denominator's leading coefficient to one removes scalar ambiguity; neither is required for the equality test.

5.2 The three specializations, and which kernel implements which

Source article	Coefficient field	Exponent group and generators	Coordinate significance	Rational powers inside the theorem?	Real closure proved?
Capability tiers (02)	$\mathbb{R}_{\text{alg}}$ in the theorem; $\mathbb{Q}$ in its kernel	$\mathbb{Z}$ -spans of $\gamma_j = \omega^{j-1}$, i.e. $t_j = \omega^{-\omega^{j-1}}$	Reversed: highest-index coordinate compared first	No	Yes, separately, as R_d with $C_d = R_d[i]$
Layered representations (04)	$\mathbb{R}_{\text{alg}}$	$\mathbb{Q}_{\text{lex}}^r$, embedded by $(q_1, \dots, q_r) \mapsto q_1 \omega^{r-1} + \dots + q_r$	First coordinate most significant	Yes , by common-denominator lattice refinement, inside the same theorem	Yes, as F_r with $C_r = F_r[i]$ algebraically closed

Source article	Coefficient field	Exponent group and generators	Coordinate significance	Rational powers inside the theorem?	Real closure proved?
Gaussian prototype (05)	An arbitrary effective field k	$\mathbb{Z}_{\text{lex}}^d$, standard basis	First coordinate most significant	No; a remark on lattice refinement only	Discussed as a design target, not proved

Two strengthenings survive from this table and must not be flattened. The *rational-power extension* belongs to the layered-representations article: choose a common positive denominator in each coordinate for all exponents appearing in the finitely many operands, replace T_j by $U_j^{N_j}$ where $U_j = t^{\beta_j/N_j}$, and the same proof applies to the algebraically independent U_j . The *arbitrary- k generality* belongs to the Gaussian-prototype article, and is what licenses the statement of [theorem 5.1](#) above. The *reversed coordinate convention* belongs to the capability-tiers article, and is the convention its kernel actually matches.

The concrete embedding shared by the second and third specializations sends $(q_1, \dots, q_r) \mapsto q_1\omega^{r-1} + \dots + q_r$, so that every earlier generator is smaller than every positive integer power of a later one; for $r = 2$ the generators are $\omega^{-\omega}$ and ω^{-1} . The first specialization's generators are $\gamma_j = \omega^{j-1}$, giving $v(t_1) = 1$, $v(t_2) = \omega$, $\dots$, so that $0 < t_{j+1} < t_j^n$ for every ordinary positive integer n , with the exponent vector $(q_1, \dots, q_d)$ ordered by its *highest-index* differing coordinate first.

Remark 5.2 (State the convention; never infer it from names). The order convention is not inferred from names such as `eps1` and `eps2`; it belongs to the domain descriptor. Different conventions give different ordered embeddings even when the same polynomial syntax is used. This report therefore fixes one global rule — *lexicographic order on a finitely generated ordered exponent group, with coordinate significance declared as part of the domain descriptor* — and then states each kernel's own convention in that kernel's own subsection, quoting each kernel's outputs in its own convention and never rewriting them into a common form ([section 17.4.4](#)).

5.3 The cancellation trap

Cancellation before leading-term extraction is essential. In the capability-tiers convention, the numerator

$$(1 + t_1)t_2 - t_2$$

has leading exponent $(1, 1)$, *not* $(0, 1)$. A heuristic inspection of the uncollected expression is therefore wrong. This is not an academic point: the shipped kernel of that article carries a check for exactly this case, and re-executing it for this report returns the exponent vector $\{1, 1\}$ ([section 18](#)).

5.4 Effective real closure and complexification

Let E denote an effective ordered rational-monomial field as in [theorem 5.1](#), with an effective coefficient subfield of the ordinary reals and an ordered additive embedding of its exponent group in **No**. Interpret its monomials by that embedding. Let F be its relative real closure realized inside **No** — the elements of **No** algebraic over E , which is a real closure because **No** is real closed — and put $C = F[i]$.

Theorem 5.3 (An implementable real-closed and algebraically closed layer). *F admits finite effective algebraic descriptions with decidable real order and equality, and C admits finite effective descriptions with decidable equality and field operations. Every nonconstant univariate polynomial over C splits into linear factors in C . Semialgebraic sentences over these effective coefficient fields can be decided, and finite polynomial systems can be treated by effective algebraic methods with the output representation appropriate to their dimension. No Hahn expansion of each algebraic element is required for these guarantees.*

Algorithmic justification. Represent a real algebraic element by a finite polynomial equation over E together with a unique ordered-root selector, for example a realizable Thom encoding: the signs of successive derivatives at the selected real root. A finite tuple of such descriptions can be handled in one real closure, and a finite tower is an alternative to reducing every operation immediately to a single primitive element. Addition, multiplication and inversion introduce finite systems expressing the operation together with the selectors of its arguments; elimination produces equations for the result and sign determination chooses the intended real root. Equality and order are likewise finite formulas in the defining equations and selectors. Real-closed-field quantifier elimination reduces every such formula to finitely many polynomial sign tests over E , and those terminate by [theorem 5.1](#). Purely algebraic quantifier-elimination and sign-determination algorithms based on Thom encodings are available independently of Archimedean topology and are a published foundation for this step [\[12\]](#).

Pairs of real elements represent C , with complex equality the equality of both coordinates. Since F is real closed, $F[i]$ is algebraically closed; consequently every nonzero polynomial over it factors there and no additional roots appear in a larger field extension. For a nonzero polynomial, root specifications can also be obtained by writing its equation in real and imaginary coordinates and selecting among its finitely many solutions. A nonzero constant has no roots; the zero polynomial vanishes on all of C and is represented by that whole solution set, not a finite root list. For a polynomial system the same real-algebraic encoding decides existential solvability and supplies descriptions of its solution conditions; a positive-dimensional solution set is represented by equations or a suitable parametric or semialgebraic description, not by a finite list of points. $\square$

Remark 5.4 (Implementability, not complexity). [theorem 5.3](#) establishes *implementability*, not acceptable complexity in every case. Elimination can be expensive and degree growth severe. In practice, maintain extension towers and specialized univariate routines and use general elimination as a completeness fallback. Quantifier elimination for a mathematical field and executable elimination over a chosen input encoding are separate assertions; an arbitrary surreal constant represented only by a symbol does not supply the comparison oracle these algorithms require. Nor does the theorem assert that a host's native `Root` or `Reduce` already accepts an arbitrary custom surreal head as its coefficient domain: an implementation must supply the reductions, the ordered-field adapter, and root-description management.

5.5 The non-density trap: why ordinary interval isolation is insufficient

A subtle implementation trap is to assume that the ground ordered field is dense in its real closure. It need not be, and all three sources give a witness — two infinitesimal and one infinite.

- In $E = \mathbb{Q}(t)$ with $t > 0$ infinitesimal there is no $q \in E$ with $\sqrt{t} < q < 2\sqrt{t}$. Any positive q has leading form ct^m with $m \in \mathbb{Z}$ and $c > 0$; if $m \leq 0$ it exceeds both endpoints, and if $m \geq 1$ it is

below both. Rational-function endpoints therefore cannot separate the roots $\sqrt{t}$ and $2\sqrt{t}$ by a point between them.

- In $F = \mathbb{R}_{\text{alg}}(t)$ with $t > 0$ infinitesimal, $\alpha = \sqrt{t}$ and $\beta = \sqrt{t} + t$ are distinct, yet no element of F lies strictly between them, by the same integer-valuation argument. Root isolation by refining endpoints inside F cannot separate this pair.
- If T is positive *infinite*, no element of $\mathbb{Q}(T)$ lies between $\sqrt{T}$ and $\sqrt{T} + 1$: a nonzero rational function of T has an integer leading power, whereas every point of that interval has leading power $T^{1/2}$.

The obstruction is the same in all three cases and is an algorithm-design obstruction, not a failure of ordered algebraic root representation. Real-root indices, Thom encodings, algebraic endpoints, or enlargement of the monomial field by fractional powers all resolve it; naively porting an interval-bisection algorithm that assumes rational density does not give a complete method. In particular, *increasing the decimal precision of a real number cannot distinguish two roots with identical real standard part and infinitesimal separation*, at any precision.

5.6 What the core deliberately omits

The countable union of finite-rank algebraic cores is useful but is not the entire surreal field. The core admits ω , ω^{-1} , $\omega^{-\omega}$, their rational powers and rational combinations at *finitely many* hierarchy levels; it does not contain arbitrary surreal exponents such as every possible infinite normal form. Algebraic closure does not imply exponential closure, support closure under every countable sum, or access to birthdays. Even an exactly named transcendental such as π is not in the minimal algebraic coefficient domain; it can be added through a larger coefficient backend, but that backend must declare what it can decide about the new constants. Higher finite rank is added by an explicit order-preserving embedding of the old coordinates into the new domain — in the first-coordinate-most-significant convention, passing from rank r to rank $r + 1$ while preserving the old scales means prepending a zero coordinate.

A symbolic parameter may instead remain algebraically independent over the base. This permits rational-function algebra, but is not an assertion that its intended real value is independent of other named constants. The difference between an indeterminate and a particular named number must survive serialization, substitution and simplification.

5.7 What transfer does and does not buy

The ordered-field laws and the real-closed algebraic theorems transfer to this core, so finite matrix identities, polynomial identities and semialgebraic geometry can be reused with a coefficient-domain adapter. The statement that t is smaller than $1/n$ for every *ordinary integer* n is not a single ordinary ordered-field formula defining an infinitesimal in $\mathbb{R}$, and passing a few inequalities to a real solver is not an implementation of that condition.

What should therefore **not** be done is to ask a native real-domain **Reduce** to find a positive real t smaller than $1/n$ for every positive standard integer n . No such real exists. Replacing the condition by finitely many inequalities only specifies a small ordinary parameter, not an infinitesimal. The mathematical transfer principle concerns finite first-order field formulas in an appropriate real-closed model; it does not turn unrestricted quantification over the integers into a field-language assumption [4, 12]. Likewise, an elementary-extension theorem for a surreal exponential structure is a semantic

theorem, not a universal decision algorithm for exponential expressions [10]; a CAS must not infer that its ordinary real simplifier now knows every surreal exponential identity.

One route that does work: perform real-closed-field quantifier elimination with symbolic parameters first, then evaluate the resulting finitely many sign conditions using the custom ordered-field backend.

6 Certified supports and finite dependency

6.1 The two conditions, and four standard non-examples

A family $(A_j)_{j \in J}$ of Hahn series is *strongly summable* when the union of supports is well ordered and every exponent receives contributions from only finitely many members; the sum is then formed by finite coefficient addition at each exponent. All six supplied manuscripts use exactly this meaning. Both conditions are necessary, and the sources give the same four standard non-examples, with $t = \omega^{-1}$:

$$\begin{array}{ll} \sum_{n \geq 0} t^n = \frac{1}{1-t} \quad (\text{valid}), & \sum_{n \geq 0} n! t^n \quad \text{is a valid Hahn sum,} \\ \sum_{n \geq 0} t^{-n} \quad \text{is not a Hahn sum,} & \sum_{n \geq 0} \frac{1}{n!} \quad \text{is not a strongly summable family.} \end{array}$$

The third support decreases indefinitely. In the fourth family infinitely many summands use exponent zero; its ordinary real sum is of course e , and a CAS may evaluate that *ordinary coefficient sum* first and embed the result — a different operation, not a relaxation of finite coefficient multiplicity. The same remark applies to $\sum_{n \geq 0} 2^{-n} = 2$. Two further standard failures: $\{1/n : n \geq 1\}$ is a set of positive exponents that is not well ordered, so positivity of every summand’s valuation is insufficient; and $\sum_{n \geq 1} t^{1/n}$ must be rejected as an unordered-support Hahn sum. The CAS must always specify which summation operation is being used.

For well-ordered supports S, T , the support of a product lies in $S + T$ with finitely many contributing pairs at each exponent. If E is well ordered and strictly positive, the monoid E^* of finite sums from E is well ordered and each exponent has finitely many word decompositions up to the usual finite permutations. For a source support S , expressions indexed by one member of S and a finite word in E have support in $S + E^*$, with finitely many contributions at each exponent. This Hahn–Neumann calculus justifies reciprocal series, infinitesimal Taylor substitution and compatible coefficient regrouping, and is exactly the mechanism used repeatedly by AG and CT [6, 7].

The factorial series deserves separate mention: it has zero ordinary radius of convergence but positive, well-ordered infinitesimal support. Its exact surreal value does not automatically identify it with a particular analytic continuation or summation of an ordinary divergent function; a special-function or resurgent-extension provider must supply that extra interpretation (section 12.7).

6.2 Compositional certificates and the five support services

An implementation should construct support certificates compositionally: a finite support supplies a base certificate; a product uses a Minkowski-sum constructor; inversion of a normalized infinitesimal perturbation uses a positive-monoid constructor; analytic substitution combines the outer support with such a monoid; finite unions and translates are available throughout. A bare statement that a user-supplied program “probably produces increasing exponents” is not such a certificate.

A useful support interface distinguishes five separate services.

- **Membership:** is a given exponent inside a support bound?
- **Coefficient:** return the actual coefficient, including cancellation.
- **Decomposition:** list the finitely many ways an exponent can arise in a product or substitution.
- **Leading term:** additionally certify that all smaller coefficients vanish.
- **Prefix:** describe everything below a chosen cutoff.

No one of these may be inferred automatically from the existence of the others. For finite supports all five are straightforward. For infinite supports, certificates can be built compositionally as above, but an arbitrary well-order proof can justify a mathematical expression without furnishing any of these operational services.

6.3 A certificate is not an evaluation algorithm

To compute a requested coefficient, a backend also needs an effective way to identify all contributing decompositions and to know when the list is complete. **A theorem that each such list is finite does not supply a terminating search that detects its end.** AG itself is credited by one of the sources with issuing this warning about its own formulas. Useful constructors therefore carry finite dependency bounds or explicit decomposer procedures in addition to semantic well-ordering evidence.

Similarly, a transfinite coefficient recursion proves the existence of a complete Hahn object without being an ordinary finite loop through the entire support. A correct implementation either produces a finite implicit object, computes specified coefficients with certified finite dependencies, or works inside a restricted effective support grammar. It must not report that it has “finished the transfinite recursion” merely because it computed many initial coefficients.

The capability-tiers article identifies precisely this missing service, and demands finite dependency bounds or explicit decomposer procedures, without supplying a construction. The other two articles supply one. We state the gap first because it is the motivation, rather than letting the lemma below silently absorb the caution.

6.4 The finite-grid dependency lemma, with both constructions

Theorem 6.1 (Finite extraction from a positive grid). *Let $e_1, \dots, e_m \in \mathbb{Q}^d$ be lexicographically positive, let $\gamma_0 \in \mathbb{Q}^d$, and let $c : \mathbb{N}^m \rightarrow k$ be a total computable coefficient rule in an effective field k . The monomial family*

$$(c(\mathbf{n}) t^{\gamma_0 + \sum_j n_j e_j})_{\mathbf{n} \in \mathbb{N}^m} \quad (6.1)$$

is strongly summable, and at any specified $\lambda \in \mathbb{Q}^d$ its coefficient is computable by a finite search — even when the map $\mathbf{n} \mapsto \gamma_0 + \sum n_j e_j$ is not injective.

Proof. The positive-monoid form of the Hahn–Neumann lemma makes the grid well ordered and gives finite fibres. For an effective bound, it suffices to produce a rational linear functional $\ell : \mathbb{Q}^d \rightarrow \mathbb{Q}$ with $\ell(e_j) > 0$ for every j . Two constructions are given below. Granting one, put $b = \ell(\lambda - \gamma_0)$. Every nonnegative integer solution of $\sum_j n_j e_j = \lambda - \gamma_0$ satisfies $\sum_j n_j \ell(e_j) = b$. If $b < 0$ there are

no solutions; otherwise

$$0 \leq n_j \leq \left\lfloor \frac{b}{\ell(e_j)} \right\rfloor \quad \text{for each } j.$$

Enumerate this finite box and test the *original vector equation*, not merely its image under ℓ . The desired coefficient is the finite sum of $c(\mathbf{n})$ over the solutions found. The same bound proves the finite-contribution requirement for strong summability directly; zero coefficients can only shrink the support. For a fixed multi-index there are finitely many ordered words and finite combinatorial multiplicities, so products and substitutions also give finite coefficient computations whenever the coefficient operations themselves terminate. $\square$

Construction I: increasing weights, forwards. Try

$$\ell_M(x_1, \dots, x_d) = M^{d-1}x_1 + M^{d-2}x_2 + \dots + Mx_{d-1} + x_d$$

for successively larger positive integers M . For each lexicographically positive e_j , its first nonzero coordinate eventually dominates the later coordinates in this expression. Because there are finitely many e_j , the search terminates. *This is the construction that is actually exercised in code:* the layered-representations article’s Python script implements it with M doubling ([section 18](#)).

Construction II: choosing weights backwards. Start with weight 1 on the last coordinate and work backwards. When choosing the weight on coordinate k , make it large enough that every e_j whose first nonzero coordinate is k has positive weighted sum, despite its already weighted later coordinates; such a first nonzero coordinate is positive by lexicographic positivity. There are only finitely many inequalities. This construction is not implemented in any delivered code.

The two constructions are genuinely different algorithms for the same object and both survive here.

Remark 6.2 (The functional is not an order embedding). Both sources that supply a construction issue the same warning independently, and it is load-bearing. The functional ℓ is an **auxiliary device on the finite generator set only**. It is *not* an order-preserving embedding of the whole lexicographic group into $\mathbb{Q}$ or $\mathbb{R}$ — no such embedding exists for rank greater than one — so it must never be reused as a substitute valuation for arbitrary cutoff comparisons. Its finite box proves termination but can be large; integer linear-equation methods, dynamic programming, sparse convolution and memoized dependencies can improve it, and must preserve exact treatment of coincident exponent sums by *adding* their coefficients rather than retaining multiple conflicting entries. A total coefficient rule still needs a supplied termination argument, or membership in a language where termination is built into the syntax.

6.5 The small-scale theorem and its effectivity gap

SA proves that an arbitrary formal series $\sum a_n X^n$ with set-many surcomplex coefficients can be evaluated after a sufficiently small rescaling; its proof chooses a surreal exponent dominating a set of exponents drawn from *all* the coefficients and shifts their supports into separated bands [1, 2]. For arbitrary surreal coefficients, positivity of $v(\epsilon)$ alone is not enough: the coefficient valuations may decrease too rapidly.

This is a valuable semantic existence theorem and **not** a generic finite procedure for discovering that bound from an arbitrary coefficient program. An effective interface should ask for a support-growth bound, a dominating-scale certificate, or a restricted coefficient grammar from which one

can be derived. The constructed scale may also lie outside the user's current workspace; domain enlargement must be explicit rather than hidden inside a purported fixed-field operation.

6.6 Inverses, positive recursions, and the reciprocal recurrence

For a known nonzero series write

$$A = ct^a(1 + E), \quad c \neq 0, \quad \text{supp}(E) \subset \Gamma_{>0}. \quad (6.2)$$

Then

$$A^{-1} = c^{-1}t^{-a} \sum_{n \geq 0} (-E)^n \quad (6.3)$$

is an exact Hahn identity. For finite E , or for a certified positive-grid representation of E , the support and finite decompositions are explicit. If a general input has no computable leading normalization, (6.3) is a *mathematical identity rather than a ready algorithm*. A package can retain an exact rational-expression inverse without expanding it when that representation is available — which is exactly what all three delivered prototypes do.

The same holds for HC's positive-operator formula $(I + T)^{-1}X = \sum_{n \geq 0} (-T)^n X$ and its nonlinear preparation recursions, whose finite coefficient dependencies are controlled by support words [5]. To make them executable an implementation needs effective word decomposition and executable ordinary coefficient operations; for a fixed-domain analytic division problem those operations can themselves involve ordinary integrals or implicitly specified functions. Such a coefficient is still representable by a well-defined symbolic operator when no elementary closed form is available, but it is not thereby in the smallest decidable scalar field.

For a normalized one-parameter series $A(t) = 1 + \sum_{n \geq 1} a_n t^n$, the reciprocal coefficients come from a recurrence needing no symbolic differentiation:

$$b_0 = 1, \quad b_n = - \sum_{k=1}^n a_k b_{n-k}.$$

Each requested b_n has finitely many dependencies. In a general Hahn group the corresponding recursion is indexed by actual exponents; a positive-support certificate supplies the well-founded relation, but a computational decomposer is still required.

7 Precision is a mathematical type, not a display setting

7.1 The $O_v(\cdot)$ convention

Write

$$x = \tilde{x} + O_v(\alpha) \iff v(x - \tilde{x}) \geq \alpha, \quad v(0) = +\infty.$$

This is an exact enclosure statement in the chosen valued workspace: a statement about a *set* of possible errors, not an equality in the field after deleting a formal O -symbol, and not ordinary real asymptotic big- O . The precision parameter α ranges over the *full ordered value group*, not over an ordinary numerical accuracy parameter. A strict bound $v(\cdot) > \alpha$ is a separately represented thing; the convention used throughout this report is the **weak** inequality unless a strict one is displayed.

7.2 Certified precision propagation

All three sources state the following identically, with the same convention.

Proposition 7.1 (Basic certified precision rules). *Suppose $A = \hat{A} + e_A$ and $B = \hat{B} + e_B$ with $v(e_A) \geq \alpha$ and $v(e_B) \geq \beta$. Then*

$$v((A + B) - (\hat{A} + \hat{B})) \geq \min\{\alpha, \beta\}, \quad (7.1)$$

$$v(AB - \hat{A}\hat{B}) \geq \min\{\alpha + v(\hat{B}), \beta + v(\hat{A}), \alpha + \beta\}. \quad (7.2)$$

If $\hat{A} \neq 0$, $\mu = v(\hat{A})$ and $\alpha > \mu$, then $A \neq 0$, $v(A) = \mu$, and

$$v(A^{-1} - \hat{A}^{-1}) \geq \alpha - 2\mu. \quad (7.3)$$

Proof. The sum error is $e_A + e_B$. The product error is $e_A\hat{B} + \hat{A}e_B + e_Ae_B$; apply the valuation inequalities. Under the last hypothesis the leading term of $\hat{A}$ cannot be cancelled by e_A , so $v(A) = \mu$. Finally $A^{-1} - \hat{A}^{-1} = -e_A/(A\hat{A})$, of valuation at least $\alpha - 2\mu$. $\square$

These formulas expose genuine precision loss through inversion of a small quantity: an error of valuation 10 in a quantity of valuation 3 gives an inverse error bound of only 4. Preserving the same absolute cutoff through division would be incorrect.

Remark 7.2 (Dependency tracking). A centre polynomial that cancels to zero does not prove the exact answer zero. Subtracting two identical truncations of independently represented numbers leaves a remainder enclosure unless exact equality of the underlying objects has been proved. Conversely, subtracting an object from itself should use dependency information and return exact zero: exact identity tracking can prove $A - A = 0$ even when only an approximate expansion of A has been computed, and treating the two appearances as unrelated error balls loses that information unnecessarily. A zero test on a truncated series may return “zero to the supplied precision”, but *that is not exact zero*. If a nonzero leading coefficient is known below the error cutoff, sign and leading valuation are certified; if all retained terms cancel, more information is required.

7.3 The positive finite-prefix criterion, and three counterexamples

Proposition 7.3 (Finite-prefix criterion for rank-one grids). *For $a, e_1, \dots, e_m, B \in \mathbb{Q}$ with every $e_j > 0$, the grid $a + \sum_j \mathbb{N}e_j$ contains only finitely many exponents below B , and those exponents can be enumerated effectively. The corresponding statement is false for general lexicographically ordered groups, even with one positive generator.*

Proof. If $a + \sum_j n_j e_j < B$ then every $n_j < (B - a)/e_j$, so a finite box contains all possibilities. For the counterexample use generator $(0, 1)$ and cutoff $(1, 0)$ in $\mathbb{Q}_{\text{lex}}^2$: every $(0, n)$ lies below that cutoff. $\square$

This positive criterion is the only one of its kind in the group, and it is what makes the rank-one case tractable. It is also exactly as far as the positive result goes. Three counterexamples, of two genuinely different kinds, delimit it, and no one of them should be allowed to stand for the family.

Higher-rank kind, witness 1 (grid). In an ordered group containing positive λ, μ with $n\lambda < \mu$ for every ordinary n , put

$$A = \sum_{n \geq 0} t^{n\lambda} + t^\mu = \frac{1}{1 - t^\lambda} + t^\mu. \quad (7.4)$$

The support $\{n\lambda : n \geq 0\} \cup \{\mu\}$ is well ordered and lies in the grid generated by λ and μ , and every first summand has exponent below μ . Hence the truncation below μ has infinitely many nonzero terms, and a finite list cannot represent that support completely. Moreover, after N geometric terms, indexed $0, \dots, N-1$, the remainder has valuation $N\lambda$ and *never* reaches μ : a refinement loop that counts terms runs forever, even though every finite step is mathematically correct.

Higher-rank kind, witness 2 (geometric remainder plus u). Take $\Gamma = \mathbb{Q}\omega + \mathbb{Q}$ and $u = t^\omega$, so $u < t^n$ for every ordinary positive n . The exact remainder identity is

$$\frac{1}{1-t} - \sum_{j=0}^{N-1} t^j = \frac{t^N}{1-t}, \quad v\left(\frac{t^N}{1-t}\right) = N < \omega. \quad (7.5)$$

No finite prefix of this expansion achieves error valuation at least ω , yet the strong sum exists and the left side is a finite exact description. The same issue affects $\sum_{n \geq 0} t^n + u$: an ordinary stream that insists on emitting all terms in increasing valuation order never reaches the u term.

Rank-one nondiscrete kind. Even rank one can fail the finite-prefix property. The supplied example

$$A = \sum_{k \geq 0} t^{1-1/(k+2)} \quad (7.6)$$

has well-ordered support of order type ω with infinitely many terms below exponent 1 [4, 5]; equivalently, the set $\{1 - 1/n : n \geq 1\}$ is increasing and well ordered but has infinitely many elements below 1. Hence “*rank one*” alone is not enough: discreteness or a suitable positive-grid bound is needed. The obstruction is therefore not merely about rank.

7.4 Three truncation interfaces, and two worked compressions

The shared conclusion of all three sources is that a single “precision” request conflates three different things, which must be separate interfaces:

1. a finite number of **leading terms**;
2. a **valuation-cutoff object**, whose representation may remain compressed;
3. a finite rectangular or total-degree **jet** in selected grid generators.

To these the layered-representations article adds a fourth request, a single fixed coefficient, which the others treat as part of the coefficient service.

There is no obstruction to an *exact compressed answer*. Two worked compressions appear in the sources. In (7.4) the truncation below μ is exactly $(1 - t^\lambda)^{-1}$. In (7.5) with $N = 10$, the exact value of $1/(1-t) - \sum_{j=0}^9 t^j$ is the rational block $t^{10}/(1-t)$, of valuation $(0, 10)$ in the two-coordinate parent $\{u, t\}$, while the requested error scale u has valuation $(1, 0)$, strictly larger than $(0, N)$ for every ordinary integer N . A correct finite-prefix routine must report that the requested cut needs an infinite block; a compressed-truncation routine returns the rational block exactly.

Remark 7.4 (Regression framing). This example is a regression test for any future precision module: *increasing an ordinary integer precision parameter must never claim to certify accuracy beyond u* . The valuation-vector comparison, not a numerical norm chosen for convenience, decides whether a claimed bound is valid. Correspondingly, a precision request whose result cannot be a finite term list must return a compressed certified subseries, an explicit unsupported-request result, or an alternative finite jet chosen by the caller. It must **never** silently stop after an arbitrary number of terms and claim the requested bound.

7.5 Nested descriptions and the ω th position

For higher-rank data, nested descriptions are useful: a coefficient at one outer scale may itself be an infinite series in a smaller-rank field, which compactly exposes a term occurring after an order-type- ω block. A whole t -block can be stored as a rational or lazy coefficient in an outer u -series; another suitable representation provides direct exponent access. The nesting order must agree with the intended ordered group, and operations must respect the resulting support conditions. A flat iterator that only returns the next natural-number-indexed term cannot reach an ω th support position in finitely many calls, and neither mechanism is equivalent to taking more digits in a single real-valued precision parameter.

7.6 Coefficient precision is a second, independent axis

Surreal scale precision and ordinary numerical precision are independent. The coefficient of $t^{1/2}$ may be known to 100 binary digits while the series is known only through exponent 3; increasing coefficient precision does not discover an omitted t^4 term, and producing more exact rational coefficients does not yield an ordinary floating-point approximation to an infinite surreal value. An interval coefficient enclosure containing zero does not determine a leading sign. For nonalgebraic ordinary coefficients, validated real or complex enclosures can sometimes certify nonvanishing and sign; if an enclosure contains zero one must refine it or retain an unresolved status. *Heuristically small is not exactly zero*, and an ordinary approximate coefficient must not be silently recast as a nearby rational value inside an exact-number constructor. A serious system records both the exponent cutoff and the coefficient-error model and refines whichever prevents the requested conclusion. Certified interval or ball arithmetic for coefficients and exact valuation truncation solve different problems.

7.7 These are not fine-topological limits

The full-class fine topology of SA uses every positive surreal tolerance, and in it **every set-indexed convergent net is eventually equal to its limit** [1, 2]. A sequence of finite truncations must therefore not be advertised as converging in that topology. Within a chosen rank-one valued field a sequence of truncations can converge in its valuation topology; the same words refer to a different structure. Nor does the ordinary sequence $1/n$ converge finely to zero in the full surreal field: a fixed positive surreal infinitesimal is smaller than every term.

An ordinary real limit and a surreal evaluation are different commands:

$$\lim_{X \rightarrow +\infty} \frac{1}{X+1} = 0 \text{ in ordinary real analysis,} \quad \text{whereas} \quad \frac{1}{\omega+1} > 0 \text{ in } \mathbf{No}.$$

The latter retains the infinitesimal information the former discards. Limit modes, Hahn summation, and finite-scale specialization must remain explicit operations rather than different spellings of one implicit “infinite evaluation” routine.

7.8 Jets, asymptotics, and exact tails

A finite Taylor jet in $K[z]/(z)^N$ has exact quotient-ring semantics: it determines products modulo $(z)^N$, not a unique analytic function. An asymptotic object records a function or value modulo an explicitly stated error class. An exact lazy object specifies all coefficients even though only finitely many are cached, and an exact rational tail specifies infinitely many coefficients in one node. These distinctions must survive serialization and printing. A finite multivariate jet is a quotient by a chosen monomial ideal in auxiliary variables; after substituting the grid generators, its omitted monomials must be converted into an explicit valuation error bound, a conversion that may be coarse and may not describe an entire valuation prefix.

7.9 Rounding and the $\Pi \oplus \mathbb{Z}$ floor

For an infinite surreal there is no ordinary integer floor in $\mathbb{Z}$: no ordinary integer brackets such a value, so an ordinary $\mathbb{Z}$ -valued floor cannot satisfy the usual defining inequalities. A package must state what its target “integers” are before defining a global floor, rather than silently reusing ordinary integer rounding.

One useful normal-form choice is the ordered subring $\Pi \oplus \mathbb{Z}$, where Π is the purely infinite additive class. Products of purely infinite parts have only negative exponents, so this is indeed a subring. For $x = p + r + \eta$ with p purely infinite, $r \in \mathbb{R}$ and η infinitesimal, define

$$\lfloor x \rfloor_{\Pi \oplus \mathbb{Z}} = p + \begin{cases} \lfloor r \rfloor - 1, & r \in \mathbb{Z} \text{ and } \eta < 0, \\ \lfloor r \rfloor, & \text{otherwise.} \end{cases} \quad (7.7)$$

This is the *unique* element $m \in \Pi \oplus \mathbb{Z}$ with $m \leq x < m + 1$: noninteger r has positive ordinary distance from its neighbouring integers, while at an integer the infinitesimal sign selects the side, and any distinct purely infinite part differs by an infinite quantity and cannot satisfy the same unit-interval condition. Thus

$$\lfloor 1 - t \rfloor = 0, \quad \lfloor \omega/2 \rfloor_{\Pi \oplus \mathbb{Z}} = \omega/2,$$

the second not an ordinary integer. The operation is effective only when the backend can split the normal form, decide the ordinary integer predicate, and determine the infinitesimal sign; a compact rational node need not expose its entire purely infinite part as a finite list. The ring $\Pi \oplus \mathbb{Z}$ is Conway’s ring of omnific integers (`foundations-and-computation/foundations`, `found:eq:omnific`), so this floor is the omnific floor; its arithmetic and notations are developed in the omnific-integer reports of the collection.

8 Polynomial algebra and root-cluster algorithms

8.1 Finite algebra over a declared coefficient domain

Over an effective field, polynomial degree is an ordinary finite integer. Long division terminates by degree descent; the Euclidean algorithm gives gcds and Bézout identities; squarefree decomposition uses the derivative; resultants and discriminants are finite determinants; matrices admit exact row reduction whenever pivot zero tests are available. These apply to the core of [section 5](#) without expanding any coefficient into an infinite Hahn normal form. Gaussian elimination decides matrix rank and solves consistent systems, characteristic polynomials and algebraic root selectors represent

eigenvalues, positive square roots give distances, and exact determinant signs decide orientation — so TR’s triangle identities can be supported without ever choosing an infinite circular phase, since normalized directions lie on the finite-coordinate unit circle [3].

For multivariate polynomials, Buchberger-type ideal algorithms depend on finite monomial orders and effective coefficient operations; a completed Gröbner basis certifies ideal membership or inconsistency by a finite identity. A zero-dimensional quotient has a finite basis and multiplication matrices, while a positive-dimensional solution set needs a different output shape: the mathematical existence of a root over an algebraically closed Hahn field is not a promise to list a positive-dimensional family.

If coefficient equality is only partial, the polynomial routines become partial or conditional. An unresolved leading coefficient can change the degree; an unresolved pivot can change matrix rank; a determinant that fails to simplify is not a proof of nonsingularity. *Testing a coefficient with an ordinary numerical heuristic and proceeding as though the result were an exact algebraic fact is unsound.* A domain adapter should expose the conditions under which a matrix result was obtained.

8.2 The initial polynomial and its exact root counts

PA supplies a particularly useful computational interface. Write

$$P(a + Y) = \sum_{j=0}^n b_j Y^j, \quad \lambda = \min_{b_j \neq 0} \{v(b_j) + j\rho\},$$

and define the nonzero ordinary residual polynomial

$$I_{a,\rho}(P)(X) = \text{st}\left(t^{-\lambda} P(a + t^\rho X)\right). \quad (8.1)$$

Its computation uses finite translation, exact coefficient valuations and leading coefficients, exponent comparison and residue arithmetic. It need not find any full root expansion.

For a complete root list α_i in a containing algebraically closed workspace, PA proves

$$I_{a,\rho}(P)(X) = C \prod_{v(\alpha_i - a) \geq \rho} \left(X - \text{st} \frac{\alpha_i - a}{t^\rho}\right), \quad C \neq 0. \quad (8.2)$$

Consequently its **degree** counts the roots in the closed valuation ball, its **order at zero** counts the roots in the open ball, and the **multiplicity of a residual root** c counts the roots in the corresponding residue-direction ball. These are exact finite conclusions even when the represented roots have infinite supports [4].

The required valuations are effective for rational monomial coefficients by [theorem 5.1](#). For algebraic or more general coefficients an additional valuation and leading-coefficient service is needed; *decidable field equality alone should not be confused with an already implemented expansion engine.* Algebraic root descriptors can remain valid while that service is still being developed.

8.3 Newton profiles without a real-valued plot

At centre zero the profile is

$$\phi_P(\rho) = \min_j \{v(a_j) + j\rho\},$$

with candidate breakpoints at the finitely many values $\rho = (v(a_j) - v(a_k))/(k - j)$ for $j \neq k$. At a breakpoint, the difference between the largest and smallest active indices is the number of nonzero

roots at that valuation. Exact division by ordinary integers in a divisible exponent group suffices; the exponents need *not* be embedded in $\mathbb{R}$ [4].

A straightforward implementation enumerates candidates, sorts them by the ordered-group comparator, and evaluates all affine expressions at each candidate; a lower-hull implementation reduces repeated work. Drawing an ordinary Euclidean Newton polygon is only a visualization for suitable rank-one inputs. For exponents such as 1 and ω , a finite algebraic profile is more faithful than a misleading numerical plot.

The general leading-valuation condition behind this is elementary: for $P(X) = \sum_j a_j X^j$ with a nonzero root beginning ct^λ ,

$$\min_j (v(a_j) + j\lambda) \quad \text{must be attained at least twice,} \quad (8.3)$$

since otherwise the unique term of least valuation could not cancel; the coefficients attaining the minimum give the initial equation $\sum_{j \in J} \text{lc}(a_j) c^j = 0$, whose nonzero roots are the candidate leading coefficients. Substituting $X = t^\lambda(c + Y)$ and normalizing produces a smaller perturbation problem; fractional λ can force ramification. Two small examples need no infinite search:

$$X^2 - 2X + 1 - t = 0 \implies X = 1 \pm t^{1/2}, \quad X^2 + t = 0 \implies X = \pm i t^{1/2}.$$

Both roots of the first have standard part 1 but are distinct exact elements; storing only standard parts loses multiplicity and branch information. The second has no real roots with uncertain sign. These are the algebraic steps behind Newton–Puiseux and generalized Newton-polygon methods, for which grid-based treatments supply a developed framework [14].

Example 8.1 (The cubic). Let $P(z) = z^3 - t^2z - t^5$. Its coefficient profile is $\min\{5, 2 + \rho, 3\rho\}$. The active indices at $\rho = 1$ are $\{1, 3\}$ and at $\rho = 3$ are $\{0, 1\}$, so two roots have valuation 1 and one has valuation 3. The residual polynomials are

$$I_{0,1}(P) = X^3 - X, \quad I_{0,3}(P) = -X - 1,$$

whose nonzero residual roots select the leading terms t , $-t$, $-t^3$. After these simple residual branches have been selected, ordinary coefficient recursion gives

$$\begin{aligned} \alpha_+ &= t + \frac{1}{2}t^3 - \frac{3}{8}t^5 + O_v(7), \\ \alpha_- &= -t + \frac{1}{2}t^3 + \frac{3}{8}t^5 + O_v(7), \\ \alpha_0 &= -t^3 - t^7 + O_v(11). \end{aligned}$$

These finite displays do not replace the exact root descriptors; they are certified expansions attached to them.

Example 8.2 (The quartic and derivative-root counts). For $P(z) = z(z - t^3)(z - t)(z - 1)$ the same machinery gives the residual weights and polynomials at $\rho = 0, 1, 3$, and the corresponding counts for P' . The general statement from PA is that a ball containing $k \geq 1$ root multiplicities contains $k - 1$ derivative-root multiplicities. The **occupation hypothesis** must be retained: a ball containing no roots of P can still contain a root of P' . These routines count roots; they do not locate them.

8.4 Hensel lifting, clusters, and algebraic fallback

When a monic polynomial has pairwise coprime ordinary residual factors, PA constructs unique lifted cluster factors: at each positive exponent the correction is obtained by the same finite linear inverse, and the support lies in the positive monoid generated by the input correction support [4]. With effective support decomposition this is an implementable coefficient algorithm.

A *multiple* residual root is different. For $z^m - t$ every root reduces to zero and the first useful scale is $t^{1/m}$; a routine that only attempts ordinary simple-root Newton iteration at zero will fail. It should split or rescale the cluster, enlarge the exponent group when required, or return an exact algebraic object with an unresolved expansion. The algebraic fallback is valuable precisely because it separates exact solvability from the cost of finding a convenient normal form. At multiple roots the correct interface may return a finite *cluster* object rather than arbitrarily numbered individual branches: the preparation and coprime-factor lifting results retain the support of *factor coefficients*, while individual roots can require fractional scales, and factor coefficients are often stable under perturbation even when individual root labels are not.

Newton corrections $x_{k+1} = x_k - P(x_k)/P'(x_k)$ require a justification of the invertibility of $P'(x_k)$ and of the precision improvement.

Remark 8.3 (Cofinality is not free in higher rank). There is **no unrestricted guarantee** that a natural-number sequence of successive corrections becomes cofinal in every desired higher-rank cutoff: in a higher-rank field, repeatedly doubling a positive error valuation need not reach a cutoff in a strictly higher Archimedean class. Newton iteration is therefore a coefficient-refinement method with a stated target policy, not a universal convergence proof in the full surreal topology. PA’s general lifting proof is a well-founded support recursion. Classical Newton–Puisseux is a rank-one specialization and should be identified as such rather than extrapolated to arbitrary program-described Hahn coefficients [11, 4].

8.5 Root-perturbation thresholds: two different sharp theorems

Two of the sources give sharp perturbation results. They are *not* two versions of one theorem, they come from two different supplied manuscripts, and they concern two different situations: univariate coefficient perturbation with a multiplicity-driven exponent loss, and multivariate system perturbation with an inverse-Jacobian-driven uniqueness ball. They are printed separately and must not be merged into one “root stability” statement.

8.5.1 Univariate coefficient perturbation (from PA)

For monic degree- n polynomials with finite coefficients, PA proves that coefficient errors of valuation at least $\varepsilon > 0$ permit a matching of root multisets with

$$v(\alpha_i - \beta_i) \geq \varepsilon/n. \quad (8.4)$$

The exponent loss is **sharp**: compare z^n with $z^n - t^\varepsilon$. For a squarefree integral polynomial, put

$$d_i = v(P'(\alpha_i)), \quad \delta_i = \max_{j \neq i} v(\alpha_i - \alpha_j).$$

PA gives the sharper sufficient condition $\varepsilon > \max_i (d_i + \delta_i)$ with

$$v(\beta_i - \alpha_i) \geq \varepsilon - d_i,$$

and the easily stated coefficient-level sufficient condition $\varepsilon > v(\text{Disc } P)$ [4]. These are certificates for stable cluster labels, not heuristic tolerances. The strictness matters, and the collision occurs exactly at the threshold:

$$z(z - t^a) + \frac{t^{2a}}{4} = \left(z - \frac{t^a}{2}\right)^2. \quad (8.5)$$

Before applying these estimates to infinite root locations, the polynomial must be normalized to the stated finite-coefficient setting and the error transformed with it.

Remark 8.4 (Operational framing). A software request for roots to valuation ρ may require coefficient information through approximately $n\rho$ near an unresolved multiple cluster. *Ignoring multiplicity overstates the precision.*

8.5.2 Multivariate system perturbation (from AG)

For a simple zero a of a nonnegative-support system, AG sets

$$\kappa_a = \max\left(0, -\min_{i,j} v((DF(a)^{-1})_{ij})\right). \quad (8.6)$$

For a perturbation D with $\delta = \min_i v_H(D_i) > 2\kappa_a$, AG proves a *unique* continued zero in the ball $v_{\min}(b - a) > \kappa_a$, with the displacement bound

$$v_{\min}(b - a) \geq \delta - \kappa_a. \quad (8.7)$$

These are exact inequalities in the full ordered value group [6].

The strict inequality is **sharp**: $z^2 - t^{2\eta}$ perturbed by $t^{2\eta}$ collapses two simple roots into a double root precisely at $\delta = 2\kappa_a$. *The equality case cannot be accepted.*

Remark 8.5 (Operational framing). A root tracker can use this as a certificate-producing operation: return the old root descriptor, the perturbation bound, the inverse-Jacobian bound, and the new implicit root with a verified uniqueness ball. It need not enumerate an infinite Newton sequence. These guarantees remain conditional on obtaining the exact bounds: *a numerical estimate at one real specialization of the monomials is not a proof of the higher-rank inequality*, and interval bounds for ordinary coefficients cannot replace exponent-group comparisons.

8.6 Implicit analytic roots

Suppose an ordinary analytic equation has $f(c) = 0$ and $a = f'(c) \neq 0$, and consider $f^\sharp(z) + \tau g^\sharp(z) = 0$ for a positive infinitesimal τ . SA gives the unique solution in the monad of c as a support-controlled implicit germ, whose first coefficients are

$$z = c - \frac{g(c)}{a}\tau + \left(\frac{g(c)g'(c)}{a^2} - \frac{f''(c)g(c)^2}{2a^3}\right)\tau^2 + O(\tau^3). \quad (8.8)$$

A CAS can store the implicit equation, the selected monad and a nonzero-Jacobian certificate as the exact answer, computing coefficients on demand; the proof is coefficient recursion, not a numerical root search in a large finite approximation [1].

More generally, for a formal system $\Phi(X, Y) = 0$ with specified initial point and invertible Jacobian in Y , each new homogeneous degree's coefficient is determined by one fixed finite matrix inverse; when the coefficient operations and input generators are effective, the solution's finite jets are

computable. A natural exact object is `ImplicitSeries[equations, variables, initialData, certificate]`: the equation and uniqueness certificate denote the whole solution, and a computed jet is only a view of it. A returned solution should include the root centre, the leading branch data, the Jacobian certificate, and support or precision bounds. *A singular Jacobian is not a proof that no solution exists*; it means this simple-root procedure is inapplicable and a preparation, ramification or finite-algebra method is needed.

8.7 Near-tangency conditioning is algebraic information

TR proves, for $0 < \delta \in \mathfrak{m}$,

$$q(\delta) = \arccos(1 - \delta) = \sqrt{2\delta} \left(1 + \frac{\delta}{12} + \frac{3\delta^2}{160} + \frac{5\delta^3}{896} + O(\delta^4) \right), \quad (8.9)$$

and, for $e \neq 0$ with $e \prec \delta$, the branch-stability formula

$$v(q(\delta + e) - q(\delta)) = v(e) - \frac{1}{2}v(\delta). \quad (8.10)$$

The inverse derivative therefore *loses half the valuation of the tangency parameter*. This is not a roundoff anecdote: it is an exact input–output precision rule that a branch-aware algebraic or trigonometric evaluator can propagate [3].

9 Exact functions: formal germs, ordinary lifts, coherent data, radius-free germs

9.1 Formal evaluation versus ordinary lift

Let $k \subseteq \mathbb{C}$ be an admitted coefficient field and $A(X) = \sum_{n \geq 0} a_n X^n$ a specified formal power series over k . For an infinitesimal ϵ with a certified well-ordered positive support, the evaluation

$$A(\epsilon) = \sum_{n \geq 0} a_n \epsilon^n \quad (9.1)$$

is strongly summable: the support lies in $(\text{supp } \epsilon)^*$ and finite word multiplicity gives finitely many contributions at each exponent. **No ordinary radius of convergence is required** [2].

Thus both $\sum t^n/n!$ and $\sum n! t^n$ have exact Hahn interpretations, but only the first is the Taylor series of an ordinary holomorphic function near zero. The second may be stored as a formal-series value with a recurrence and a support certificate. Calling both objects ordinary analytic germs would erase a distinction on which the whole coherence theory depends.

9.2 The Taylor lift, and what it does not confer

For an ordinary holomorphic germ f at $c \in \mathbb{C}$ — with a branch specified where necessary — and $\epsilon \in \mathfrak{m}$, define

$$f^\sharp(c + \epsilon) = \sum_{n \geq 0} \frac{f^{(n)}(c)}{n!} \epsilon^n. \quad (9.2)$$

Proposition 9.1 (Correctness of infinitesimal substitution). *Suppose a function descriptor specifies an ordinary analytic germ at c , an exact centre, and an effective Taylor-coefficient procedure, and let ϵ have certified well-ordered positive support E . Then (9.2) is a strong Hahn sum with support contained in E^* , and supplies an exact symbolic value. If the support and coefficient dependencies are effective, each requested finite coefficient calculation terminates. Sums, products and compositions of such analytic germs are respected whenever the ordinary centres match and the relevant germs are defined. **Equality of arbitrary resulting values is not guaranteed by this construction alone.***

Proof. Every coefficient contribution is indexed by a finite word in E , and the positive-support lemma of section 6.1 gives strong summability, so the semantic existence is immediate; the descriptor fixes every coefficient, hence a unique value. At a fixed exponent the formal Taylor identities for sums, products and composition involve only finitely many contributing terms and therefore remain valid after substitution. Centre values are handled as ordinary coefficients before the infinitesimal expansion is performed. Each specified effective coefficient request uses finitely many certified dependencies, and theorem 6.1 turns this into a coefficient algorithm when the supports are given as effective grids. The final qualification follows from theorem 3.5: a general formal-series constructor cannot automatically acquire a complete zero test. $\square$

The domain is $U^\sharp = \{c + \epsilon : c \in U, \epsilon \in \mathfrak{m}\}$, *not* every surcomplex number whose printed formula resembles an ordinary argument. Ordinary values such as $f(c)$ are evaluated or named first; it is not correct to insert a nonzero ordinary c directly into a Hahn power series about zero and invoke ordinary convergence. Local instances:

$$\exp^\sharp(c + \epsilon) = e^c \sum_{n \geq 0} \frac{\epsilon^n}{n!}, \quad (9.3)$$

$$\log^\sharp(c + \epsilon) = \log(c) + \sum_{n \geq 1} \frac{(-1)^{n+1}}{n} \left(\frac{\epsilon}{c}\right)^n, \quad c \neq 0, \quad (9.4)$$

$$\sin^\sharp(c + \epsilon) = \sin(c) \cos^\sharp(\epsilon) + \cos(c) \sin^\sharp(\epsilon). \quad (9.5)$$

The logarithm's ordinary germ must be chosen; there is no holomorphic logarithm germ at zero. At $c = 0$ the exponential coefficients lie in $\mathbb{Q}$, whereas at an algebraic nonzero c the coefficient e^c may require a transcendental constant descriptor: domain closure must be checked at both the coefficient and the exponent level. For finite real $r + \eta$ one likewise evaluates the ordinary constants first,

$$\sin(r + \eta) = \sin r \sum_{n \geq 0} \frac{(-1)^n \eta^{2n}}{(2n)!} + \cos r \sum_{n \geq 0} \frac{(-1)^n \eta^{2n+1}}{(2n+1)!}, \quad (9.6)$$

$$\cos(r + \eta) = \cos r \sum_{n \geq 0} \frac{(-1)^n \eta^{2n}}{(2n)!} - \sin r \sum_{n \geq 0} \frac{(-1)^n \eta^{2n+1}}{(2n+1)!}. \quad (9.7)$$

The construction is also more general than its simplifier. The expression $\sin^\sharp(\pi + t) = -\sin^\sharp(t)$ is exact once the ordinary constants and functions are admitted, but a minimal $\mathbb{R}_{\text{alg}}$ -coefficient implementation does not thereby gain a complete decision procedure for every ordinary constant generated by arbitrary special-function evaluation. *A value constructor may be more general than its simplifier.*

9.3 Which familiar functions can be included

At regular ordinary centres the mechanism applies to elementary analytic functions and to specified branches of special functions: exponential, logarithm away from its selected cut, trigonometric functions, gamma away from its poles, error functions, Airy functions, Bessel functions with their parameter and branch conditions, and hypergeometric functions at regular parameter and argument values. This is a rule about known ordinary germs, not an assertion that every possible parameter value or singularity is regular.

Ordinary CAS differentiation and series tools are valuable coefficient engines and supply exact special-function expressions, derivatives and local expansions [38]. A successful coefficient call does not by itself certify an entire surreal support or a global function domain, and the extension layer must preserve that missing information. Calling `SeriesCoefficient` on a supported classical function can supply coefficients, but a returned unevaluated expression is not a termination guarantee for all requested indices, and user-supplied recurrences require their own totality and domain assumptions.

Coefficient-domain promotion. $\Gamma^\sharp(2+t)$ is an exact value given by the ordinary germ at 2. Its coefficients may involve constants such as Euler’s constant and derivatives of gamma, which lie *outside* the real-algebraic coefficient core. The system should promote to an exact ordinary-constant domain with correspondingly weaker general equality guarantees, not silently call those constants algebraic.

Many useful special-function germs can instead be specified by a finite differential equation or recurrence, initial values and a chosen ordinary domain. A linear differential equation at an ordinary point provides recursive Taylor coefficients; algebraic equations provide implicit-function recurrences at simple branches. Parameter singularities, resonance, irregular singular points and branch changes must be part of the input checks.

9.4 Singular centres and the summability boundary

At an ordinary pole one uses a finite-principal-part Laurent germ on a punctured infinitesimal neighbourhood, factoring the meromorphic germ into a finite negative power and a holomorphic germ; evaluation then requires a nonzero displacement. At an algebraic branch point one uses a ramified coordinate and a selected Puiseux branch. Formula (8.9) is a basic example: the ordinary Taylor expansion of $\arccos$ at 1 is not an analytic power series in δ , but becomes a convergent ordinary series after the positive square-root uniformizer is introduced.

At an *essential* singularity a Laurent expression can fail Hahn summability outright. If $\epsilon \neq 0$ is infinitesimal, the terms $\epsilon^{-n}/n!$ have strictly *decreasing* valuations, so

$$\sum_{n \geq 0} \frac{\epsilon^{-n}}{n!}$$

is **not** a strong Hahn definition of $e^{1/\epsilon}$. SA explicitly distinguishes this failed substitution from a separately selected global exponential evaluated at $1/\epsilon$ [1, 2]. A separate global real exponential or surcomplex extension may assign a value, but that is a different construction.

9.5 Formal, analytic, and asymptotic descriptors are different

A formal series with computable coefficients can be meaningful at infinitesimal inputs even when it is not an ordinary analytic germ, and an asymptotic expansion of an ordinary function is a third kind

of data. A general smooth function does not have a unique infinitesimal extension determined by its Taylor coefficients: an ordinary smooth function flat at zero has the same Taylor jet as the zero function there, and assigning only that jet to all infinitesimals discards information about its nonzero values away from zero. CT explicitly separates analytic target coefficients from merely smooth parameter coefficients, and a CAS should retain the same distinction [7]. See also [section 12.6](#).

9.6 Hahn-coherent data on a fixed common ordinary domain

The first of the two coherence certificates in this report is the one all three sources define.

Definition 9.2 (Common-domain Hahn-coherent datum). A Hahn-coherent datum on a *fixed* ordinary domain $U \subseteq \mathbb{C}$ is

$$F = \sum_{\gamma \in S} f_\gamma t^\gamma, \quad f_\gamma \in \mathcal{O}(U), \quad S \text{ well ordered}, \quad (9.8)$$

where $\mathcal{O}(U)$ denotes the ordinary holomorphic functions on U , with *one* common well-ordered support and *one* common domain. Its value at $c + \epsilon \in U^\sharp$ is

$$F^\sharp(c + \epsilon) = \sum_{\gamma \in S} \sum_{n \geq 0} t^\gamma \frac{f_\gamma^{(n)}(c)}{n!} \epsilon^n, \quad (9.9)$$

with source support bound $S + (\text{supp } \epsilon)^*$.

Coefficient functions must share the stated ordinary domain. **Merely holding an unrelated germ at each centre does not provide coherent data**, and pointwise knowledge that selected values are infinitesimal is no substitute for the uniform support and domain condition [2, 5]. A computational object should therefore contain both a support description and an ordinary analytic domain descriptor, plus a coefficient-function provider. Finite support with rational or explicitly holomorphic coefficient functions is a particularly accessible initial implementation; infinite coherent data need further operational certificates, just as infinite scalar series do.

HC's Hartogs and gluing theorems are mathematically useful but **should not be exposed as unconditional algorithms** that discover a global support or decide arbitrary holomorphic identities. To invoke them computationally, the input must contain the relevant function descriptions and hypothesis certificates, or a restricted syntax in which those hypotheses can be checked.

9.7 The radius-free Hahn-germ algebra: a strictly weaker certificate

The second certificate is present in only one of the three sources, imported from AG, and it is *strictly weaker* than [theorem 9.2](#). The two must carry distinct names, and a system must not silently identify them.

Definition 9.3 (Radius-free Hahn germ). Put

$$R_n = \mathbb{C}\{z_1, \dots, z_n\}, \quad \mathcal{A}_n = R_n((t^\Gamma)). \quad (9.10)$$

Each coefficient is an ordinary *convergent germ at zero*, but different coefficients need not share any common ordinary radius.

A common-domain family in $\mathcal{O}(U)((t^\Gamma))$ is a stronger and different certificate [6]. A finite computational descriptor must therefore **separate its Hahn-support program from its ordinary coefficient-germ program**.

The distinction has executable consequences. Let

$$H(z) = \sum_{n \geq 1} \frac{t^n}{1 - nz}. \quad (9.11)$$

Every coefficient is a valid ordinary germ at zero, and there is *no* positive ordinary disk on which all of them are holomorphic, since their poles are $1/n$. Nevertheless $H(a)$ is well defined for every infinitesimal a , by strong Taylor substitution. **Calling an ordinary contour routine on a fixed disk containing all coefficient domains is an error**; infinitesimal rescaling is the correct alternative (section 11.4). Substituting $a = t^2$,

$$H(t^2) = \sum_{n \geq 1} \frac{t^n}{1 - nt^2} = t + t^2 + 2t^3 + 3t^4 + 5t^5 + 9t^6 + 16t^7 + 31t^8 + \dots, \quad (9.12)$$

and at exponent m the coefficient is the finite integer sum

$$[t^m]H(t^2) = \sum_{\substack{n \geq 1, k \geq 0 \\ n+2k=m}} n^k. \quad (9.13)$$

The coefficient algorithm terminates with an explicit finite dependency set even though the coefficient germs share no radius. The values for $m = 1, \dots, 16$ are

$$1, 1, 2, 3, 5, 9, 16, 31, 61, 125, 266, 579, 1305, 3009, 7120, 17255,$$

independently checked by symbolic computation (section 18). CT identifies the same value as a microscopic Cauchy integral at an appropriate radius [7]. *None of the three delivered prototypes represents the outer infinite sum*; (9.13) describes a straightforward next lazy-series module.

AG also distinguishes $\mathcal{A}_n$ from $K[[z]]$. The formal expression

$$(z - t^\eta)^{-1} = - \sum_{j \geq 0} t^{-(j+1)\eta} z^j \quad (9.14)$$

is a power series over K but is **not** an element of $\mathcal{A}_1$: its combined Hahn exponents are not well ordered. In $\mathcal{A}_1$, the function $z - t^\eta$ actually *vanishes at an infinitesimal point*, so it must not be marked as an invertible germ on the whole monad. A CAS needs separate types for a formal completion at a point and a function algebra on the entire infinitesimal neighbourhood [6].

9.8 Composition contracts and the blind-substitution failure

Formal power-series composition $A(B(X))$ is immediately defined when the inner constant term is zero, because each resulting coefficient depends on finitely many terms. A nonzero inner constant generally requires an analytic re-expansion or a separate summability condition; at finite surreal inputs the ordinary-centre Taylor lift supplies that re-expansion when the ordinary analytic domains match.

In the coherent category, for $F = f_0 + E$ with nonnegative support, E of positive support, and $f_0(U) \subseteq V$, the sources define

$$H \circ F = \sum_{\delta, n} \frac{h_\delta^{(n)} \circ f_0}{n!} t^\delta E^n, \quad H = \sum_{\delta} h_\delta t^\delta \in \mathcal{H}(V), \quad (9.15)$$

with support contained in $\text{supp } H + (\text{supp } E)^*$. This common-domain condition is stronger than pointwise existence. For a transseries function at infinity, admissible composition uses its large-argument theory instead. These contracts must not be collapsed into one unchecked substitution routine. An executable implementation needs a verified ordinary-domain inclusion, ordinary derivatives and compositions, and effective support bookkeeping; a symbolic domain obligation can remain in a conditional result when it cannot be resolved automatically.

Blind substitution fails. The geometric expansion $\sum_{n \geq 0} t^n z^n$ is coherent on the finite plane and equals $(1 - tz)^{-1}$ there. Substituting $z = t^{-2}$ into the expansion produces exponents $-n$, which are *not* well ordered. The rational expression still has a value at t^{-2} , but its valid expansion is now

$$\frac{1}{1 - t^{-1}} = -\frac{t}{1 - t} = -\sum_{n \geq 1} t^n, \quad (9.16)$$

and at $z = t^{-1}$ there is a pole. A failed series substitution therefore requires re-expansion, a different representation, or rejection, depending on the actual argument.

9.9 Evaluation, translation, and finite jets

For $F = \sum_{\gamma} f_{\gamma} t^{\gamma}$ and an infinitesimal tuple a , AG defines

$$F(a) = \sum_{\gamma, \alpha} t^{\gamma} \frac{\partial^{\alpha} f_{\gamma}(0)}{\alpha!} a^{\alpha}, \quad (9.17)$$

$$(\tau_a F)(z) = \sum_{\gamma, \alpha} t^{\gamma} \frac{\partial^{\alpha} f_{\gamma}(z)}{\alpha!} a^{\alpha}, \quad (9.18)$$

with support certificate $S + E_a^*$, where $S = \text{supp } F$ and E_a is the union of the coordinate supports of a . A coefficient request can be compiled into a finite dependency graph when those supports have effective presentations and the ordinary germ derivative requests terminate. This gives an actual algorithmic interpretation of AG's formula, *not* an assumption that every convergent germ is already a CAS object.

Finite jets are particularly accessible: at a represented infinitesimal a ,

$$\mathcal{A}_n / (z - a)^N \simeq K[X_1, \dots, X_n] / (X_1, \dots, X_n)^N. \quad (9.19)$$

Once the required finite derivatives have been computed, the right-hand side uses ordinary sparse polynomial arithmetic with a degree cutoff, and a separate Hahn precision cutoff may be applied to its coefficients. **The Taylor degree and the Hahn valuation are different axes of approximation and must not be compressed into one integer option.**

9.10 Preparation and division as operator algorithms

AG's one-variable preparation and its several-variable division formulas are useful implementation specifications. For $F = f + E$ with leading ordinary germ f regular in a selected variable w and $v_H(E) > 0$, and with D and Π the ordinary Weierstrass quotient and remainder operators,

$$Q = (\text{id} + DM_E)^{-1} DG = \sum_{r \geq 0} (-DM_E)^r DG, \quad R = \Pi(G - EQ), \quad (9.20)$$

with support contained in $\text{supp } G + (\text{supp } E)^*$. These are exact formulas from AG [6], **not** an implemented routine in any of the three delivered packages.

Three executability caveats survive and all three matter.

- The system needs a *concrete ordinary division backend*. For polynomial or algebraic coefficient germs, polynomial reduction and selected local algebra procedures are natural choices; for a suitable differential or analytic descriptor, additional constructive division methods may be possible. Even a finite output coefficient can call an ordinary operation unavailable for the given germ language.
- *An arbitrary choice-based complex-linear splitting is not executable* merely because it preserves the space of analytic germs. AG’s arbitrary existence of a linear splitting is sufficient for its theorem but not for a program.
- The ordinary leading order and regularity direction must be found or supplied, and *if detecting whether a coefficient germ is zero is only semidecidable, searching for the first nonzero homogeneous part need not terminate on zero input*. A reliable API accepts certified leading data, tries supported methods, and otherwise returns a structured unresolved result rather than asserting a Weierstrass polynomial.

9.11 A nonpolynomial algebraic cluster: two developments of $\sin^2 z = t$

Two of the sources develop the same example in two compatible ways, and both survive.

Local monad development. $\sin^2 z - t$ has exactly two zeros in the monad of zero, counted with multiplicity:

$$z_{\pm} = \pm \arcsin^{\sharp}(\sqrt{t}) = \pm \left(t^{1/2} + \frac{t^{3/2}}{6} + \frac{3t^{5/2}}{40} + \frac{5t^{7/2}}{112} + \cdots \right),$$

representable locally as two analytic-root descriptors after ramification. The exact prepared cluster polynomial is

$$P(z) = z^2 - (\arcsin^{\sharp}(\sqrt{t}))^2. \quad (9.21)$$

It is polynomial in z , but its coefficients lie in an *analytic extension* of the minimal rational monomial field.

Remark 9.4 (Algebraicity is relative to the ambient domain). A package must not label these roots “algebraic over the original coefficient field” merely because they occur in a prepared polynomial over a newly enlarged field. The ambient coefficient domain is part of every algebraicity claim.

Full-strip development. On the canonical strip modulo 2π , putting $U = E_{\mathcal{H}}(iz)$ (see [section 13.4](#)) turns the equation into

$$-U^4 + (2 - 4t)U^2 - 1 = 0, \quad (9.22)$$

which has four roots counted with multiplicity; the two other angular roots lie near π modulo 2π . A local monad solver and a full-strip periodic solver are answering different, compatible questions.

10 Finite quotient algebras and residues

10.1 Remainders, traces, and norms without choosing roots

Let $P \in K[z]$ be monic of degree n over an effective surcomplex coefficient domain K , and put $A_P = K[z]/(P)$. Polynomial remainders represent elements of this n -dimensional algebra, and multiplication matrices give traces, norms, characteristic polynomials and resultants *without choosing individual roots*. The residue functional of PA is

$$\lambda_P(H) = [z^{n-1}](H \bmod P), \quad (10.1)$$

with trace identity $\text{Tr}(m_H) = \lambda_P(P'H)$. Both are finite exact coefficient calculations [4].

10.2 Residue pairing versus trace pairing

The pairing $(G, H) \mapsto \lambda_P(GH)$ has determinant $(-1)^{n(n-1)/2}$ in the power basis, **even at repeated roots**. The trace pairing, by contrast, has determinant $\text{Disc } P$ and can degenerate. This is an algorithmic distinction, not a curiosity: a residue calculation by remainders stays regular through a root collision even where formulas using the individual weights $1/P'(\alpha_i)$ develop apparent singularities.

The design policy that follows is to *compute symmetric invariants, remainders and finite-algebra traces before requesting labelled root expansions*.

10.3 An analytic observable in a polynomial quotient, by two routes

For $P = z^2 - t$, the relation $z^2 = t$ gives the formal reduction $e^z \bmod P = A(t) + zB(t)$ with

$$A(t) = \sum_{n \geq 0} \frac{t^n}{(2n)!}, \quad B(t) = \sum_{n \geq 0} \frac{t^n}{(2n+1)!},$$

both strongly summable at infinitesimal t . Hence

$$\lambda_P(e^z) = B(t) = 1 + \frac{t}{6} + \frac{t^2}{120} + \frac{t^3}{5040} + \cdots = \frac{\sinh^\sharp(\sqrt{t})}{\sqrt{t}}. \quad (10.2)$$

The same value arises by the contour and residue route. With $M(z) = e^z/(z^2 - t)$ and $s = t^{1/2}$, the poles $\pm s$ have residues $e^s/(2s)$ and $-e^{-s}/(2s)$, each of which is *infinite*; their sum is nevertheless the exact finite value

$$\frac{1}{2\pi i} \int_{|z|=1}^H M(z) dz = \frac{\sinh s}{s} = \sum_{n \geq 0} \frac{t^n}{(2n+1)!} \quad (10.3)$$

[1, 2]. Here the exponential on the infinitesimal poles is the canonical lift and the contour is the ordinary circle with the coefficientwise functional of [section 11.2](#).

Both routes survive because they teach different lessons. The quotient computation avoids both a branch choice for the intermediate square root and the subtraction of two infinite leading residue weights. The contour computation shows that a correct CAS must combine the exact residue expressions *before* applying standard part or coarse truncation: treating the individual residues as generic infinities produces an indeterminate expression and destroys a computable answer, whereas keeping their algebraic dependencies makes the cancellation automatic. For more general analytic observables an exact remainder constructor may be supported by the coherent division theorem and a certificate rather than by a finite polynomial reduction alone.

10.4 Normal forms, multiplication matrices, and the perturbed residue

For a leading isolated complete intersection $f = (f_1, \dots, f_n)$ with $\mu = \dim_{\mathbb{C}} R_n/(f_1, \dots, f_n) < \infty$ and a positive-support perturbation $F = f + E$, AG proves preservation of the finite algebra length and supplies multiplication matrices, residues and trace identities [6]. This is an attractive CAS target because the result is represented by finitely many matrices even when their entries are exact lazy Hahn series.

Choose a monomial basis $b_1, \dots, b_\mu$ of the leading quotient and write an ordinary splitting $G = i p(G) + \sum_j f_j h_j(G)$ with $pi = \text{id}$ and $hi = 0$. For an effective implementation, p and the h_j must be *algorithms* for the chosen coefficient-germ presentation; AG's arbitrary existence of a linear splitting is sufficient for its theorem but not for a program. With $T = \sum_j E_j h_j$, the normal-form coordinates are

$$\text{NF}_F(G) = p(\text{id} + T)^{-1}G = \sum_{r \geq 0} (-1)^r p T^r G, \quad (10.4)$$

and the matrices of multiplication are

$$(M_H)_{rs} = \text{NF}_F(H b_s)_r. \quad (10.5)$$

A finite-grid support backend and a constructive ordinary splitting give effective coefficient access to every entry; matrix multiplication, determinant, trace and characteristic polynomial then reduce to the corresponding scalar operations, none of which requires isolating any perturbed root.

The perturbed residue is

$$\text{Res}_F(H) = \sum_{k \in \mathbb{N}^n} (-1)^{|k|} \text{res}_{f^{k+1}}(H E_1^{k_1} \cdots E_n^{k_n}), \quad (10.6)$$

with support in $\text{supp } H + E^*$. At a requested exponent an effective dependency routine reduces the calculation to finitely many ordinary residues; when the leading denominators are coordinate powers those ordinary residues are simply Taylor coefficient extraction, and more general finite-dimensional residues have established algebraic computation methods [22]. The trace identity is

$$\text{Res}_F(H J_F) = \text{Tr}(M_H), \quad J_F = \det \left(\frac{\partial F_i}{\partial z_j} \right), \quad (10.7)$$

giving both a computation and a consistency check; for $H = 1$ the answer is the ordinary integer μ . A finite-algebra object should record basis, multiplication matrices, defining relations, a residue vector or pairing matrix, and enough reduction data to verify the relations — a representation that retains the multiplicities and nilpotents a list of distinct points would lose.

Remark 10.1 (Verification is not installation). Verifying the Jacobian trace identity in a worked example does not install it as a universal rewrite rule for all coherent quotients, and HC distinguishes its proved multidimensional residue pairing from a general identification of every localized functional with a chosen Grothendieck-residue theory. A software implementation should retain that boundary.

10.5 Length preservation and the length-drop counterexample

AG's length-preservation theorem concerns n equations in n variables with an isolated leading complete intersection. **An overdetermined system is not covered.** AG's own example $(z^2, 0) \mapsto (z^2, t^\eta z)$ drops the length from two to one. An implementation must *test or demand* the isolated-complete-intersection certificate rather than applying a conservation rule to every infinitesimal perturbation [6].

10.6 The coupled system at two independent scales

Both articles that treat this example imported it from their own supplied manuscript sets — one from HC, the other from AG and CT — and they present it at different generality. The two-scale form is strictly more informative and its higher-rank instantiation is the whole point, so we state it as the general case and record the single-scale specialization separately.

The general two-scale form. Consider

$$F_1 = x^2 - Ay, \quad F_2 = y^2 - Bx, \quad A, B \neq 0 \text{ infinitesimal}, \quad (10.8)$$

with basis $(1, x, y, xy)$ and $q = AB$. The coordinate multiplication matrices are

$$M_x = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & q \\ 0 & A & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}, \quad M_y = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & B & 0 \\ 1 & 0 & 0 & q \\ 0 & 1 & 0 & 0 \end{pmatrix}. \quad (10.9)$$

They commute and satisfy $M_x^2 = AM_y$ and $M_y^2 = BM_x$, with characteristic polynomials

$$Q_x(T) = T(T^3 - A^2B), \quad Q_y(T) = T(T^3 - AB^2).$$

All of these are finite polynomial calculations. The four roots are the origin and

$$(\rho\zeta^j, \rho^2\zeta^{2j}/A), \quad j = 0, 1, 2, \quad \rho^3 = A^2B,$$

for ζ a primitive cube root of unity, with nonzero coordinate valuations $(2v(A) + v(B))/3$ and $(v(A) + 2v(B))/3$. These roots require an algebraic or exponent extension *even though the multiplication matrices do not*; keeping the finite algebra avoids forcing that extension before computing traces or total residues.

The Jacobian is $J_F = 4xy - AB$, and the simple residue weights for the constant numerator are

$$-\frac{1}{AB}, \quad \frac{1}{3AB}, \quad \frac{1}{3AB}, \quad \frac{1}{3AB},$$

whose sum is **exactly zero** although each individual weight is infinite, while $\text{Res}_F(xy) = 1$ and $\text{Res}_F(J_F) = 4$. A truncation or floating-point specialization that loses cancellation can therefore fail on a quantity with an especially simple exact answer. For a monomial numerator the closed formula is

$$\text{Res}_F(x^p y^r) = \begin{cases} A^{(2p+r-3)/3} B^{(p+2r-3)/3}, & \frac{2p+r-3}{3}, \frac{p+2r-3}{3} \in \mathbb{N}, \\ 0, & \text{otherwise.} \end{cases} \quad (10.10)$$

Taking $A = t$ and $B = u = t^\omega$ preserves the higher-rank distinction without changing the matrix algorithms.

The single-scale specialization. Setting $A = B = t$ gives $F_1 = x^2 - ty$, $F_2 = y^2 - tx$, with the same basis, dimension four, roots $(0, 0)$ and $(t\zeta, t\zeta^2)$ for $\zeta^3 = 1$, matrix entries in t and t^2 , and characteristic polynomial $X^4 - t^3X$. *This is the form in which the corresponding symbolic identities were actually executed* in the layered-representations archive ([section 18](#)). The residue pairing there has nonzero constant determinant despite infinite individual evaluation weights.

Both residue-pairing results survive because they are different: the residue-versus-trace determinant distinction of [section 10.2](#), and the Gram matrix of determinant 1 together with the vanishing of the summed infinite weights above.

More generally, HC treats coupled coherent systems $F_i = w_i^{m_i} + E_i$ with positive-support errors on one ordinary product domain, constructing a finite-free algebra of rank $\prod_i m_i$ with normal forms supported in $S + E^*$ [5]. This motivates a lazy `CoherentQuotientAlgebra` constructor whose multiplication matrices are computed coefficientwise. It is **not** a generic theorem for arbitrary analytic systems or arbitrary ordinary leading complete intersections; the source explicitly delimits those extensions.

11 Integration and contour operations

11.1 Primitives by category

A formal analytic germ $\sum a_n X^n$ has the normalized formal primitive $\sum_{n \geq 0} a_n X^{n+1}/(n+1)$, representable by a new coefficient generator with the same local semantic obligations. A Laurent germ with finite principal part is handled termwise, except that an X^{-1} term requires a logarithm descriptor or remains a formal logarithmic primitive; the residue is simply the coefficient of X^{-1} . In a formal power-series ring of characteristic zero, termwise integration with zero constant term is effective when the coefficients are.

Rational integration in a formal variable z over a characteristic-zero coefficient field is a well-defined algebraic problem: partial fractions show that polynomial terms and higher-order poles have rational primitives while a simple pole contributes a logarithmic obstruction, so a rational differential has a rational primitive exactly when all its simple-pole residues vanish, after passing to an algebraic splitting field as necessary. For rational functions over an effective complexified core, polynomial division and finite partial fractions give effective residue computations once algebraic roots and multiplicities are available. An antiderivative need not remain rational; logarithms and their branch information must be added.

For common-domain Hahn families, SA defines differentiation and integration coefficientwise, and a **coherent primitive exists precisely when the ordinary closed-cycle Hahn periods vanish**. The same manuscript exhibits fine-locally constant functions with zero derivative but different values on different fine-open pieces. *There can therefore be no unrestricted endpoint integral satisfying the fundamental theorem for every fine-locally analytic function* [2, 7].

A proposed integration API should distinguish a rational primitive, an analytic-germ primitive, a coherent primitive on a stated domain, and an antiderivative for a specified surreal derivation. **A successful result in one category is not a result in any other category.** Additive constants can be normalized locally, but a global primitive also needs compatibility across the relevant domain. In a broader analytic coefficient domain a definite integral may be stored exactly without a closed-form reduction: existence, representability by an integral node, and reduction to elementary functions are separate outcomes. Closure of the whole surreal field under a chosen antiderivation is not a universal finite algorithm on arbitrary inputs.

11.2 The coefficientwise contour functional, and what does not justify it

For an ordinary piecewise smooth path γ in U and common-domain Hahn data the sources define

$$\int_{\gamma}^H G(z) dz = \sum_{\lambda} t^{\lambda} \int_{\gamma} g_{\lambda}(z) dz, \quad (11.1)$$

with the result's support contained in that of G . Each ordinary coefficient integral can be evaluated by the underlying CAS when possible, or retained as an exact integral descriptor; an effective infinite coefficient family again needs an effective coefficient procedure. On a simply connected ordinary domain, normalized primitives are constructed coefficientwise; on a multiply connected domain, periods matter. Nonstandard endpoints add SA's Taylor corrections to ordinary endpoint integrals.

Remark 11.1 (Scope of (11.1)). This is **the supplied manuscripts' functional**, not a newly asserted full-class Riemann integral, and it is explicitly *not* justified by taking Riemann sums in the surreal fine topology: that topology does not make ordinary nonconstant paths into fine-continuous paths, and the contour in (11.1) is not a fine-continuous real-parameter path through the surcomplex class [1, 7]. Turning (11.1) into a computable scalar coefficient needs an ordinary exact or validated integration method for that coefficient class; *exchanging a general numerical quadrature result for an exact coefficient would change the contract*. A pole on the contour is not a zero winding number; it is outside the definition unless a separate regularization has been selected. The system must not infer a Cauchy theorem merely from the fact that a path is closed: null-homology or an appropriate residue calculation is necessary, and the data type of a contour must include orientation and pole-avoidance conditions.

Costin and Ehrlich's work on surreal extension and integration is established background for a large class of resurgent functions and a relevant published direction [17]. It is **explicitly not identified** here with the local Taylor-lift interface of section 9.2 or with the coefficientwise functional (11.1), and it is not a license to identify every formal asymptotic series with a unique global surcomplex function.

11.3 Algebraic rational periods and winding by certified crossings

For a rational differential and a finite semialgebraic cycle avoiding its poles, CT defines an algebraic period

$$I_C^{\text{alg}}(R dz) = 2\pi i \sum_{a \text{ a finite pole}} w_F(C, a) \text{res}_a(R dz), \quad (11.2)$$

where the winding number can be obtained from certified ray crossings or appropriate real-algebraic index computations [7]. This is a practical arbitrary-scale operation when the scalar ordered field is effective.

Remark 11.2 (The normalized period may stay algebraic). $I_C^{\text{alg}}/(2\pi i)$ may remain in an algebraic coefficient domain even when multiplying by $2\pi i$ requires adjoining a named transcendental constant. CT states agreement with Hahn integration on their common admissible contours.

11.4 Microscopic rescaling

CT rescales a radius-free germ by $z = a + t^s w$, with a infinitesimal and $s > 0$. At each Hahn exponent the resulting coefficient in w is an ordinary polynomial, because only finitely many Taylor degrees contribute. **This supplies a common ordinary domain *after* rescaling where none**

existed before — exactly the situation of the radius-free germ (9.11). The support certificate, rather than a guessed radius, is what authorizes microscopic Cauchy and residue operations [7].

11.5 The separating torus, and why the determinant matters

For the finite algebra of section 10, form the coordinate characteristic polynomials

$$Q_i(T) = \det(TI - M_{z_i}) = T^\mu + \sum_{j < \mu} q_{ij} T^j.$$

Using support-controlled normal forms, CT obtains a matrix C with $Q_i(z_i) = \sum_j C_{ij} F_j$. Choose positive s_i satisfying

$$(\mu - j)s_i < v(q_{ij}) \quad \text{for each nonzero } q_{ij}, \quad (11.3)$$

and put $r_i = t^{s_i}$. CT's separating-torus formula is

$$\text{Res}_F(H) = \frac{1}{(2\pi i)^n} \int_{\mathbb{T}_r}^H \frac{H \det C}{\prod_i Q_i(z_i)} dz_1 \wedge \cdots \wedge dz_n, \quad (11.4)$$

the torus being parameterized by $z_i = r_i e^{i\theta_i}$ with ordinary angular parameters. Formula (11.4) is imported *conditionally* from CT's stated finite-algebra and transformation hypotheses [7].

Why $\det C$ cannot be dropped. For the system (10.8) take

$$C = \begin{pmatrix} x^2 + Ay & A^2 \\ B^2 & y^2 + Bx \end{pmatrix}, \quad C \begin{pmatrix} F_1 \\ F_2 \end{pmatrix} = \begin{pmatrix} x^4 - A^2 Bx \\ y^4 - AB^2 y \end{pmatrix},$$

so that

$$\det C = x^2 y^2 + Bx^3 + Ay^3 + ABxy - A^2 B^2. \quad (11.5)$$

The *separated* system has sixteen simple grid points, while the original system has four. At a grid point that does not solve the original system, $CF = 0$ with $F \neq 0$, forcing $\det C = 0$, so the extraneous simple residue vanishes. Dropping the determinant would compute the wrong invariant. At the algebraic level, expanding the reciprocals,

$$Q_x(x)^{-1} = x^{-4} \sum_{k \geq 0} (A^2 B)^k x^{-3k}, \quad Q_y(y)^{-1} = y^{-4} \sum_{\ell \geq 0} (AB^2)^\ell y^{-3\ell},$$

which are strongly supported on an admissible scaled torus; extracting $x^{-1}y^{-1}$ after multiplying by $H \det C$ reproduces (10.10). This is a finite coefficient calculation, not a numerical integration over a fine-topological torus.

The pipeline is therefore concrete: compute multiplication matrices, characteristic polynomials, an ideal-membership transformation matrix and the valuation inequalities (11.3); then reduce the integral to coefficient extraction. The original denominators F_i need not avoid the chosen product torus, and a real n -torus is not the whole real $(2n - 1)$ -dimensional boundary of a polydisk. These are mathematical conditions that a symbolic simplifier can otherwise accidentally erase.

11.6 Stokes as a certificate-based rewrite

CT's Hahn differential forms have ordinary smooth coefficient forms with a common well-ordered support. Exterior derivative, wedge product, admissible pullback and integration along compact ordinary parameter chains satisfy the coefficientwise Stokes identity, and positive-support deformations enlarge the class of chains under stated analyticity conditions [7].

A CAS can use this as a certificate-based rewrite: replace the integral of $d\omega$ over a certified chain by the integral of ω over its oriented boundary, or prove invariance under a certified pole-avoiding homotopy. This is **a proposed use of an identified theorem, not an algorithm deciding whether every arbitrary input map is admissible**. Nor does definable Jordan separation alone furnish a general field-valued integration operation.

11.7 Three circles that a single real precision parameter would confuse

In $\Gamma = \mathbb{Q}\omega + \mathbb{Q}$ take

$$R(z) = \frac{1}{z-t} + \frac{1}{z-u}, \quad u = t^\omega.$$

A positively oriented microscopic circle $|z| = t^s$ has normalized period

$$\frac{1}{2\pi i} \int_{|z|=t^s}^H R(z) dz = \begin{cases} 2, & s = 1/2, \\ 1, & s = 2, \\ 0, & s = 2\omega. \end{cases} \quad (11.6)$$

A pole t^a is inside exactly when $a > s$, and both residues equal one, so the computation reduces to *two exact exponent comparisons*. The example is from CT and is independently checked at the valuation-vector level (section 18). The rational-monomial core already supplies the essential order operations, while a complete contour wrapper would still need its chain and pole-avoidance certificates.

11.8 Coherent primitives, fine-local constants, and a category warning

Coherent primitives exist exactly when the ordinary closed-cycle Hahn periods vanish. SA's fine-locally constant functions have zero derivative yet take different values on different fine-open pieces.

Remark 11.3 (The function category is part of the theorem's type). A CAS must **not** combine SA's global fine-analytic logarithm (section 13.6) with a coherent fundamental theorem to deduce that the integral of dz/z around a loop is zero. The global logarithm is a right inverse of SA's exponentials and has derivative $1/z$, but is *not* a Hahn-coherent primitive on the punctured ordinary base. The function category is part of the theorem's type, and the order-geometric route of section 8.1 does not create a fine-continuous contour integration routine either.

12 Exponentials, logarithms, and transserial scale semantics

12.1 Gonshor exp and log as exact constructors

The established surreal exponential $\text{Exp}_{\mathbf{No}} : \mathbf{No} \rightarrow \mathbf{No}_{>0}$ and its inverse $\text{Log}_{\mathbf{No}}$ are canonical structures: an ordered-group isomorphism from $(\mathbf{No}, +)$ onto $(\mathbf{No}_{>0}, \cdot)$ extending the ordinary real

exponential and logarithm [9, 10]. Hence finite expressions such as

$$\text{Exp}_{\mathbf{No}}(\omega), \quad \text{Log}_{\mathbf{No}}(\omega + 1), \quad \text{Exp}_{\mathbf{No}}(\omega + \omega^{-1})$$

have exact meanings, and a finite node $\text{Exp}_{\mathbf{No}}[x]$ has an exact meaning whenever x does. The fact that a chosen implementation cannot yet expand one of these expressions does not make its denotation inexact. For the last example the exact group law gives

$$\text{Exp}_{\mathbf{No}}(\omega + \omega^{-1}) = \text{Exp}_{\mathbf{No}}(\omega) \sum_{n \geq 0} \frac{\omega^{-n}}{n!}, \quad (12.1)$$

useful as a structured monomial-scale expression even without normalizing the first factor in the smallest Hahn backend. *Supporting the constructor is easier than supporting a complete zero, comparison or normal-form algorithm for every expression built from it.*

12.2 The exp and log splits, and why the naive series fails

Writing $x = p + r + \eta$ with p purely infinite, $r \in \mathbb{R}$ and η infinitesimal,

$$\text{Exp}_{\mathbf{No}}(x) = \text{Exp}_{\mathbf{No}}(p) e^r \text{Exp}_{\text{inf}}(\eta), \quad \text{Exp}_{\text{inf}}(\eta) = \sum_{n \geq 0} \frac{\eta^n}{n!}. \quad (12.2)$$

The infinitesimal factor is directly computable by support-controlled expansion. The purely infinite factor requires Gonshor-compatible monomial or transseries machinery, or must remain an exact semantic node. **Defining it by $\sum p^n/n!$ would be invalid:** for nonzero purely infinite p the term valuations decrease. Correspondingly, the compatibility identity $\text{Exp}_{\mathbf{No}}(x + \epsilon) = \text{Exp}_{\mathbf{No}}(x) \sum_n \epsilon^n/n!$ for infinitesimal ϵ does *not* say that the normal form of $\text{Exp}_{\mathbf{No}}(x)$ can be computed by the ordinary exponential Taylor series at an arbitrary infinite x .

For a positive normal form $a = ct^\gamma(1 + \eta)$ with $c > 0$,

$$\text{Log}_{\mathbf{No}}(a) = \log c + \text{Log}_{\mathbf{No}}(t^\gamma) + \text{Log}_{\text{inf}}(1 + \eta), \quad \text{Log}_{\text{inf}}(1 + \eta) = \sum_{n \geq 1} \frac{(-1)^{n+1} \eta^n}{n}. \quad (12.3)$$

The middle term is *not* a licence to conflate the Conway monomial map with arbitrary exponential powers (section 2.2); a backend must implement the applicable monomial-logarithm identities or retain that term as an atom.

For an effective implementation, the exponential should decompose the argument into its large, constant and small parts, apply the small-part series where valid, and introduce or recognize a new growth monomial for the large part. Normalization, comparison and logarithmic inversion then require a transseries or surreal-exponential backend, not just a new display head. A fixed Hahn workspace containing the input may not be closed under these transcendental operations; closure under algebraic roots in a divisible complex Hahn field is a separate issue. Bournez and Guilmant study set-sized surreal fields stable under exponential, logarithmic, derivative and antiderivative operations [21]; their structural closure conditions help choose mathematical domains, but *a bound on birthdays is not by itself a finite executable representation or a complexity estimate.*

12.3 An exact exp–log expression language

One practical language starts with a decidable algebraic core and permits $\text{Exp}_{\mathbf{No}}$ and $\text{Log}_{\mathbf{No}}$ on certified positive arguments, plus positive-base real powers. Its expression graphs are finite and

denotationally exact; it can simplify group identities, cancel inverse functions on their domains, extract known leading scales, and perform infinitesimal expansions.

It should not claim that every equality between arbitrary such graphs has a known terminating decision procedure. Broadening the ordinary constant domain already complicates equality before any surreal exponent is introduced. A partial simplifier can return a smaller equivalent graph together with a proof of its rewrites, without producing a canonical normal form for the entire language.

The backend should distinguish a **structural constructor** from a **normalization request**. The former can often succeed immediately; the latter may return a normalized object, a conditional result, a residual exact node, or a resource-limit result. Conflating these operations either rejects useful exact expressions unnecessarily or encourages unsupported canonicalization claims. It should also keep positive-real powers $\text{Exp}_{\mathbf{No}}(b \text{Log}_{\mathbf{No}} a)$ distinct from Conway monomial notation, and have a documented rewrite theory with a declared transseries grammar when one is used.

12.4 Lambert W as an exact implicit constructor

There are useful exact constructors beyond a fixed list of elementary functions. A positive-real Lambert function can be specified by

$$W_+(y) \geq 0, \quad W_+(y) \text{Exp}_{\mathbf{No}}(W_+(y)) = y, \quad y \geq 0.$$

The ordinary real existence statement is first order in the ordered exponential language, so the elementary-extension result supplies existence for nonnegative surreal y as well [10]. Uniqueness follows directly: the map $x \mapsto x \text{Exp}_{\mathbf{No}}(x)$ is strictly increasing for $x \geq 0$, by positivity and the strict increase of the real exponential.

Consequently a selected implicit-root node gives an exact value such as $W_+(\omega)$ even before a normalizer has computed its logarithmic-scale expansion; a transseries inversion algorithm can provide that expansion within its supported class. *The existence and uniqueness argument is a certificate for this particular function, not a generic transcendental root-finding algorithm*; general equations without comparable certificates remain conditional or partial.

12.5 Transseries backends and their published antecedents

An exponential–logarithmic monomial hierarchy, small-monomial grids, coefficient algorithms and support-preserving substitutions provide a more systematic backend than ad hoc rewrite rules. It can expose formal differentiation, integration within its supported differential class, composition for admissible large positive arguments, and functional inversion under the appropriate hypotheses, with extensive mathematical antecedents [14]. The embedding into surreal numbers matters: it identifies the formal large variable with ω and specifies the exponential structure. A set of independent formal labels called $\log \omega$ and e^ω without their structural relations is not the same ordered exponential field.

Three pieces should be recorded separately: the abstract transseries expression, the chosen realization in a surreal field, and the permitted function-composition or differential structure. Berarducci and Mantova construct surreal derivations and study transseries as surreal function germs [15, 16]. Mantova’s 2026 preprint, in its May 9 revision, establishes monotonicity of omega-series composition in its second argument and a Taylor approximation theorem, with an intermediate-value consequence for the interpreted functions [19].

Remark 12.1 (Scope of the transseries results). These are **semantic theorems for specified classes**, not universal decision algorithms. They do not justify substituting arbitrary surreals into every transseries expression without checking the chosen framework’s hypotheses, do not identify every arbitrary Hahn series with a globally composable function on all of $\mathbf{No}$, and are not a claim that arbitrary CAS expressions or arbitrary coefficient programs can be normalized. *The entire mathematical field of grid-based transseries is not a finite-input format when arbitrary coefficient arrays are allowed.* A backend’s scope should be stated by grammar and algorithms, not by saying that it “supports all transseries”; terms outside the supported class can remain exact formal names with reduced capabilities, provided their mathematical meaning has actually been fixed. A reasonable first target is a finite-depth exponential–logarithmic tower with explicit dominant-term certificates. Hyperseries [18] are a longer-term representational direction, not a prerequisite for a first useful implementation.

12.6 Asymptotic data do not determine exact values

The real functions $f(x)$ and $f(x) + e^{-x}$ have the same asymptotic expansion in inverse powers of x whenever the first expansion exists, because e^{-x} is smaller than every such power as $x \rightarrow +\infty$. Under a selected compatible surreal extension, however, the additional term at $x = \omega$ is a *nonzero infinitesimal*, namely $\text{Exp}_{\mathbf{No}}(-\omega)$, not zero. Knowing all algebraic orders of an asymptotic expansion therefore does not by itself select one exact value.

An exact extension of a special function at infinity must state how exponentially small data, sectors, branches and any summation choices are fixed. Similarly the strong Hahn value $\sum n! t^n$ is *not* automatically a Borel sum, a lateral resummation, or the value of a particular analytic function at $1/t$; such identifications require a summation and extension theorem and can carry sector and Stokes data. An exact formal Hahn descriptor and an exact analytic-function descriptor must remain distinct objects.

12.7 The factorial series and its exponentially small homogeneous solution

The exact formal-germ value $F(t) = \sum_{n \geq 0} n! t^n$ is meaningful by Hahn summability and satisfies the coefficient identity

$$t^2 D_t F + (t - 1)F = -1. \quad (12.4)$$

Indeed, at degree $n \geq 1$ the contribution is $(n - 1)(n - 1)! + (n - 1)! - n! = 0$, and the constant term is -1 . This provides an efficient recursive coefficient descriptor. As an ordinary complex power series it diverges, which does not prevent its strong Hahn value but does prevent identifying it, without further choices, with an ordinary analytic germ at zero.

The associated homogeneous equation has the solution $e^{-1/t}/t$ on positive ordinary t , as direct differentiation verifies, and it is smaller there than every power of t . An asymptotic power-series solution therefore cannot detect the addition of a multiple of this homogeneous term. In a Gonshor-compatible infinite-scale interpretation at $t = \omega^{-1}$, the corresponding positive term is $\omega \text{Exp}_{\mathbf{No}}(-\omega)$, **not zero**. Analytic solutions sharing the same expansion may thus differ invisibly to every power of t .

Two further cautions attach to this example. It does not specify a unique ordinary analytic function at positive real t by convergence of that series. And if a scalar derivation with $\partial t = -t^2$ is chosen, the displayed equation is *not* obtained by blindly replacing D_t by ∂ (section 14). The example simultaneously tests summation semantics and derivative semantics.

12.8 All-scale rigidity as a category boundary

SA proves, under its explicit all-scale coherence definition, that functions coherent entire at *every surreal radius* are precisely the polynomials, and that the analogous all-scale meromorphic functions are precisely the rational ones. The quantifier ranges over all surreal scales and uses the manuscript’s uniform ordinary-coefficient chart condition [1, 2].

Remark 12.2 (What rigidity does and does not say). The correct conclusion is a **separation of categories**, not a prohibition on transcendental surreal functions and not a universal nonexistence claim. A package that admits a global exponential should not also label it “all-scale Hahn-entire” in that sense. A global special-function object should instead advertise local germs, selected coherent charts, a transserial interpretation at infinity, or a sectorial extension provider, with the compatible charts and transition rules part of the object. No claim is made here that any one such global atlas has already been implemented.

Remark 12.3 (Sharpness over a fixed value group). Two companion reports in this repository bear on the boundary above. The rigidity theorem for class functions on all of $\mathbf{No}[i]$ is stated in `docs/surcomplex/analysis/`; the sharpness witness below is recorded not there but in `docs/foundations-and-computation/foundations/`, whose worked example carries the support proof. Over the *fixed* value group of $K = \mathbb{C}((t^{\mathbb{Q}}))$, the function $f(z) = \sum_{n \geq 0} t^{n^2} z^n$ is nonpolynomial yet has a Hahn-coherent chart on the whole ordinary complex plane at every positive radius in $\mathbb{R}((t^{\mathbb{Q}}))$. The rigidity proof cannot run there because it requires a bound Δ with $|v(a_n)| < \Delta/4$ for all n , while $v(a_n) = n^2$ is unbounded in $\mathbb{Q}$; the proper class of multiplicative scales available for class functions on the full $\mathbf{No}[i]$ supplies an infinite Δ , and the fixed value group does not. This is a **sharpness result about the scope of the theorem**, not a correction to it: it shows that the all-scale rigidity statement is specific to the full-class scale system. A CAS that works inside one fixed set-sized parent is in the second situation, not the first, and must not invoke rigidity to conclude that its nonpolynomial coherent objects cannot exist.

12.9 Oscillation is a separate layer

Oscillatory functions deserve their own layer. An expression such as $\sin x$ at large positive real x does *not* have an eventual sign in the same sense as a nonzero ordered logarithmic–exponential transseries. Treating oscillation as just one more positive ordered monomial would lose its essential behaviour. For infinite surreal arguments the extension convention of [section 13.9](#) must also be addressed.

13 Surcomplex arithmetic, angles, branches, and infinite phases

13.1 Pairs and squared modulus

Represent $z = x + iy$ as a pair of real-surreal objects with a common compatible domain. Then

$$\begin{aligned} (x + iy)(u + iv) &= (xu - yv) + i(xv + yu), \\ \bar{z} &= x - iy, \quad |z|^2 = x^2 + y^2, \\ z^{-1} &= \frac{x - iy}{x^2 + y^2} \quad (z \neq 0). \end{aligned}$$

If a real domain is an effective ordered field F , pairs implement $F[i]$ with these formulas. The squared norm lies in the real base field; the norm $|z| = \sqrt{x^2 + y^2}$ may require its real closure, which

is why a minimal rational-monomial package implements *squared* modulus but not unrestricted modulus. Conjugation, real and imaginary parts, and squared modulus are finite field operations; integer powers stay in the field, and rational powers generally require algebraic extension and root selection.

No ordering should be imposed on the complexification as a field: a complex field has no compatible order, and comparisons must specify real part, modulus, valuation, or another intended relation. Real comparison gives **Min**, **Max** and piecewise evaluation when all required signs are decidable.

13.2 All directions admit finite surreal angles

TR proves that the unit circle

$$\mathbb{T}(\mathbf{No}) = \{u \in \mathbf{SC} : u\bar{u} = 1\}$$

is the image of canonical cis on *finite* real surreals, with kernel $2\pi\mathbb{Z}$. **Thus every nonzero surcomplex number has a modulus and a finite angle class, even when its coordinates or its modulus are infinite [3].**

For computation, let $u = z/|z|$ and $u_0 = \text{st}(u)$. Then

$$u = u_0 \text{Exp}_{\text{inf}}(i\delta), \quad \delta = -i \text{Log}_{\text{inf}}(u/u_0) \in \mathfrak{m} \cap \mathbf{No}. \quad (13.1)$$

The pair (u_0, δ) avoids any arbitrary angle cut: it is an exact, *cut-free* phase representation separating the ordinary unit direction from the infinitesimal correction. Extracting u_0 still requires the scalar backend's standard-part capability.

13.3 The Cayley coordinate

An entirely algebraic alternative, avoiding angle evaluation, is

$$u(q) = \frac{1+iq}{1-iq} = \frac{1-q^2}{1+q^2} + i \frac{2q}{1+q^2}, \quad q \in \mathbf{No}. \quad (13.2)$$

It parametrizes the circle except the point -1 , which must be checked separately in solving and integration. *Infinite q is allowed and describes the monad of the omitted direction.* The Cayley coordinate is used twice below: as a cut-free exact circle parametrization here, and as a reduction of circle positivity to a semialgebraic question over the real-closed core in [section 13.11](#).

13.4 Canonical strips for exponential and trigonometry

Write

$$\mathcal{H} = \{x + iy : y \text{ finite}, x \in \mathbf{No}\}, \quad \mathcal{V} = \{x + iy : x \text{ finite}, y \in \mathbf{No}\}.$$

TR's canonical strip exponential is

$$E_{\mathcal{H}}(x + iy) = \text{Exp}_{\mathbf{No}}(x) \text{cis}(y), \quad (13.3)$$

which is onto $\mathbf{SC}^\times$ with kernel $2\pi i\mathbb{Z}$. On $\mathcal{V}$, canonical sine and cosine are

$$\text{Sin}(x + iy) = \sin x \cosh y + i \cos x \sinh y, \quad (13.4)$$

$$\text{Cos}(x + iy) = \cos x \cosh y - i \sin x \sinh y, \quad (13.5)$$

with the real hyperbolic functions defined via $\text{Exp}_{\mathbf{No}}(\pm y)$. These formulas admit exact symbolic nodes even at infinite y , conditional only on the scalar backend's capabilities. In particular

$$\text{Sin}(i\omega) = i \sinh \omega$$

is **canonical in this framework**, whereas $\sin \omega$ and $E(i\omega)$ are **not** fixed by its canonical finite-angle data. The distinction is about the real circular phase, not merely about whether the argument is large [3].

13.5 Inverse trigonometry, and why an infinite argument needs no infinite phase

The inverse functions have the natural real-surreal domains $\arctan : \mathbf{No} \rightarrow (-\pi/2, \pi/2)$, $\arcsin : [-1, 1] \rightarrow [-\pi/2, \pi/2]$ and $\arccos : [-1, 1] \rightarrow [0, \pi]$. For positive infinite x ,

$$\arctan x = \frac{\pi}{2} - \arctan(1/x),$$

so an exact finite-angle value is available without defining circular sine at x itself. Concretely,

$$\arctan \omega = \frac{\pi}{2} - \arctan t = \frac{\pi}{2} - t + \frac{t^3}{3} - \frac{t^5}{5} + \frac{t^7}{7} + O_v(9),$$

with $t = \omega^{-1}$: *an infinite argument causes no infinite phase choice*, unlike a request for $\sin \omega$. Endpoint branches must be explicitly selected; (8.9) is the endpoint example, requiring the positive square-root uniformizer.

13.6 Two logarithm branches must not share one cache key

An *actual principal-angle* convention chooses a representative in $(-\pi, \pi]$. SA instead also defines a *standard-direction-anchored* logarithm

$$\mathcal{L}(z) = \text{Log}_{\mathbf{No}} |z| + i \text{Arg}(u_0) + \text{Log}_{\text{inf}}(u/u_0), \quad u = z/|z|, \quad u_0 = \text{st}(u), \quad (13.6)$$

where $\text{Arg}(u_0)$ is an ordinary principal argument. The ordinary argument convention is part of the definition. This function is fine-locally analytic on the full punctured class plane with finite imaginary part, and is a right inverse of SA's E_α (section 13.9); it is **not** a coherent primitive of $1/z$ around the ordinary puncture [1, 2]. See theorem 11.3.

The two choices differ at infinitesimal crossings of the ordinary branch cut. For $\eta > 0$ infinitesimal and $z = -1 - i\eta$:

$$\text{actual principal angle} = -\pi + \arctan \eta, \quad \text{anchored angle} = \pi + \arctan \eta,$$

a difference of exactly 2π . Here $|z| = \sqrt{1 + \eta^2}$ and $\text{Log}_{\mathbf{No}} |z| = \frac{1}{2} \text{Log}_{\text{inf}}(1 + \eta^2)$, so the real parts agree; the imaginary parts differ by 2π ; and both exponential images under (13.3) equal z . The anchored value is permitted to lie *infinitesimally above* π and **must not be silently reduced** back into the principal interval while retaining a claim of the same global fine-local analytic behaviour. A branch-keyed cache can store both without inconsistency.

A function descriptor therefore needs a branch policy token such as **ActualPrincipal** or **StandardDirectionAnchored**, and the chosen policy belongs to exact expression identity, domain checks, differentiation and serialization.

Two further logarithm-category warnings survive. The local identity $\log(1 + \epsilon) + \log(1 + \eta) = \log((1 + \epsilon)(1 + \eta))$ is valid for the infinitesimal logarithm at infinitesimal ϵ, η ; a global identity $\log(zw) = \log z + \log w$ is *not* justified by that local calculation, and **PowerExpand**-style rewrites cannot be applied without recording the relevant branch and domain assumptions. A safe simplifier may use $\text{Exp}_{\mathbf{No}}(\text{Log}_{\mathbf{No}} x) = x$ for positive real surreal x , and a chosen surcomplex right-inverse identity where the chosen constructions justify it; it must not use an unrestricted $\log(\exp z) = z$ or cancel arbitrary complex powers without branch hypotheses. Returning a set of roots is often preferable to imposing a hidden principal-root policy on a nonstandard complex plane. A local ordinary logarithm branch, a global fine-local logarithm, and the real positive logarithm are different function types even when their printed name is `Log`.

13.7 General complex powers and the strip obligation

For a selected logarithm L , the expression

$$z^w = E_{\mathcal{H}}(wL(z))$$

is canonical by this formula **only when** $wL(z) \in \mathcal{H}$: its imaginary part need not be finite for arbitrary surcomplex w . Outside that strip one must select a global exponential extension or retain an explicitly unresolved power node.

Integer powers need none of this and should remain algebraic. Ordinary rational powers can be implemented by algebraic root selection and finite-angle branch rules rather than by introducing an unrestricted exponential. For positive real base and real surreal exponent the real definition $\text{Exp}_{\mathbf{No}}(w \text{Log}_{\mathbf{No}} z)$ is available. *A single unqualified **Power** rewrite table cannot safely cover all these cases.*

13.8 Infinite phases I: the character classification (from TR)

Let Π be the additive class of purely infinite real surreals; normal forms give the additive splitting $\mathbf{No} = \Pi \oplus \mathcal{O}_{\mathbb{R}}$, $x = p + \text{fin}(x)$. TR classifies extensions of the circular group law by characters

$$\chi : (\Pi, +) \longrightarrow (\mathbb{T}(\mathbf{No}), \cdot),$$

setting

$$U_{\chi}(p + u) = \chi(p) \text{cis } u, \quad u \text{ finite.} \tag{13.7}$$

The real and imaginary parts give global cosine and sine satisfying the addition laws and TR's fine-local differential identities, and **conversely every extension with this group-law property has that form** — a completeness claim *within TR's framework* [3].

The convention $\chi = 1$ gives $U_0(x) = \text{cis}(\text{fin}(x))$, hence $\sin_0 \omega = 0$. It is an *available convention, not an identity forced by the finite-angle theory*; other characters give different phases at ω . An exact CAS should default to an **ExtensionRequired** result for such a circular phase unless the user selects a provider, and must never silently adopt the trivial character.

A provider must respect additivity across its domain: independently assigning phases to arbitrary related inputs can violate the group law. The phase at $2p$ must be the *square* of the phase at p , and a phase assigned at $p/2$ must be a compatible square root. *A character descriptor is structural data, not a mutable table of unrelated evaluations.*

Proposition 13.1 (Every global extension acquires an infinite period). *Every extension (13.7) has an infinite period in each nonzero purely infinite coset $p + \mathcal{O}_{\mathbb{R}}$.*

Proof. Finite-angle cis is already onto the entire surreal unit circle, so choose finite a with $\text{cis } a = \chi(p)^{-1}$. Then $U_\chi(p + a) = 1$. If $p \neq 0$ the element $p + a$ is infinite, and the homomorphism law makes it a period. This reproduces the obstruction established in TR. $\square$

Consequently a global phase provider **must not also advertise that its only periods are ordinary integer multiples of 2π** ; those two requirements are incompatible in this framework. Exact results computed with different providers must retain their provider identifiers, even when they happen to agree on all ordinary test values.

13.9 Infinite phases II: the E_α family (from SA)

A separate nonuniqueness phenomenon, for a *different* family from a *different* supplied manuscript, is SA's explicit one-real-parameter family of global exponentials. For a real surreal y let $\text{fin}(y)$ retain its nonnegative t -exponent terms and let $\ell(y)$ be its coefficient at $t^{-1} = \omega$. For an ordinary real α put $\theta_\alpha(y) = \text{fin}(y) + \alpha\ell(y)$ and

$$E_\alpha(x + iy) = \text{Exp}_{\mathbf{No}}(x)(\cos^\# \theta_\alpha(y) + i \sin^\# \theta_\alpha(y)) = \text{Exp}_{\mathbf{No}}(x) \exp_{\text{fin}}(i\theta_\alpha(y)). \quad (13.8)$$

According to SA these are all global fine-analytic exponential group homomorphisms extending the ordinary complex exponential and satisfying $E'_\alpha = E_\alpha$. Nevertheless

$$E_0(i\omega) = 1, \quad E_\pi(i\omega) = -1. \quad (13.9)$$

Agreement on ordinary complex numbers, on the group law, and on fine-local analyticity therefore do *not* select a unique value at $i\omega$ [1, 2].

Remark 13.2 (Both accounts stand; neither may be extended into the other). These are two separate statements and this report does not merge them.

- [section 13.8](#) is a *completeness statement about circular group-law extensions within TR's framework*. It must not be extended to SA's exponential family: that would attribute a completeness result to a source that did not supply it.
- [equation \(13.8\)](#) is an *explicit family from SA demonstrating nonuniqueness*, used as a manuscript result and an interface warning. Both articles carrying it state their non-claims explicitly: one says the statement “does not assert that no other published convention can be chosen” — it says only that SA's stated local axioms do not make that choice — and the other says it uses the contrast “as a manuscript result and an interface warning, not as a published uniqueness classification”. **We therefore do not write “the classification of global surcomplex exponentials is the E_α family”.**

Remark 13.3 (Effectivity caveat, given independently by both carriers). Formula (13.8) is not automatically an effective algorithm on every exact Hahn generator. Extracting $\text{fin}(y)$ and $\ell(y)$ is a *support-cut operation*: it is mathematically well defined on normal forms, but need not be computable by scanning an arbitrary coefficient stream. On finite sparse data and suitably certified structured data it is straightforward; on an unrestricted program representation its operational availability must be declared. The representation problem therefore reappears even after the semantic choice has been settled.

Remark 13.4 (The cache-key rule, three ways). All three sources state a version of the same rule, and all three survive.

1. The different E_α extensions **cannot share a cache keyed only on the printed expression** $\text{Exp}[z]$.
2. **A coefficient computed under one phase convention cannot be reused under another**; caches should be keyed by normalized parent, exponent, branch, derivation and extension policy, and memoized normal forms should distinguish an exact result from a precision-limited result and never silently upgrade a guessed leading term to a certificate.
3. Branch, phase, support and derivation identifiers belong in *persistent* cache keys: reusing a result under a changed interpretation is **an exactness bug even when all printed numbers look identical**.

A proposed $\text{SurcomplexExp}[z]$ must therefore restrict its input domain, specify a particular extension policy, or remain unassigned at such arguments, and must not acquire an unqualified value merely by extending ordinary Taylor syntax. The same warning applies to unrestricted trigonometric functions and to expressions such as $E_\alpha(1/z)$ near a microscopic essential singularity.

13.10 Finite trigonometric polynomials and the $2n$ root bound

Let n be an ordinary finite integer and consider, on $\mathcal{V}$,

$$T(z) = \sum_{k=-n}^n c_k E_{\mathcal{H}}(ikz), \quad c_k \in \mathbf{SC}.$$

Setting $U = E_{\mathcal{H}}(iz)$ gives

$$T(z) = U^{-n} P(U), \quad P(U) = \sum_{k=-n}^n c_k U^{k+n}. \quad (13.10)$$

TR proves that the nonzero roots of P correspond, with multiplicity, to the zeros of T modulo 2π in the canonical strip; their number is at most $2n$, and exactly $2n$ when both extreme coefficients are nonzero [3].

This gives an implementable exact solver: build P with the scalar polynomial backend, solve it in the appropriate complex algebraic extension, discard $U = 0$, retain multiplicities, and convert a nonzero U to a strip angle class through a logarithm. That final analytic reconstruction may leave the algebraic coefficient core even when solving for U stays inside it. To select finite real angles, impose $U\bar{U} = 1$; alternatively use the Cayley parametrization (13.2) with a real algebraic solver, remembering the omitted point -1 .

Remark 13.5 (An infinite frequency is not a polynomial degree). An “infinite integer frequency” is not an ordinary polynomial degree. **Replacing n by ω in this algorithm is meaningless.** Infinite-frequency oscillation requires a different function class and, in general, an infinite-phase provider.

13.11 Positivity must see infinitesimal angles

For positive infinitesimal ε , TR’s example

$$T(\theta) = (\cos \theta - \varepsilon)^2 - \varepsilon^4 \quad (13.11)$$

is positive at *every ordinary real angle* but is *negative* at the finite surreal angle $\theta = \arccos \varepsilon$. A positivity test using only ordinary angular samples or standard parts is therefore **unsound** [3].

Two remedies are available for coefficients in the real-closed algebraic core. One reduces circle positivity to a semialgebraic question using the Cayley coordinate (13.2), inspecting the omitted circle point separately. The other uses TR's Fejér–Riesz theorem for a finite algebraic factorization route. Both test the actual ordered field, not only its ordinary shadow.

14 Differentiation: three operators

14.1 The three operators and their worked contrasts

A system must distinguish differentiation of a function in a variable from differentiation of its scalar coefficients.

The coordinate derivative D_z . For a polynomial or coherent function in a variable z ,

$$F(z) = \sum_{\gamma} f_{\gamma}(z)t^{\gamma}, \quad D_z F = \sum_{\gamma} f'_{\gamma}(z)t^{\gamma},$$

so that every t^{γ} is a *coefficient* and $D_z(t^{\gamma}) = 0$; in particular $D_z\omega = 0$ and $D_z(\omega^{-1}) = 0$. This is the derivative used in the supplied analytic manuscripts; it commutes with coefficientwise Hahn assembly while holding monomials constant, and it is the appropriate operation for local holomorphic identities, Jacobians, residues and CT's contour calculus. Concretely,

$$D_z(\omega z^2 + t \sin z) = 2\omega z + t \cos z.$$

The formal parameter derivative D_t (or D_T). For a formal power series in an auxiliary indeterminate T , the formal derivative satisfies $D_T T = 1$. In a rational or Puiseux field with t an independent parameter and constant rational exponent q , $d(t^q)/dt = qt^{q-1}$; this changes valuations and may decrease precision. In a multivariate rational-monomial field, each formal partial derivative treats the other independent variables as constants. Evaluating a formal series at a fixed surreal t produces a *number*, not an ordinary variable on which D_z should suddenly act.

A surreal-field derivation ∂ . A surreal derivation is additional global structure on **No**; the Berarducci–Mantova construction supplies a distinguished differential-field framework compatible with the exponential structure [15]. Normalizing by $\partial\omega = 1$ and putting $t = \omega^{-1}$ gives

$$\partial t = -t^2, \quad \partial(\omega^{-1}) = -\omega^{-2}, \quad \partial(t^q) = -q t^{q+1} \quad (q \in \mathbb{Q}).$$

Contrast this with $D_t(t^3) = 3t^2$ and $\partial(t^3) = -3t^4$, and with $D_z(\omega^{-1}) = 0$.

These are **not contradictory outputs**: they are different operators. Extending a scalar derivation through a Hahn sum requires its own strong-additivity hypotheses. For general monomials with nonconstant surreal exponents one must use the chosen derivation's rules, not an unqualified power rule copied from ordinary integer exponents: the rational-power identity $\partial(t^q) = -q t^{q+1}$ must not be extended to every surreal exponent of the omega-map by treating that exponent as an ordinary constant.

14.2 Three proposed name sets, printed as three proposals

All three sources propose API names for these operators, and none of the three is implemented in any delivered file. We print all three as proposals rather than choosing one.

Operator	Capability-tiers proposal	Layered-representations proposal	Gaussian-prototype proposal
Coordinate derivative	<code>FunctionDerivative</code>	<code>CoordinateD</code>	a separate operator and parent from the surreal derivation
Formal series derivative	<code>FormalSeriesDerivative</code>	<code>FormalD</code>	formal parameter derivative in the declared parent
Surreal derivation	<code>ScalarDerivation</code> , with a mandatory derivation identifier	<code>SurrealDerivation[... , derivationID]</code>	invoked through a separate operator and parent

All three sources agree that converting between the interpretations must be a deliberate operation, that a derivation identifier is mandatory for the third operator, and that automatic use of a native `D` is reasonable only after the chosen coordinate semantics have been established. *A default `D` overload cannot infer all these intentions from a displayed monomial*, and applying a constant-exponent rule when the exponent is itself nonconstant for that derivation can simply be wrong. A package should record whether a symbol is a coordinate, a formal parameter, or a designated surreal constant.

14.3 Differential equations must name their category

A fine-local solution of $f' = f$ with a prescribed value at zero need *not* be globally unique in the entire fine-local category, as the E_α family of [section 13.9](#) shows. A local formal initial-value problem can still be uniquely solved under the usual coefficient-recursion hypotheses. **A solver must state the category in which its uniqueness claim is made.**

15 Host integration: what Wolfram Language supplies and what an adapter must add

15.1 A documentation-based assessment

The following assessment uses official documentation inspected for the source articles on September 21, 2026. It identifies reusable facilities. It is **not** a claim that every function automatically accepts an arbitrary user-defined field, and **not** an exhaustive survey of third-party packages. *No documented native object with all of the arbitrary-Hahn and surreal contracts developed here was identified in the inspected material.*

Facility	What it contributes	What still needs a surreal adapter
Root, AlgebraicNumber, RootReduce	Exact ordinary algebraic coefficients and implicit-root precedents.	Ordered monomial domains, non-Archimedean selectors, custom coefficient semantics; do not borrow ordinary complex root ordering blindly.
Together, Cancel, polynomial routines	Exact fraction and finite polynomial arithmetic. The engine <code>RationalHahn</code> and <code>HahnRational</code> actually use; <code>SurrealCASCore</code> uses none of these, its sparse layer being a hand-built canonical map and its fraction wrapper deliberately unreduced.	Declare the independent monomials and their ordered exponent parent before interpreting the result.
SeriesData, SeriesCoefficient	Finite power and Puiseux series views, coefficient extraction, order tracking.	Arbitrary ordered exponent groups, infinite-prefix handling, Hahn support certificates, exact-tail semantics.
Series, InverseSeries, composition	Ordinary local expansions and reversion, including supported fractional or logarithmic forms.	A proof that the requested surreal evaluation is admissible and that the chosen branch is the intended one.
Asymptotic, AsymptoticSolve	Ordinary asymptotic approximations and local equation expansions.	Exact Hahn denotation, beyond-all-orders data, and selected surreal extension semantics.
DifferentialRoot	A finite ordinary-function representation by linear differential equations plus initial conditions.	Strong evaluation, support and singular-point certificates, coefficient-domain closure.
D, polynomial and matrix routines	Reusable finite algebra, ordinary coefficient derivatives, symbolic coefficient manipulation.	Exact zero tests; identify which variables and monomials are constant for the chosen derivation; not a generic automatic custom-field interface.
UpValues, custom heads, Inactive	Extension mechanisms, controlled symbolic syntax, preserved unevaluated descriptions.	Mathematical validity, termination policy, branch conventions, deliberate numerical behaviour; avoid broad rewrites and implicit parent mixing.
NumericQ, N	Ordinary numerical evaluation protocol.	A separate scale-aware numerical interface; see section 15.5 .
PossibleZeroQ	A documented <i>heuristic</i> facility.	Nothing: it must not be used as a proof of vanishing or nonvanishing at all.
Infinity	Extended-limit arithmetic.	Nothing: it must never be redefined to mean ω .

The representation details are documented in [23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38]. The proposed adapters in the last column are not native-feature claims.

15.2 Do not understate the current scope of Root

Two of the three sources give the same self-correction, in nearly identical words, and it is worth stating plainly because the opposite error is the tempting one. Current documentation distinguishes indexed polynomial-root representations from neighbourhood-based representations, and includes

exact roots of certain transcendental equations and systems [23]. **The assertion “Root can only represent algebraic numbers over $\mathbb{Q}$ ” would be incorrect.** Its ordinary algebraic subset remains particularly useful as an exact coefficient backend.

That said, none of those documented ordinary numerical neighbourhood mechanisms by itself chooses between roots whose standard parts are identical but whose surreal displacements differ, and an ordinary symbolic parameter in **Root** does not automatically acquire the ordering of an infinitesimal. A new **SurrealAlgebraicRoot** can reuse the same general idea of an implicit exact object while supplying its own ordered-domain and branch certificates; native ordinary **Root** objects can still be used for residual algebraic constants.

15.3 **SeriesData** is a view, not the universal storage format

A basic **SeriesData** object stores a finite coefficient list, integer bounds, and a common denominator for the exponent progression, together with omitted-order information, and it supports arithmetic and calculus operations [26]. It is well suited to a finite rational-power jet and to a one-parameter adapter. It is *not* a representation of every well-ordered surreal exponent support; the fact that **Series** can also produce certain logarithmic and other expansions does not change that limitation of the basic exponent-lattice format [28]. Nested **SeriesData** objects can be useful when a fixed iterated expansion convention is supplied; their existence does not automatically define a general multi-rank Hahn truncation policy.

Converting with Normal discards the omitted-order term, and it must not be used internally as a proof that an infinite exact series equals its displayed polynomial. Round-tripping a jet through plain syntax should retain its precision certificate in a wrapper or explicitly announce its loss. A regression suite should check that **Normal** or a similar jet-to-expression conversion cannot silently promote a truncation to an exact equality.

15.4 **PossibleZeroQ** is heuristic

PossibleZeroQ is documented as a heuristic facility [36]. It must **never** serve as the proof of a leading coefficient’s nonvanishing or vanishing. Testing a coefficient with an ordinary numerical heuristic and proceeding as though the result were an exact algebraic fact is unsound (section 8.1).

15.5 **NumericQ**, **N**, and the Archimedean impossibility

Proposition 15.1 (No order-preserving embedding). *There is no order-preserving embedding of an ordered field containing a positive infinitesimal into $\mathbb{R}$ that fixes the rationals.*

Proof. If $0 < \epsilon < 1/n$ for every positive integer n , its image would be a positive real below every $1/n$, contradicting the Archimedean property of $\mathbb{R}$. Arbitrary precision does not change the argument. $\square$

Three distinct numerical requests are nevertheless useful and should be separate operations.

1. **Coefficient approximation:** approximate the ordinary coefficients while retaining exact monomials.
2. **Standard part:** extract the ordinary constant coefficient of a finite value, discarding infinitesimals by definition, and only with a certificate of finiteness.

3. **Scale specialization:** replace finitely many monomials by selected real parameters for a numerical experiment.

Remark 15.2 (A finite-scale specialization is a simulation). The third request is *not* a faithful model of all surreal inequalities: no fixed positive real u stays below every positive power of a fixed $0 < t < 1$. A specialization such as $t_1 = 10^{-6}$ can illustrate a one-scale expression, but it is not its surreal value. For several scales, numerical choices can respect any prescribed *finite* collection of hierarchy inequalities, not the entire statement $t_2 < t_1^n$ for all ordinary n . Results from such a specialization must be **labelled as simulations**. They cannot prove exact non-Archimedean order and cannot resolve an expression whose leading coefficient is symbolically unknown. A finite algebraic identity survives any defined field homomorphism, but an arbitrary infinite Hahn expansion need not survive replacement of its scale by a small positive real number — the nondiscrete-support and factorial-series examples give two independent reasons.

Accordingly, `NumericQ` is not a declaration of membership in a non-Archimedean ordered field; its documented role concerns numeric expressions [35]. Marking an infinite surreal as an ordinary numeric value can send unrelated numerical algorithms into an impossible real approximation, or make numerical solvers demand a conventional machine or arbitrary-precision number the object cannot supply. Use a custom exact-domain predicate and supply numerical views only through explicitly described coefficient approximation or finite-scale specialization. A generic `N[value]` rule should not pretend to return an ordinary complex approximation to an arbitrary surreal number, and converting all infinities to native `Infinity` and all infinitesimals to zero is neither an injective representation nor a field homomorphism. A written `N[surreal,p]` requires a declared meaning: a certified coefficient approximation, a valuation jet, or a scale-aware structured result.

15.6 Infinity

**Never redefine `System`Infinity` to mean  $\omega$ .** The latter is a field element with  $\omega - \omega = 0$ ; the former has extended-limit semantics [37]. In all three delivered prototypes `Infinity` appears only as the distinguished valuation marker $v(0) = +\infty$.

15.7 Conservative dispatch, explicit operations before upvalues

Inert symbolic heads and controlled evaluation keep the host from simplifying away a domain distinction before the extension sees it, and `Inactive` and upvalue mechanisms are documented tools for such an integration [34, 33]. But the first reference implementation should expose *explicit* operations — all three delivered prototypes do exactly this — because their domains are easier to test than an early collection of global automatic rewrites. Once the core is stable, narrowly targeted upvalues can provide natural notation for operations whose semantics are unconditional in the operands' domains.

Do not globally unprotect `Plus`, `Times`, `Power`, `Exp`, `Sin`, `Log` or `Less` and redefine them for arbitrary expressions. Branch-dependent transformations should remain explicit or return their conditions. The package must avoid evaluation loops, ambiguous mixed-domain coercions and branch-changing rewrites. Upvalues and `Inactive` do not create mathematical correctness on their own, and they do not remove the responsibility to check mixed parents and analytic hypotheses. Keep raw monomial symbols internal or require them to be unassigned; promoting two parents requires an explicit embedding preserving coefficient arithmetic, exponent relations, order and function policy.

Results should distinguish invalid input, an undefined mathematical operation, an unsupported algorithm, and an unknown proposition. **A failed zero test must not return False**; a negative power at zero is genuinely undefined; a square root outside a rational parent asks for an extension rather than proving impossibility. On general symbolic inputs, a conditional result should preserve its conditions instead of assuming them to simplify the output.

15.8 Cache keys

Cache keys must include the coefficient assumptions, the exponent order, the embedding, the branch convention, the derivation interpretation and the extension policy. A normal form computed under one convention must not be reused after any of those change. See [theorem 13.4](#) for the three forms in which the sources state this rule.

15.9 Serialization and the code-execution risk

Serialization should preserve semantic constructors and parent identifiers rather than pretty output, and should reference shared expression nodes rather than copy exponentially growing trees. It should also preserve whether a displayed term is exact or carries a remainder.

Remark 15.3 (Security). **Coefficient programs and function definitions may contain executable Wolfram code, so loading untrusted serialized programs is a code-execution risk.** A production format should use a validated expression grammar and explicitly trusted extension modules rather than automatically evaluating arbitrary input strings. Importing an arbitrary coefficient program must not execute it automatically: mathematical admissibility and safe program execution are separate checks. Explicit resource budgets and opt-in evaluation of external code are suitable engineering boundaries. Relatedly, none of the three delivered prototypes is a hardened parser for arbitrary or untrusted input, and in all three the internal data heads are private: *manually forging them bypasses validation and lies outside each package's own contract.*

15.10 Lossless coercion before convenient notation

Embedding a smaller ordered monomial field into a larger one is legitimate when the map is explicit and order preserving, and adjoining algebraic roots is legitimate. Replacing a named transcendental constant by an independent symbol, replacing an exact value by a jet, changing a branch, or discarding an infinitesimal by taking a standard part is **not** a lossless coercion. The public interface should make those operations visibly intentional: taking a standard part requires a certificate of finiteness, and scale specialization must distinguish ordinary parameter specialization from evaluation at an actual surreal monomial. Promoting a rational node to a complex pair or an algebraic extension must preserve the embedding rather than simply discard the older domain tag.

15.11 Precedents in other systems

The same distinctions are visible in other CAS designs and are useful architectural precedents. Sage's lazy-series documentation explicitly distinguishes coefficients computed on demand from equality, which is decidable only in special cases [39]. Its asymptotic rings separate coefficient rings, growth groups, exact terms and order terms [40], working with growth and error terms rather than purporting to encode every surreal. Existing domain definitions should be reused where they fit, not relabelled wholesale as the full surreal class. These systems are sources of algorithms and interfaces,

not substitutes for the semantic choices made here. A CAS other than Wolfram Language can follow the same architecture: the mathematical requirements are an exact coefficient engine, programmable expression types, an ordered exponent representation, and validated adapters for its polynomial and series facilities.

16 A proposed architecture — explicitly not delivered

Every statement in this section is a proposal. Nothing named `SurrealDomain`, `SurrealRational`, `SurrealAlgebraic`, `CertifiedHahn`, `SurrealExpression`, `SurrealJet`, `Surcomplex`, `AnalyticGerm`, `CoherentFunction`, `ExactSurreal`, `SurrealEqual`, `SurrealExpand`, `SurrealCoefficient`, `SurrealRoot`, `CoherentLift`, `HahnParent`, `HahnValue`, `CoherentFamily` or `SurcomplexFunction` exists in any delivered file described in [section 17](#). No capability named in this section may be reported as implemented.

16.1 Architecture requirements all three sources agree on

Stripped of syntax, the three proposals agree on what a domain, an exact value, a view, a function and a result must record.

- A **domain** (or parent) should record: the coefficient domain; the ordered exponent domain, including its order and coordinate significance; the embedding into the surreal numbers; the support presentation or support grammar; any exponential, branch, extension and derivation conventions; and the list of available algorithms.
- An **exact value** is a finite expression or certified construction *in* a named domain, together with its certificates.
- A **view** is a computed expansion, algebraic root approximation, or enclosure of an exact value. *Computing another view must not silently change the value or its domain.* A rational expression, an algebraic root description and a Hahn expansion can denote the same element while serving different purposes; keeping all three as related representations is more useful than forcing every result into one preferred infinite normal form.
- A **function** should record its variables, coefficient parent, ordinary or nonstandard domain, regularity category, branch, and extension policy.
- **Capabilities should be queryable, not guessed from a head.** A domain should advertise separately whether equality and order are decidable, whether coefficients and leading terms are computable, and whether requested cutoff expansions are finite. Crucially, *some capabilities are local to a value or a request*: a particular series can have a certified leading term even when its representation class has no general leading-term algorithm, and a cutoff may admit a finite expansion although a larger cutoff does not.
- **Certificates should be checkable more cheaply than the search that found them.** Useful certificates include an exact coefficient normal form; a nonzero leading term with a lower-support exclusion; an algebraic root selector; a positive-grid decomposition bound; a nonvanishing ordinary denominator on a specified domain; a polynomial residual with a valuation lower bound; a finite support; a positive-support monoid; an exact polynomial identity; a host algebraic-number sign certificate; or a branch identity under recorded inequalities. The checker need only verify the relevant finite statement.

- **Unproved is not false, and a resource limit is not a counterexample.** A missing zero proof is not a mathematical domain error, and an input outside a chosen sine strip is not a proof that no extension exists. The constructor should state whether its evidence has been *checked, supplied as an assumption, or inherited from a proved construction*. *User assertions must not be relabelled as kernel-verified theorems.*

16.2 Three concrete syntax sketches, side by side

The three proposals are printed as three alternatives, each labelled with its origin. No fourth invented syntax is synthesized here, and none of the three is silently adopted.

Sketch A — capability-tiers proposal (positional).

```
SurrealDomain[coefficientDomain, exponentDomain, interpretation]
SurrealRational[domain, numerator, denominator]
SurrealAlgebraic[domain, polynomial, rootSelector]
CertifiedHahn[domain, supportDescriptor, coefficientDescriptor]
SurrealExpression[domain, expressionGraph]
SurrealJet[domain, center, remainderDescriptor]
Surcomplex[realPart, imaginaryPart]
AnalyticGerm[functionDescriptor, center, branch, domain]
CoherentFunction[ordinaryDomain, support, coefficients]
(* explicit operations: SRAdd, SRSin *)
```

Sketch B — layered-representations proposal (option association).

```
dom = SurrealDomain[
  "Coefficients" -> "RealAlgebraic",
  "Exponents"    -> LexicographicRationals[2],
  "Embedding"    -> ConwayMonomials[{Omega, 1}]];
x = ExactSurreal[representation, dom];
SurrealEqual[x, y, "Evidence" -> True]
SurrealExpand[x, "ValuationCutoff" -> {1, 0}]
SurrealCoefficient[x, {0, 17}]
SurrealRoot[p, "Selector" -> certifiedSelector]
CoherentLift[f, "Domain" -> U, "Branch" -> branchData]
(* explicit operations: SAdd, SMultiply, SInverse, SCompare *)
```

The embedding record in Sketch B means that exponent (a, b) denotes $a\omega + b$. It is not an instruction to identify an unspecified host symbol with the entire surreal class, and an implementation should store such records in a validated internal format rather than rely on their printed appearance.

Sketch C — Gaussian-prototype proposal (parent/value split).

```
HahnParent[coefficientDomain, exponentDomain,
  "MonomialInterpretation" -> embedding,
  "SupportClass" -> supportGrammar]
HahnValue[parent, representation, certificates]
AnalyticGerm[center, definition, initialData, domain, branch]
CoherentFamily[parent, ordinaryDomain, coefficientProgram]
SurcomplexFunction[definition, domain, extensionPolicy]
(* explicit operations: HAdd, HMul, HCompare *)
```

Remark 16.1 (Two head collisions among the proposals). The three sketches are *not* compatible, and two of the collisions are between heads with the same name and different argument shapes.

- `SurrealDomain` appears in Sketch A as a *positional triple* and in Sketch B as an *option-value association*. A caller who writes one and reads the other gets nonsense.
- `AnalyticGerm` appears in Sketch A with the argument list `[functionDescriptor, center, branch, domain]` and in Sketch C with `[center, definition, initialData, domain, branch]`. The arities and the argument orders both differ.

Any phrasing implying that `SurrealDomain`, `CertifiedHahn`, `HahnParent`, `SurrealRoot` or `CoherentLift` exists in code would be false three times over.

16.3 Three result protocols, side by side

The three sources propose result protocols at three different granularities of the same invariant. All three are printed; none is picked.

Origin	Result kinds	Notes
Capability tiers	Five-valued: <code>Exact</code> / <code>CertifiedApproximation</code> / <code>Conditional</code> / <code>Unknown</code> / <code>Failure</code>	Comes with named <i>standing obligations</i> that may attach to a conditional node: a positive argument for a real logarithm; a finite real part for canonical strip sine; positive support for an infinitesimal series substitution; a root selector valid in the current ordered field; a common ordinary domain for coherent composition; a compatible phase character for infinite circular input. Division by a <i>certified</i> zero is an error; division by an object of <i>unknown</i> zero status creates a nonzero obligation.
Layered representations	Boolean-plus-witness / <code>Undetermined[reason, obligations]</code> / <code>Failure</code>	A successful equality request returns a Boolean and a checkable witness; an unsupported or exhausted request returns <code>Undetermined</code> ; invalid input returns a failure.
Gaussian prototype	Four result kinds: invalid input / undefined mathematical operation / unsupported algorithm / unknown proposition	A failed zero test must not return <code>False</code> ; a negative power of zero is genuinely undefined; a square root outside a rational parent asks for an extension rather than proving impossibility.

The common invariant is the one stated in [section 16.1](#): an unproved equality is not a proved inequality, and a resource limit is not a mathematical counterexample. Each operation should additionally report the capability actually used — a normalization can succeed in the rational backend while leaving one transcendental coefficient symbolic; a positivity query can succeed from a certified leading coefficient without full normalization; a valuation cutoff can produce a compressed exact block rather than a flat list. These are useful, distinct *successful* results.

16.4 Component responsibilities

Component	Responsibilities and explicit guarantees
Coefficient backend	Exact arithmetic; zero and sign decisions where available; ordinary analytic identities; coefficient enclosures; conversion to larger coefficient domains; whether its ordinary constants admit certified numerical enclosures.
Exponent (monomial) backend	Addition, equality, mathematical order, rational division when supported, rational-root promotion, order-preserving changes of scale, and the embedding used to interpret it inside the surreal field.
Exact algebra	Sparse sums, rational expressions, finite algebraic extensions, polynomial and matrix operations.
Support engine	Certified constructors; finite decomposition bounds; coefficient production; <i>separate</i> membership, coefficient, decomposition, leading-term and prefix services, with a declaration of which are total; and a semantic proof of well ordering kept distinct from algorithms.
Precision engine	Valuation bounds, generator-index jets, ordinary coefficient error models, refinement, and dependence tracking.
Function layer	Formal germs, ordinary domains, Hahn-coherent data, radius-free germs, branch selectors, derivative modes, admissible composition rules, and explicitly selected transserial interpretations.
Presentation layer	Readable notation, lossless serialization, evidence inspection, and safe adapters to native CAS facilities.

An operation result should preserve its exact origin and its obligations. If the answer is a truncation it must include a remainder descriptor and the error calculus used to obtain it. If the answer is conditional, the condition must remain visible in further transformations. If an unsupported request has a well-defined semantic node, the system may retain that node without pretending to have evaluated or normalized it.

16.5 Proof-carrying expressions without a proof assistant

A practical certificate can be a small derivation tree built from trusted constructors, with a checker smaller and more deterministic than the expression-search engine. A proof-oriented interface could export finite polynomial identities, support derivations, valuation inequalities and root-selection certificates to such a checker, which would verify specific claims without needing to decide every question about every surreal object.

Remark 16.2 (The trust boundary). This does **not** amount to a machine-checked formalization of the surreal class. Where certificates rely on trusted mathematical theorems from the supplied manuscripts, *that trust boundary must be documented*. A later formal checker could verify the finite algebraic kernel and the soundness of selected support constructors without requiring every analytic extension to be formalized at once. No such formalization exists in any of the three delivered packages ([section 18](#)).

17 The three delivered prototypes

*There are **three** shipped Wolfram Language packages here, not one. They have similar names — two of them are near-anagrams — similar API verbs, and all three are described in their own source articles as implementing “the effective rational monomial core”. They are nevertheless three different artifacts with three different envelopes. This report never writes “the prototype”. Each is named: **RationalHahn**, **SurrealCASCore**, **HahnRational**. No capability of one may be attributed to another.*

17.1 RationalHahn — the capability-tiers kernel, over $\mathbb{Q}$

File. `RationalHahn.wl`, 78 lines, approximately 4.3 KB; context `RationalHahn'`; shipped with `Checks.wl` (34 checks) and a driver `RunChecks.wl`. In this repository the three files are `code/02-capability-tiers-and-rational-kernel-rationalhahn.wl`, `code/02-capability-tiers-and-rational-kernel-checks.wl` and `code/02-capability-tiers-and-rational-kernel-runchecks.wl`.

Coefficient field: $\mathbb{Q}$, and only $\mathbb{Q}$. The predicate is `rationalQ[c] := IntegerQ[c] || Head[c] === Rational`, applied through `CoefficientRules` to every coefficient of both numerator and denominator. There is **no i** , **no Gaussian rationals**, **no complex arithmetic** and **no surcomplex pairs anywhere in this kernel**. Machine reals (1.0) and symbolic transcendentals (Pi) are rejected with `Failure["CoefficientDomain"]`. The article's design target $\mathbb{R}_{\text{alg}}$ is explicitly *not* what the kernel implements: “An initial program may use $\mathbb{Q}$ instead, as our reference kernel does.” *Anything this kernel proves or demonstrates is a statement about a real kernel.*

Exponent lattice: integer vectors $\mathbb{Z}^d$. Elements are quotients of ordinary multivariate polynomials in d formal symbols, so monomial exponents come from `CoefficientRules` in $\mathbb{N}^d$ and valuations lie in $\mathbb{Z}^d$. There are no rational or fractional exponents, no ramification, and no $t^{1/2}$ anywhere in this kernel; Puiseux data and root selectors appear only as later design layers in the prose.

Rank and parent identity. Arbitrary finite d , fixed at construction by the variable list passed to `RHMake`. Domain identity is `SameQ` on that variable list (`sameDomainQ`), so there is no automatic embedding between domains: `RHAdd` of a `{t,s}` object and a `{t}` object returns `Failure["DomainMismatch"]`. The shipped check suite uses $d = 2$ with variables `{t,s}`.

Monomial interpretation. The ordered variables $\{t_1, \dots, t_d\}$ denote Conway monomials $t_j = \omega^{-\omega^{j-1}}$, so $v(t_1) = 1$, $v(t_2) = \omega$, ..., $v(t_d) = \omega^{d-1}$ and $0 < t_{j+1} < t_j^n$ for every ordinary positive integer n . They are explicitly not ordinary numeric variables with small floating-point values.

Coordinate significance: REVERSED. The leading term is the `CoefficientRules` entry whose exponent vector is least *after reversing* its coordinates, so the *highest-index* (highest-rank, most infinitesimal) variable is the first comparison coordinate. Literally:

```
leadingPolynomial[p_, variables_List] := Module[{r},
  r = First[SortBy[CoefficientRules[p, variables],
    Reverse[First[#]] &]];
  {First[r], Last[r]}];
```

Standard part uses `vectorSign`, the sign of the first nonzero entry of the *reversed* vector: positive means infinitesimal, so `st = 0`; the zero vector returns the leading coefficient; negative returns `Failure["NotFinite"]`.

Data model. `RHData[variables, numerator, denominator]`, a private head; fractions are reduced by `Cancel` at construction and the parts are `Expanded`.

Exported operations (13, with usage messages). `RHData` (private data head), `RHMake`, `RHExpression`, `RHAdd`, `RHNegate`, `RHSubtract`, `RHMultiply`, `RHInverse`, `RHSign` (exact $-1/0/1$), `RHCompare`, `RHLeadingData` (returns {valuation vector, leading coefficient}; `Missing["ZeroHasNoLeadingTerm"]` for zero), `RHValuation` (`Infinity` only as the distinguished valuation marker for zero), `RHStandardPart`.

Failure taxonomy. Variables (empty, duplicate or non-symbol), `CoefficientDomain`,

ZeroDenominator, DomainMismatch, DivisionByZero, NotFinite.

Not implemented (stated in the package comment, the README and the article): the effective real closure R_d , complex pairs and C_d , algebraic root extensions and root selectors, arbitrary exponent groups, lazy or arbitrary infinite Hahn supports, transcendental functions, general complex powers, and proof checking. These are subsequent design layers, *not hidden features*.

Session, in this kernel's own convention. The opening `Get` succeeds only against the one-character repair of §18.3; the file as delivered does not parse.

```
Get["code/RationalHahn.wl"]; (* after the repair of 18.3 *)
Clear[t, s]; (* t = omega^-1 ; s = omega^(-omega) *)
a = RHMake[1 + t, t, {t, s}];
b = RHMake[s, t^10000, {t, s}];
RHSign[a] (* 1 *)
RHValuation[a] (* {-1, 0} *)
RHStandardPart[b] (* 0 *)
RHCompare[b, RHMake[1, 1, {t, s}]] (* -1 *)
RHEXpression[RHInverse[a]] (* t/(1 + t) *)
RHValuation[RHMake[(1 + t) s - s, 1, {t, s}]] (* {1, 1} *)
```

The last line is the cancellation trap of [section 5.3](#): the leading exponent is $(1, 1)$, not $(0, 1)$.

Remark 17.1 (The 10000-power comparison is one instance). The suite constructs both inputs explicitly:

```
RHCompare[RHMake[s, 1, {t, s}],
          RHMake[t^10000, 1, {t, s}]] === -1
RHSign[RHMake[t^10000-s, 1, {t, s}]] === 1
```

These are finite higher-rank order checks. Bare symbolic expressions are not arguments of `RHCompare` or `RHSign`; the kernel has no automatic coercion from them. *The checks are not the proof for every n* ; the universal statement follows from the exponent-group definition and [theorem 5.1](#).

17.2 SurrealCASCore — the layered-representations kernel, over $\mathbb{Q}(i)$ with $\mathbb{Q}^r$ exponents

File. `SurrealCASCore.wl`, 82 lines, 4689 bytes; context `SurrealCASCore`; shipped with the harness `TestCore.wls` and an independent SymPy script `verify_examples.py`. In this repository: `code/04-layered-representations-and-adapter-surrealcascore.wl`, `code/04-layered-representations-and-adapter-testcore.wls`, `code/04-layered-representations-and-adapter-verify-examples.py`.

Coefficient field: exact Gaussian rationals $\mathbb{Q}(i)$. The predicate is `gaussianQ[x] := ratQ[Re[x]] && ratQ[Im[x]]` with `ratQ[x] := MatchQ[x, _Integer | _Rational]`. Machine reals are rejected (`HCreate[{{1}, 0.5}, 1]` is a tested `Failure`). So: $\mathbb{Q}(i)$, not $\mathbb{Q}$, not $\mathbb{R}_{\text{alg}}$, not algebraic numbers.

Exponent lattice: RATIONAL vectors $\mathbb{Q}^r$. Each exponent is a list of length exactly r whose entries each satisfy `ratQ`. Irrational exponent coordinates are rejected (`HCreate[{{Sqrt[2]}, 1}, 1]` is a tested `Failure`), and the suite *exercises* half-integer exponents — $1/2$, and `RandomInteger[{-4, 6}]/2`. **This is the only one of the three kernels with fractional exponents.**

Rank and parent identity. r is a fixed positive ordinary *integer* field stored inside `HData[rank, terms]`; the guard is `r_Integer /; r > 0`. Parent identity is equality of that integer. Operations between data of different rank are rejected, with no automatic coercion or rank-raising. The tests use $r = 1$ and $r = 2$ only.

Coordinate significance: plain lex, FIRST coordinate most significant. Implemented by `lexLess`, which finds the first index where the vectors differ and compares there. Terms are stored sorted ascending by exponent, so the leading (least) exponent sits at position `[[2,1,1]]`. The surreal interpretation, stated in the package header, the README and the article, is $(q_1, \dots, q_r) \mapsto q_1\omega^{r-1} + \dots + q_r$ with Hahn monomial $\omega^{-\gamma}$ at exponent γ , so *a larger exponent vector means a smaller number*; for $r = 2$ the generators are $\omega^{-\omega}$ (exponent $(1, 0)$) and ω^{-1} (exponent $(0, 1)$).

Data model: TWO representations. A canonical sparse map `HData[rank, terms]` from exponent vector to coefficient — `HCreate` validates all terms, *gathers* coincident exponents and *sums* their coefficients, *drops* zero coefficients, and sorts by `lexLess`, with `validQ` re-running `HCreate` to check canonicity of any datum handed to an operation. And separately a *fraction wrapper* `RData[num, den]` of two such sums. **The fraction wrapper is deliberately NOT gcd-normalized:** structurally different `RData` wrappers may be equal, and `REqual`, not `SameQ`, is the intended comparison. This is exactly the canonicity/non-canonicity contrast of [section 1.2](#), present inside one package.

Exported operations (17 with usage messages). Sparse layer: `HCreate`, `HAdd`, `HNegate`, `HMultiply` (naive pairwise exponent sums, $O(mn)$ before collection), `HPower` (*nonnegative* integer exponent only, by `Nest` — deliberately not an optimized exponentiation), `HConjugate` (coefficientwise Gaussian conjugation), `HValuation`, `HLeadingCoefficient`, `HCompareReal` (returns $-1/0/1$ via the sign of the leading coefficient of $a - b$, and returns `Failure["NonrealInput"]` if *any* coefficient of either operand has nonzero imaginary part). Fraction layer: `RCreate` (rejects a zero denominator), `RAdd`, `RMultiply`, `RInverse`, `REqual` (cross multiplication), `RValuation` (componentwise difference, so it returns a *vector*, e.g. `{-2}`). Plus the two private data heads `HData` and `RData`.

Absent from this kernel. *No standard part.* No `HRe`, `HIm` or squared modulus. No inverse on the sparse layer — only `RInverse` on fractions.

Not implemented (stated in the package comment, the README, the test report and the article, though no two of them phrase the list identically and none of them mentions standard part, which is recorded above as absent by inspection rather than by the package’s own disclosure): arbitrary surreal cuts, algebraic coefficient extensions, algebraic root isolation and selectors, infinite series generators, certified analytic domains, general Hahn-coherent functions, the transserial layer, and standard part. No definitions are attached to `System‘Plus`, `System‘Times`, `System‘Power`, `System‘Infinity`, `System‘N` or `NumericQ`.

Session, in this kernel’s own convention.

```
Get["SurrealCASCore.wl"];
one = HCreate[{{0}, 1}, 1];
t   = HCreate[{{1}, 1}, 1];
a   = HAdd[one, HNegate[t]];
b   = HAdd[one, t];
HMultiply[a, b]
(* HData[1, {{0}, 1}, {{2}, -1}] *)
REqual[RCreate[one, a], RCreate[b, HMultiply[a, b]]]
(* True *)

s = HCreate[{{0, 1}, 1}, 2];
```

```
u = HCreate[{{1, 0}, 1}, 2];
HCompareReal[HPower[s, 100], u]
(* 1 *)
```

The first result is the exact identity $(1-t)(1+t) = 1-t^2$. The second checks $1/(1-t) = (1+t)/(1-t^2)$ *without expanding either geometric series*; no infinite summation routine is needed. In the third, $s = \omega^{-1}$ and $u = \omega^{-\omega}$, and the output 1 means $s^{100} > u$.

Remark 17.2 (The exponent-100 test is one instance). More generally $s^n > u$ for every positive ordinary integer n , because $(0, n) < (1, 0)$ lexicographically. *That general assertion follows from the domain's order definition, not from extrapolating the particular test with exponent 100.*

17.3 HahnRational — the Gaussian-prototype kernel, over $\mathbb{Q}(i)$ with $\mathbb{Z}^d$ exponents

File. HahnRational.wl, 97 lines, 5666 bytes; context HahnRational'; shipped with test_HahnRational.wl (88 checks), a short demo examples.wl, and an independent SymPy script verify_examples.py. In this repository: code/05-gaussian-rational-monomial-prototype-hahnRational.wl and companions with the same prefix.

Coefficient field: $\mathbb{Q}(i)$. Enforced by `gaussianQ[c] := MatchQ[c, _Integer | _Rational | Complex[_Integer|_Rational, _Integer|_Rational]]`, applied to every coefficient of both parts at construction. π , machine floats and any transcendental coefficient are rejected with `Failure["CoefficientDomain", ...]`. Not plain $\mathbb{Q}$; not algebraic numbers; not Root objects.

Exponent lattice: **INTEGER** vectors $\mathbb{Z}^d$ **only**, of rank $d = \text{Length}$ of the declared variable list. **Rational exponents are not supported and are actively rejected:** `HCreate[{u,t}, Sqrt[t]]` is a tested rejection — fractional powers need a parent extension. Rational-exponent lattice refinement is discussed in the article as future work only.

Rank and parent identity. Arbitrary finite d , fixed by the declared variable list; parent identity is `SameQ` on that list (`sameParent`). **A reversed variable list is a DIFFERENT parent** and mixing is rejected: `HAdd[h[t], HCreate[{t,u}, t]]` fails with `ParentMismatch`. Tests and examples use $d = 2$.

Coordinate significance: plain lex, **LEFTMOST coordinate most significant**, with variables listed from the *most significant* valuation coordinate to the least. `polyLeading` is `First[SortBy[CoefficientRules[Expand[p], vars], First]]` — with *no* reversal. For the parent $\{u, t\}$: $v(u) = \{1, 0\}$, $v(t) = \{0, 1\}$, with the concrete surreal reading $t = \omega^{-1}$ and $u = t^\omega = \omega^{-\omega}$; *larger valuation means smaller magnitude*, so $u < t^n$ for every ordinary positive integer n .

Data model. `HR[vars, numerator, denominator]`, a private head, holding normalized expanded polynomials with the denominator's lexicographically-leading coefficient *normalized to 1*. Construction goes through `Cancel[Together[expr]]` and requires `PolynomialQ` of both parts in the declared variables. This is *not* an expanded Hahn series.

Exported operations (19 with usage messages). `HCreate` (the *only* public value constructor), `HExpression`, `HVariables`, `HAdd`, `HMul`, `HInv`, `HPower` (integer exponent; $n = 0$ returns 1; a negative power of zero is rejected), `HScale` (by a $\mathbb{Q}(i)$ scalar), `HEqualQ` (cross-multiply and `Expand`; does not require gcd), `HZeroQ`, `HValuation` (numerator minus denominator exponent vectors; `Infinity` for zero, as metadata only), `HLeadingCoefficient`, `HConjugate`, `HRe`, `HIm`, `HNormSquared` ($a\bar{a}$, no square root), `HSign` (rejects input not equal to its own conjugate), `HCompare` (returns $-1/0/1$; *rejects nonreal inputs even when their difference is real*), `HStandardPart` (0 for infinitesimals; the

leading coefficient — possibly a Gaussian rational such as $2 + 3i$ — at valuation-sign zero; and `Failure["InfiniteValue"]` for infinite values).

This is the only one of the three with the full surcomplex surface: conjugation, real and imaginary parts, squared modulus, sign and comparison with nonreal rejection, and standard part.

Not implemented (stated by both the README and the article): general Hahn-series coefficient programs, rational-exponent parent refinement, algebraic closure, algebraic root selection, arbitrary analytic germs, transseries, global exponential extensions, unrestricted modulus (only *squared* modulus), contour integration, AG's germ division and preparation, and CT's integration theory.

Session, in this kernel's own convention.

```
Get["code/HahnRational.wl"];
Clear[u, t];
a = HCreate[{u, t}, 1/(1-t) + u];
b = HCreate[{u, t}, 1/(1-t)];
HValuation[HAdd[a, HScale[b, -1]]]
(* {1, 0} *)
HCompare[HCreate[{u, t}, u], HCreate[{u, t}, t^100]]
(* -1 *)

z = HCreate[{u, t}, t + I u];
HExpression[HNormSquared[z]]
(* t^2 + u^2 *)
HStandardPart[HCreate[{u, t}, (2 + 3 I + t)/(1 - u)]]
(* 2 + 3 I *)
HStandardPart[HCreate[{u, t}, t/u]]
(* Failure["InfiniteValue", ...] *)
```

The first operation *cancels an entire infinite geometric block exactly* and reveals a term at a new valuation rank; a coefficient stream with no way to recognize the equal geometric tails could fail to reach it. The second comparison is exact and does not depend on choosing a large real value for ω . The quotient t/u is infinite and has no finite standard part in this convention, so the package returns a `Failure` rather than a floating-point infinity.

17.4 Comparison, symbol collisions, and four cross-attribution warnings

17.4.1 The full property table

Property	RationalHahn (02)	SurrealCASCore (04)	HahnRational (05)
File and size	<code>RationalHahn.wl</code> , 78 lines, ≈ 4.3 KB	<code>SurrealCASCore.wl</code> , 82 lines, 4689 B	<code>HahnRational.wl</code> , 97 lines, 5666 B
Context	<code>RationalHahn`</code>	<code>SurrealCASCore`</code>	<code>HahnRational`</code>
Data model	Reduced quotient <code>RHData[vars,num,den]</code> ; Cancel at construction	Two: canonical sparse <code>HData[rank,terms]</code> plus a deliberately <i>unreduced</i> fraction <code>RData[num,den]</code>	Reduced quotient <code>HR[vars,num,den]</code> ; <code>Cancel[Together[...]]</code> , denominator leading coefficient scaled to 1
Coefficient field	$\mathbb{Q}$ only — no i anywhere	$\mathbb{Q}(i)$	$\mathbb{Q}(i)$
Exponent lattice	$\mathbb{Z}^d$	$\mathbb{Q}^r$ — rational exponents , actually exercised	$\mathbb{Z}^d$

Property	RationalHahn (02)	SurrealCASCore (04)	HahnRational (05)
Rank / parent identity	Arbitrary d ; SameQ on the variable list	An integer field r ; equality of that integer	Arbitrary d ; SameQ on the variable list
Coordinate significance	REVERSED : highest-index coordinate compared first	Plain lex, first coordinate most significant	Plain lex, first coordinate most significant
Monomial reading	$t_j = \omega^{-\omega^{j-1}}$; $v(t_1) = 1$, $v(t_2) = \omega$	$(q_1, \dots, q_r) \mapsto q_1 \omega^{r-1} + \dots + q_r$	$(a_1, \dots, a_d) \mapsto a_1 \omega^{d-1} + \dots + a_d$; $u = \omega^{-\omega}$, $t = \omega^{-1}$
Standard part	Yes (RHStandardPart)	No	Yes (HStandardPart)
Conjugation / Re / Im / $ z ^2$	None at all	HConjugate only	HConjugate, HRe, HIm, HNormSquared
Real comparison	RHSign, RHCompare	HCompareReal, rejecting nonreal input	HSign, HCompare, both rejecting nonreal input
Inverse	RHInverse	Only RInverse , on fractions	HInv
gcd-reduced fractions	Yes	No — use REqual, not SameQ	Yes
Exported symbols	13	17	19
Rejections tested	machine reals, Pi, Sin[t], zero denominator, zero inverse, domain mismatch, duplicate/empty variables	machine reals, Sqrt[2], exponents, rank mismatch, negative power, zero denominator, zero inverse, nonreal ordering	Pi, machine reals, Exp[t], Sqrt[t] (needs a parent extension), parent mismatch, zero inverse, negative power of zero, nonreal ordering
Not implemented	real closure, complex pairs, algebraic roots, arbitrary exponent groups, lazy supports, transcendental functions, general complex powers, proof checking	algebraic coefficient fields, algebraic root selectors, infinite series generators, certified analytic domains, arbitrary Conway cuts, transserial layer, standard part	general Hahn coefficient programs, rational-exponent refinement , algebraic closure, arbitrary analytic germs, transseries, global exponential extensions, contour integration
What was executed	Definitions submitted as code ; 31/34 then three corrected test expressions; the shipped file does <i>not</i> parse	Real files Get-loaded ; 110/110; plus SymPy 69/69	Get[StringToStream[...]] ; 88/88; plus SymPy 323/323

17.4.2 The symbol-name collision

Remark 17.3 (Six colliding public symbols). **SurrealCASCore** and **HahnRational** both declare, with `::usage`, the six public symbols

HAdd, HConjugate, HCreate, HLeadingCoefficient, HPower, HValuation,

with incompatible signatures. The clearest instance is the constructor:

- SurrealCASCore: `HCreate[{exponentVector, coefficient},...], rankInteger]`
- HahnRational: `HCreate[{u,t,...], rationalExpression]`

They also diverge on the names of operations that do the same job — `HMultiply` versus `HMul`, `HCompareReal` versus `HCompare`, `RInverse` versus `HInv`. **Loading both packages into one session silently breaks both**, since one set of definitions shadows the other.

Additionally, note the *name* clash at the file level: `RationalHahn.wl` (02) and `HahnRational.wl` (05) are near-anagrams of each other and are two entirely different artifacts.

17.4.3 Four cross-attribution warnings

- (a) $\mathbb{Q}(i)$, conjugation and surcomplex operations belong to **SurrealCASCore** and **HahnRational**, and **NOT** to **RationalHahn**, whose kernel is $\mathbb{Q}$ -only and has no conjugation, no real or imaginary part, and no modulus of any kind.
- (b) **Rational and fractional exponents belong to SurrealCASCore ONLY.** **RationalHahn** and **HahnRational** both have integer exponent lattices, and **HahnRational**’s suite *actively rejects* `Sqrt[t]` as needing a parent extension. So a construction needing a fractional exponent — a ramified root, a Puiseux-type uniformizer — is representable in **SurrealCASCore** and not in the other two.
- (c) **Standard part belongs to RationalHahn and HahnRational, and NOT to SurrealCASCore**, which has no standard-part operation at all.
- (d) **gcd-reduced fractions belong to RationalHahn and HahnRational, and NOT to SurrealCASCore**, whose fraction wrapper is deliberately unreduced, so that structurally different wrappers may be equal and `REqual` rather than structural equality is the intended comparison. Correspondingly, claims about “not expanding an infinite tail” by means of a separate *fraction object* layered over a canonical sparse sum belong to **SurrealCASCore**; the other two carry a rational function directly.

Remark 17.4 (The sentence never to write). Any sentence of the form “the prototype implements the rational monomial core over $\mathbb{Q}(i)$ with rational lexicographic exponents, conjugation and standard part, and passes its checks” is **true of no delivered artifact**. It silently claims a union of capabilities that none of the three has.

17.4.4 The reversed variable-order convention

Remark 17.5 (The conventions are reversed between two of the three kernels). **RationalHahn lists variables least-significant-valuation first; HahnRational lists them most-significant first.** Verified directly against the sources:

- **RationalHahn:** $v(t_1) = 1, v(t_2) = \omega, \dots, v(t_d) = \omega^{d-1}$, so $0 < t_{j+1} < t_j^n$. *The first-listed variable is the largest element and has the least valuation.* Exponent vectors are compared after **Reverse**.
- **HahnRational:** “Variables are listed from the most significant valuation coordinate to the least. For $\{u, t\}$, u is smaller than every positive power of t .” *The first-listed variable is the smallest element and has the greatest valuation.* Checked by hand: $v(u) = \{1, 0\}$, $v(t^n) = \{0, n\}$, and lexicographically $\{1, 0\} > \{0, n\}$, hence $u < t^n$.

HahnRational’s own README warns that *a reversed variable list is a different parent*. **Confusing the conventions can reverse comparisons between scales.** For example, $(1, 0)$ and $(0, 1)$ exchange order when coordinate significance is reversed, whereas $(1, 1) > (0, 0)$ holds in both orders. **SurrealCASCore** agrees with **HahnRational** on coordinate significance, and the two agree on the interpretation $(q_1, \dots, q_r) \mapsto q_1\omega^{r-1} + \dots + q_r$; they differ only on whether the q are rational or integral.

All three kernels express the same mathematical fact, $\omega^{-\omega} < \omega^{-n}$ for every ordinary n , under two different coordinate layouts and — in **SurrealCASCore**’s case — with reversed argument order in the comparison call. The three displayed comparisons are:

- **RationalHahn:** the fully constructed calls in [theorem 17.1](#) give comparison -1 and difference sign 1 .
- **SurrealCASCore:** `HCompareReal[HPower[s,100], u] === 1`, meaning $s^{100} > u$ — note the reversed argument order relative to the other two.
- **HahnRational:** `HCompare[h[u], h[t^100]] === -1`, where the suite defines `h[e_] := HCreate[{u,t},e]`.

None of these has been rewritten into a common form, and this report states no single global coordinate convention that these quoted outputs would then contradict. The global rule stated once in [theorem 5.2](#) is that coordinate significance is *declared as part of the domain descriptor*; each kernel’s own declaration is given in its own subsection above.

17.5 Recovered on audit

An audit of this merge against its three sources found three scope disclaimers stated in a source and missing from the merged text. They are restored here.

- **Imported proofs were not re-audited.** This report does not re-audit the companion analytic-geometry manuscript’s Noetherian, Jacobson or deformation proofs, nor the contour manuscript’s general residue-transformation argument. Both are leaned on — the first for the normal-form and perturbed-residue material, the second for the torus formula — and their computational consequences are conditional on those stated results *and* on effective input operations. “Not independently machine-verified” is a weaker statement than this one, and was the only form the merged text carried.
- **Every imported algorithm would still need its own review.** A complete implementation would require a separate engineering and proof review of each algorithm imported here, beyond anything the executed checks establish.
- **An umbrella layer inherits no guarantees.** A general symbolic layer can compose expressions freely, but it cannot claim the algorithmic guarantees of its strongest subdomain for every expression it contains. The architecture proposed in [section 16](#) is precisely such a layer, which is where the prohibition most needs stating.

17.6 Shared contract boundaries of all three prototypes

Six statements hold of all three delivered packages and are recorded here once.

1. **None of the three is the architecture.** Each source article states this in its own text, and [section 16](#) repeats it: the proposed object layers, capability queries and result protocols are *proposed interfaces*, and most of that architecture is exported by no package at all. A small, separately delimited prototype demonstrating exact sparse or rational arithmetic does not purport to implement it.
2. **Only the public constructors’ outputs may be passed to the arithmetic routines.** The internal heads — `RHData` in `RationalHahn`, `HData` and `RData` in `SurrealCASCore`, `HR` in `HahnRational` — are private. *Manually forging them bypasses validation and lies outside each package’s own contract.*

3. **None of the three is a hardened parser** for arbitrary or untrusted Wolfram expressions, and arbitrary malformed inputs are not handled as a production-grade general symbolic language would handle them.
4. **Rejection is a statement about the core, not about mathematics.** When a kernel rejects an input — `Pi`, a machine float, `Exp[t]`, `Sqrt[t]`, an irrational exponent coordinate, a mismatched parent — that means *this core does not implement that representation*, not that the corresponding exact surreal value cannot be defined. The first belongs in a larger coefficient parent, the second in an exponent or algebraic extension, the third in a functional or lazy-series extension.
5. **The three non-implementation lists stand separately and must not be pooled.** `RationalHahn` omits the real closure, complex pairs, algebraic root extensions, arbitrary exponent groups, lazy infinite supports, transcendental functions, general complex powers and proof checking. `SurrealCASCore` omits algebraic coefficient fields, algebraic root selectors, infinite series generators, certified analytic domains, arbitrary Conway cuts and the transserial layer; it also has no *standard part*, which is an omission observed by inspecting its exported symbols rather than one the package declares. `HahnRational` omits general Hahn coefficient programs, *rational-exponent parent refinement*, algebraic closure, arbitrary analytic germs, transseries, global exponential extensions and contour integration. Merging these into one list would understate each package’s gaps and overstate the group’s coverage.
6. **No canonical infinite circular phase is claimed**, by any of the three, and none of the three implements any phase provider. The trivial character is a convention and never a default ([sections 13.8](#) and [13.9](#)).

18 What was actually executed

*Three separate provenance records follow, never one aggregate. **The Wolfram check counts 34, 110 and 88 must not be added, and the Python counts 69 and 323 must not be added.** One of the source articles explicitly forbids that arithmetic for its own two suites, and the same reasoning applies across articles: different suites check different things, and their counts are not a proof of that many general theorems.*

18.1 What the three archives recorded

RationalHahn (capability tiers). The arithmetic definitions corresponding to `RationalHahn.wl` and the checks corresponding to `Checks.wl` were **submitted as code** to a stateless connected Wolfram Language evaluator. *The local package path was explicitly not loaded into a separately installed kernel.* The first evaluation reported **34 checks, 31 passed**. Three *test expressions* were then corrected — the inverse-cosine endpoint expansion needed the uniformizer $2 \arcsin(u/\sqrt{2})/(\sqrt{2}u)$ rather than `ArcCos[1-u^2]` used without an explicit positive square root, and the Cayley and trigonometric-algebraization identities needed **Together** rather than **Cancel** alone on a sum of rational terms — and all three passed in a subsequent evaluator call. The distributed suite contains the corrections.

Three disclosures of this record survive verbatim.

- “This is not reported as one original 34/34 run.”
- The first connector evaluation emitted warnings about global-symbol preprocessing and messages

during the inverse-cosine series request, and **no claim of a warning-free initial evaluation is made**.

- An additional request to simplify $\arccos(1 - u^2) = 2 \arcsin(u/\sqrt{2})$ for $0 < u < 1$ **remained unevaluated** and *is explicitly not counted as a passed computer check*; the positive-branch identity is justified by the ordinary half-angle identity and inverse-function ranges, and a failure to simplify it is not evidence against that mathematical proof.

The promised 34/34 local run is described only as an *expected* reproducibility result, not a performed one. Of the 34 checks, only 22 touch the kernel (eight of those are negative or rejection tests); the remaining 12 are ordinary **Series, Together** and **Simplify** identities that never call the kernel at all.

SurrealCASCore (layered representations). The *actual* `SurrealCASCore.wl` and `TestCore.wls` files were transferred as literal text, written into one temporary directory, and `TestCore.wls` was `Get`-loaded — it in turn loads the package from the same directory. The recorded result was

```
<|"Kernel" -> "15.0.1 for Linux x86 (64-bit) (July 2, 2026)",
  "Checks" -> 110, "Passed" -> 110, "Failures" -> {}|>
```

with no warning messages on the final run: **20 deterministic checks plus 90 seeded property checks**, using `SeedRandom[20260921]`. The record discloses that *an earlier inline-definition exploratory run passed its 105 checks but produced a static undefined-global-symbol warning, and is explicitly not the reported result*; five additional deterministic tests were included in the final run. A separate exact-symbolic script `verify_examples.py` was recorded as running under **Python 3.13.5** and **SymPy 1.14.0** with **69 of 69 checks passing**, using `random.Random(20260921)` and pinning `sympy==1.14.0`.

HahnRational (Gaussian prototype). The package's executable definitions and the test body were loaded via `Get[StringToStream[...]]` in the connected kernel, preserving sequential package-context parsing. The recorded result was

```
<|"Kernel" -> "15.0.1 for Linux x86 (64-bit) (July 2, 2026)",
  "Tests" -> 88, "Passed" -> 88, "Failures" -> {}|>
```

with `SeedRandom[20260921]` for the randomized polynomial examples. The record discloses that *a preliminary direct evaluation also passed the same tests but emitted namespace and documentation setup messages*, and that the clean package-loading run is the recorded final validation. An independent Python script was recorded as passing **323 exact checks** under **Python 3.13.5** and **SymPy 1.14.0**, covering quotient-algebra matrices, trace and residue identities, separating denominators, radius-free coefficients through exponent sixteen, exact geometric remainders, local power-series identities, and valuation-based contour counts.

Common to all three records. The same kernel — **Wolfram Language 15.0.1 for Linux x86 (64-bit), July 2, 2026** — reached through a *connected evaluator*, on the same date (September 21, 2026), with **no local installation exercised by any of the three**. Two of the three additionally used Python 3.13.5 and SymPy 1.14.0. **None of the three original packages supplied a Lean file, formalization blueprint, formalization audit, or pinned proof-assistant dependencies.**

18.2 What was independently reproduced for this report

The following is the merged report’s record of a separate Windows run on September 21, 2026, distinct from the three archives’ recorded runs. It is preserved as historical verification evidence, not a claim about the current review environment.

Environment. Wolfram Engine at C:\ProgramFiles\WolframResearch\Wolfram\15.0\wolframscript.exe, reporting **15.0.1 for Microsoft Windows (64-bit) (July 2, 2026)** — the same release and build date as the archives’ Linux kernel, on a different platform. **Python 3.14.4** with **SymPy 1.14.0**; since the archives recorded their Python runs under 3.13.5, the Python results reproduce across a minor Python version as well.

Package	Wolfram recorded	Wolfram reproduced	Python recorded	Python reproduced
RationalHahn	34 checks, corrected suite	Does not parse as shipped (12/34: exactly the 12 checks that never call the kernel); 34/34 after a one-character fix	—	—
SurrealCASCore	110/110	110/110	69/69	69/69
HahnRational	88/88	88/88	323/323	323/323

The Windows record reports matching check counts once **RationalHahn** is repaired; as delivered that package loads in no kernel at all, so the 22 rational-kernel checks its archive records cannot be run from the shipped file (§18.3). The quoted sessions of [section 17](#) were also re-executed and produce the outputs shown, including the cancellation trap $\{1, 1\}$, the acceptance of the half-integer exponent $1/2$ by SurrealCASCore, its rejection of $\sqrt{2}$, and all three displayed order comparisons in their own conventions.

18.3 The one real defect: RationalHahn cannot be loaded as distributed

Remark 18.1 (A verified packaging defect, found by this merge). **RationalHahn.wl does not parse.** The kernel’s own diagnosis is

```
Syntax::sntue: Unexpected end of file (probably unfinished expression).
(line 79 of ../RationalHahn.wl)
```

The cause is in **RHInverse**, at lines 48–49 as shipped:

```
RHInverse[a_RHData] := If[a[[2]] === 0,
  Failure["DivisionByZero",<||>],RHMake[a[[3]],a[[2]],a[[1]]];
```

The `If[` opened on the first line is *never closed*. A brace census gives exactly one unmatched `[` in the whole file (179 against 178, outside comments and strings), whereas the other two packages balance exactly; the whole file parses into 38 top-level expressions once a single `]` is inserted before the `;`. **The fix is one character:** `...a[[1]]];` for `...a[[1]]];`.

Consequence. `Get["code/RationalHahn.wl"]`, the driver `RunChecks.wl`, and the README’s `wolframscript -file code/RunChecks.wl` instruction cannot load the package as delivered. The recorded Windows run of the unrepaired driver returned `Checks -> 34, Passed -> 12`, the twelve

passing checks being precisely the ordinary `Series/Together/Simplify` identities that never call the kernel; all 22 kernel checks failed. After the one-character repair the same driver returned `Checks -> 34, Passed -> 34, Failures -> {}` with no warnings.

Therefore, as delivered, the archive’s “expected 34/34 local run” is not merely unverified but currently impossible. A merged report that repeated that `wolframscript` instruction without this note would be printing a false reproducibility claim; the instruction is repeated in [section D](#) *only* together with the repair.

This is consistent with, and explains, the archive’s own verification record, which states that the definitions were submitted as code to the stateless evaluator and that “the local package path itself was not loaded into a separately installed kernel”. The package was never `Get`-loaded because it *cannot* be. The archive does not claim otherwise; but neither does it report the defect — it is not among its list of three corrected checks, which were all harness-level. Its README also lists `SHA256SUMS.txt` among the distribution files. That is not a defect: the archive did ship one, and this repository does not carry it, because the collection ships no checksum manifests.

SurrealCASCore and HahnRational were recorded as loading cleanly, with their harness counts matching the archives. Their source files are also bracket-balanced, though that syntax check alone does not establish a successful Wolfram run.

18.4 What the executed checks do and do not establish

During the maintained documentation review, fresh temporary copies of the two unchanged Python scripts passed their separate suites under Python 3.13.14 and SymPy 1.14.0: 69/69 for SurrealCASCore and 323/323 for HahnRational. The Wolfram sources were inspected for their public contracts and the documented packaging defect; no new Wolfram execution is claimed by this review.

The executed checks are exact finite algebra and formal-series checks. They are **not** a formal verification of: Hahn summability; the surreal foundations; the effective real-closure construction or real-closed-field quantifier elimination; global analytic structures; the phase classification or the E_α family; AG’s germ division or preparation; the ISSAC 2026 transseries zero-test; CT’s integration theory; or any infinite or transfinite recursion. A finite series check is evidence about *those coefficients only*, and a successful driver run does not upgrade finite coefficient calculations into proofs of the infinite constructions. None of the three prototypes executes real-closed-field quantifier elimination, constructs all algebraic roots, solves arbitrary support-recursive equations, or certifies the supplied manuscripts’ analytic results. **Compatibility with Wolfram versions other than 15.0.1, and with Mathics, was not tested in those runs.** The original packages supplied no proof-assistant verification. Consult `docs/FORMALIZATION.md` for the containing repository’s current Lean coverage, which must be matched to each statement and its hypotheses.

19 Implementation priorities, costs, and regression boundaries

19.1 A union staged plan

All three sources propose staged plans; the union below is their common shape with each source’s distinctive emphasis retained. *The ordering is not a claim that later stages are easy*; it prevents their difficulties from blocking a valuable exact algebraic system.

Stage 1 — exact ordered rational algebra and surcomplex pairs. Validated coefficient and exponent domains; rational fractions; arithmetic, equality, sign, valuation and finite standard parts; surcomplex pairs and exact squared norms; efficient multivariate rational normalization; exponent-domain embeddings; expression sharing; exact matrix and polynomial adapters; a stable serialization format; and a precise public data contract whose *errors* are tested as thoroughly as its successful identities. This stage already supports many all-scale geometric calculations, and many polynomial-manuscript examples including resultants, discriminants, finite quotient algebras and initial polynomials for coefficients whose valuations are available.

Stage 2 — algebraic extensions and root selectors. Real-root selectors, ordered extension towers, exact comparison, complex polynomial roots, and their surcomplex pairs; finite-dimensional linear algebra and basic polynomial tools over those coefficients; rank-one or nested Puiseux computations. **Keep root selection independent of ordinary numerical approximation:** use exact Newton data as a *view* of a root, never as a substitute for its selector, and preserve branch identity under refinement. The central deliverable is exact algebraic closure with robust branch selection, not a universal root expansion.

Stage 3 — support-certified series, grids, precision and compressed cutoffs. Positive-grid constructors (finite unions), finite coefficient-dependency procedures, reciprocal and binomial expansions, precision propagation, rank-one finite truncations, rational-grid coefficient extraction, and compressed cutoff objects. Initially restrict support grammars enough that every advertised coefficient request terminates, and preserve exact origin graphs behind truncations. **Implement [theorem 6.1](#)'s dependency guarantee before offering unrestricted-looking sum and composition syntax.** High-rank requests should remain compressed or return an informative unsupported-finite-view result.

Stage 4 — analytic lifts, coherent data, quotient algebras, residues and periods. Ordinary-centre Taylor lifts; finite-angle trigonometry; inverse functions with endpoint ramification; logarithm branch policies; finite trigonometric polynomial algebraization; regular implicit germs; Hahn-coherent positive perturbations with executable coefficient rules; coherent ordinary-contour functionals for a controlled coefficient-function grammar; finite quotient algebras, complete-intersection deformation, residue pairings, and rational contour periods. Then add preparation, division and residue methods from the supplied sources *under their stated hypotheses*. The arbitrary radius-free analytic-germ generalization should follow only when an effective ordinary-germ division backend exists for the chosen language. Reuse host coefficient functions only with domain checks.

Stage 5 — exp-log and transserial scales, with phases separate. Develop the exp-log hierarchy and its compatible surreal interpretation; admit forms of transserial composition and differential operations, each with its own support and leading-term algorithms; keep general semantic exp-log nodes available even when normalization is incomplete. **Surcomplex phase policies, sectorial summation, resurgent special-function extensions, hyperseries and global chart compatibility are separate projects.** They should not delay a usable exact algebraic implementation, and must not be marked implemented merely because the parser accepts their names.

19.2 Complexity must include representation and output size

Multiplying sparse supports of sizes m and n initially produces at most mn pairs before collection. This is an arithmetic-operation count, not a bit-complexity estimate: the bit lengths of rational coordinates and algebraic coefficients matter as much as the number of terms, and exponent comparison and coefficient growth can dominate the actual cost. Repeated cross multiplication causes fraction growth, so gcd reduction, common-subexpression sharing and delayed expansion are important improvements; a straightforward power routine is intentionally not an optimized exponentiation algorithm. A compact rational expression can have a very large series expansion, and an algebraic root can have complicated nested coefficients, so store shared exact graphs, cache verified leading data, and make expansion demand-driven. Recursive exponent comparison may dominate an otherwise small calculation.

For grid coefficient extraction the finite box of [theorem 6.1](#) proves termination but can be large; integer linear-equation methods, dynamic programming and memoized convolution can improve it while preserving exact treatment of coincident sums. Finite-algebra computations scale with the quotient dimension and coefficient complexity. Polynomial root descriptions can have large algebraic degrees, and *quantifier elimination is a sound foundation, not a guarantee of low computational cost*; specialized Newton and Hensel methods can provide far better useful views without replacing the exact fallback.

Remark 19.1 (Output-representation limits are not complexity). **No algorithm can print an infinite cutoff prefix as a finite explicit term list.** This is an output-representation limitation, not merely poor complexity. Likewise, **no time budget transforms the halting reductions of [section 3](#) into decision procedures.** A useful system reports whether a request is expensive, outside its implemented domain, impossible in the requested output format, or undetermined by its current evidence. Outside a decidable domain, every budget exit must remain distinguishable from mathematical falsehood. A total decision procedure need not be the first procedure tried: fast domain-specific simplifiers can precede an expensive complete algebraic fallback.

Resource accounting should include coefficient bit sizes, polynomial degrees, extension degrees, exponent-tree sizes, cached coefficient counts and support-dependency counts. *A single “precision” integer is inadequate for a hierarchy of infinitesimal scales.* Interactive output should show the leading scales and permit inspection of the exact underlying descriptor.

19.3 A union regression and acceptance list

A regression suite should include much more than successful expansions. The union of the three sources’ lists:

- Cancellation to an exact finite value; zeros hidden behind long coefficient prefixes; exact identity tracking proving $A - A = 0$.
- A mathematically valid but *nonenumerable* finite-cutoff request; preservation of the rank-crossing counterexample (7.5), so that increasing an ordinary integer precision parameter never claims accuracy beyond u ; and prevention of a high-rank cutoff being handled by an endless rank-one loop.
- Rejection of $\sum_{n \geq 1} t^{1/n}$ as an unordered-support Hahn sum; rejection of positive but non-well-ordered supports and of well-ordered unions with infinitely many contributions at one exponent.

- Rejection of the geometric expansion of $\sum t^n z^n$ at $z = t^{-2}$ while *permitting* the separate rational continuation (9.16); distinguishing $\exp(t)$ from an invalid strong expansion of $\exp(1/t)$.
- Retention of the difference between an exact factorial Hahn series and an ordinary divergent series after real specialization.
- Division by *exact* zero versus division by an object of *unknown* zero status; standard part of an infinite input; incompatible exponent domains and parent mismatches.
- Root branches separated only infinitesimally, which ordinary decimal neighbourhoods cannot separate; algebraic root modules verifying defining equations, branch separation, parent embeddings and multiplicities including repeated roots.
- Quotient-algebra tests including repeated roots, so that a nondegenerate *residue* pairing is not confused with a nondegenerate *trace* pairing; the four-point example; the overdetermined length-drop counterexample; and the sharp Hensel threshold example (8.5).
- Contour modules detecting poles on the contour, using the transformation determinant $\det C$, and distinguishing ordinary-parameter Hahn chains from fine-continuous paths.
- Actual versus anchored logarithms; preservation of branch conditions in logarithmic rewrites; unsupported infinite circular phases returning an obligation; and a global surcomplex exponential module making the two values of (13.9) explicit under the two named policies and *never caching them as if they belonged to one function*.
- Differentiation under each named convention; refusal to decide coefficient equality from insufficient numerical precision; and a check that **Normal**-style conversion cannot silently promote a truncation to an exact equality.

For each successful approximate calculation, verify the defining residual and the claimed error certificate whenever that check is available. A displayed root expansion without a branch certificate and residual bound is only a *candidate*; with those certificates it is a dependable computed view of an exact root object. **Negative tests are essential:** rejecting an inadmissible sum or returning a phase obligation is often the correct mathematical answer, not missing functionality. These acceptance tests target errors that ordinary complex arithmetic tests would miss, and they *do not replace proofs of the general algorithms*.

20 Capability and operation matrix

The following is the union of the two capability matrices in the sources, with an added column naming which of the three kernels, if any, implements each row. “Effective” always includes the stated coefficient and exponent operations. “Conditional” means that support, branch, coefficient-algorithm or category hypotheses must be supplied and checked; it is not a claim of a general terminating algorithm.

Layer or operation	Exact representation available	Algorithmic promise and restriction	Implemented in
Rational monomial core: $+$, $\times$	Polynomial fractions with exact coefficients	Terminating arithmetic and decidable equality	All three

Layer or operation	Exact representation available	Algorithmic promise and restriction	Implemented in
Inverse	Rational node	Exact for a proved nonzero value	RationalHahn; HahnRational; SurrealCASCore <i>on fractions only</i>
Equality and real order	Rational or algebraic data	Total for the rational core; partial in general; total in supported structured classes including appropriate transseries zero-test domains	All three (real comparison)
Valuation and leading coefficient	Rational node	Exact finite polynomial calculation; cannot be total for arbitrary coefficient programs	All three
Standard part	Finite value with known leading data	Exact on finite inputs; may need a coefficient query plus a finiteness certificate	RationalHahn; HahnRational. Not SurrealCASCore
Conjugation, Re, Im, $ z ^2$	Complex rational core	Exact, coefficientwise	HahnRational (all four); SurrealCASCore (conjugation only); not RationalHahn
Rational / fractional exponents	Refined exponent lattice	Exact after common-denominator refinement	SurrealCASCore only
Real closure and complex roots	Finite equations plus ordered root selectors	Implementable terminating algebraic operations; cost not bounded	None
Modulus and algebraic roots	Selected real/algebraic extension	Exact root descriptor and branch selection; finite normal form not guaranteed	None
Infinitesimal reciprocal and powers	Rational/root node or positive-support series	Coefficients effective with finite dependency procedures; exponent/coefficient promotion allowed	None (series layer)
Coefficient at one exponent	Finite expansion or grid	Effective positive grids and nested supported descriptions; arbitrary supports need extra data	None
All terms below a cut	Compressed exact rational object	A finite list may not exist even for a valid Hahn series	None
Ordinary analytic lifts	Germ descriptor at $c + \eta$	Exact under branch and regular-centre hypotheses; coefficients may leave the algebraic core	None
Radius-free Hahn germs	AG's $\mathcal{A}_n$ descriptor	Strictly weaker than common-domain coherence; needs separate support and germ programs	None
Real exp and log	Gonshor-semantic graph	All real inputs for exp, positive for log; normalization only where the backend supports the scales	None
Canonical complex exp	Strip object on $\mathcal{H}$	Imaginary part finite; real growth may be infinite	None
Canonical sine and cosine	Finite-angle or $\mathcal{V}$ strip object	Real part finite; arbitrary imaginary growth allowed	None
Inverse real trigonometry	Finite-angle or ramified germ data	Natural real-surreal domains; endpoint branches explicitly selected	None

Layer or operation	Exact representation available	Algorithmic promise and restriction	Implemented in
Infinite circular phases	Named character extension or named E_α policy	Exact only after a coherent provider is fixed; extraction of fin and ℓ may not be effective	None
Polynomial / trig solving	Algebraic or finite cluster objects	Complete over the effective algebraic core; broader coefficients need effective zero and root capabilities	None
Function differentiation	Germ or coherent derivative descriptor	Scalars fixed; not a scalar-field derivation	None
Scalar derivation	Named differential-field structure	Only supported identities and summable families; different from formal D_t	None
Root counts and finite schemes	Polynomial algebra, matrices, selected roots	Initial polynomials and profiles are exact; deformation methods need effective ordinary division and support	None
Integration, residues, periods	Coefficientwise integral descriptor; algebraic period	Common support, ordinary contour and analytic-domain conditions retained; no universal fine-local fundamental theorem	None
General lazy Hahn equality	Finite program plus semantic certificate	No total universal procedure, even for $\{0, 1\}$ coefficients on $\mathbb{N}$	None
Numerical display	Coefficient approximation or explicit scale specialization	Not an ordinary numeric value of an infinite surreal; a specialization is a simulation	None

21 Conclusions

21.1 The representable subsets and their operations

A substantial exact symbolic surreal CAS is possible without a universal representation of all surreals. A large subset is available immediately: finite normal forms over an effective coefficient field, rational expressions in ordered monomials, and all algebraic extensions of a specified effective ordered monomial field. The real-closed fields F and their algebraically closed complexifications C of [theorem 5.3](#) make this precise — every value there has a finite algebraic description even when its Hahn normal form is infinite. This already gives exact arithmetic, comparisons, polynomial roots, linear algebra and a large part of finite geometry at infinite and infinitesimal scales.

Beyond that centre, exact representation remains possible without complete normalization: certified recurrence-defined series, positive-grid Hahn generators, regular implicit solutions, ordinary analytic lifts, Hahn-coherent functions on specified ordinary domains, radius-free germs, and selected transserial expressions. Exact field arithmetic, real comparison, conjugation, algebraic root specification, finite polynomial algebra and finite linear algebra are realistic terminating operations in the effective core. Leading terms, standard parts, Newton profiles, root-cluster counts and certified series refinement are realistic when the representation supplies the required valuation and support services. Finite quotient algebras can often compute residues, traces and multiplicities without expanding individual roots. Formal and coherent differentiation, regular implicit solving, positive-support Taylor substitution, local inverses, preparation and coefficientwise contour operations can be implemented for suitably described functions. Global real exponentiation can be admitted with its established surreal semantics; global surcomplex transcendental operations must additionally

state their extension and branch policies; and integration must name its derivative and function category.

Their exactness means that their *semantics are specified*. It does not imply that all their identities are decidable or that every requested expansion is finite. **There is no largest finite-description class that simultaneously solves all these problems**; enlarging the expression language usually changes its closure, equality, order and evaluation guarantees. The right product is a family of compatible exact domains with explicit capabilities.

21.2 Four independent sources of limitation

The boundaries have four different origins, and conflating them obscures what can be fixed and what cannot.

1. **Cardinality.** Finite syntax cannot name every real, a fortiori not every surreal ([theorem 3.1](#)). No language extension removes this.
2. **Undecidability.** Arbitrary computable coefficient streams admit no universal terminating equality test, no total support-admissibility validator, and no universal leading-term or sign algorithm in higher rank, even under a nonzero promise ([theorems 3.5, 3.10 and 3.11](#)). The representation, not the value set, is what carries the guarantee.
3. **Higher-rank support geometry.** A valid finite-valuation cutoff request need not have a finite term-list answer, and rank one alone does not suffice either ([section 7.3](#)). This is an output-representation limitation.
4. **Analytic category and branch choice.** Fine-local germs, common-domain coherent data, radius-free germs and contour-integrable data are different categories; global logarithms, infinite circular phases and global surcomplex exponentials require declared policies ([sections 9, 11 and 13](#)).

None of these makes the implementable subsets uninteresting. Together they determine a sound software architecture.

21.3 Attach the necessary information to the object

The constructive lesson is uniform across all three sources. A *parent* identifies arithmetic and order, including coordinate significance. A *support presentation* identifies coefficient dependencies. A *root descriptor* identifies a branch. A *derivative* identifies its constants. A *contour* identifies a chain category and pole-avoidance conditions. A *global surcomplex function* identifies its extension policy. With those interfaces the strong formulas of the supplied manuscripts become either genuine algorithms on effective inputs or clearly labelled conditional constructions; without them, elegant symbolic notation can conceal undefined operations or nonterminating searches.

The most useful meaning of “exact surreal support” is therefore not a single unchecked promise. It is a visible contract: an exact denotation, a known representation class, a declared set of effective operations, certified remainder information when approximating, and explicit analytic conventions. The central design principle is not to equate an infinite object with an approximation, or a mathematical existence theorem with a program: preserve exact objects, expose certified finite information about them, and make every change of representation, scale, domain or analytic interpretation explicit. That principle turns the supplied surcomplex theory into a credible computer-algebra program rather than a collection of unsafe rewrite rules. The three delivered prototypes

demonstrate the central algebraic point — *exact multi-rank arithmetic does not require truncating infinitesimals or replacing infinity by a numerical limit* — within three narrow and separately stated envelopes; the more ambitious layers are specified here as extensions, with their mathematical and computational obligations made explicit.

A Source map and boundaries of reliance

This appendix merges the two source-audit tables from the sources into one, with an *origin* column naming the manuscript and, where relevant, the revision. **References to theorem names are references to the supplied source files, not claims that those manuscripts implemented the algorithms.** The final column preserves both tables’ “what is NOT inferred” discipline.

Origin	Source result or convention	Role in this report	What is <i>not</i> inferred
SA (2) and (3)	Strong summability and the Hahn–Neumann positive-monoid lemma	Semantic certificates for products, inverses and infinitesimal evaluation	No automatic zero test; no effective support decomposer.
SA (2) and (3)	“Evaluation of arbitrary formal series” after a sufficiently small rescaling	Existence of sufficiently small analytic scales (section 6.5)	No algorithm extracting a global bound from an arbitrary program; the scale may leave the workspace.
SA (2) and (3)	Hahn-coherent data on a fixed domain, and the composition theorem	theorem 9.2 , (9.15)	No gluing of arbitrary unrelated fine-local germs; no unconditional global-support discovery.
SA (2) and (3)	Preparation and exact root lifting; implicit simple roots	(8.8); finite clusters; coefficient recursions	No completion of an arbitrary transfinite recursion by a finite loop.
SA (2) and (3)	Contour and moving-pole residue theorems	(11.1), (10.3)	No unrestricted fine-topological Riemann integration; not a full-class Riemann integral.
SA (2) and (3)	Global fine-analytic logarithm; all-scale rigidity	section 13.6 , section 12.8	No universal nonexistence of transcendental functions; not a coherent primitive of $1/z$; no implemented global atlas.
SA (2) and (3)	The E_α phase family with $E_0(i\omega) = 1$, $E_\pi(i\omega) = -1$	section 13.9 as an interface requirement	Not a published uniqueness classification ; no claim that no other convention can be chosen; not automatically effective.
SA (2) and (3)	The fine topology, in which set-indexed convergent nets are eventually constant	section 7.7	Truncation sequences are not fine-convergent; a rank-one valuation topology is a different structure.
TR	Finite-angle circle, polar decomposition, inverse functions	sections 13.2 and 13.5	No necessity for infinite circular angles; extracting u_0 still needs a standard-part capability.
TR	Canonical strip functions $E_{\mathcal{H}}$, Sin, Cos	section 13.4	No uniquely specified evaluation beyond the strips; $\sin \omega$ and $E(i\omega)$ are not fixed.

Origin	Source result or convention	Role in this report	What is <i>not</i> inferred
TR	Character classification of circular group-law extensions; infinite periods	section 13.8 , theorem 13.1	No default value of $\sin \omega$ forced by the local axioms; completeness is claimed only within TR's framework and is not extended to SA's E_α .
TR	Tangency and valuation loss (8.10) ; the $2n$ root bound; the positivity example	sections 8.7 , 13.10 and 13.11	Replacing n by ω is meaningless; ordinary-angle sampling is unsound.
PA	Initial polynomial and its ball counts; Newton profiles; Hensel cluster lifting	sections 8.2 to 8.4	The valuations must be supplied for general coefficients; no promise to list a positive-dimensional family; classical Newton–Puiseux is a rank-one specialization.
PA	Root-perturbation bounds and the squarefree/discriminant conditions	section 8.5.1	Not a multivariate statement; the polynomial must be normalized to the finite-coefficient setting first.
PA	Residue functional, trace identity, residue-vs-trace determinants	sections 10.1 and 10.2	Not a universal rewrite rule; not an identification with a chosen Grothendieck-residue theory.
HC	Fixed-domain coefficient rings; positive-operator inverses; coupled finite-free systems; $S + E^*$ bounds	sections 6.6 and 10.6	Do not synthesize finite certificates or decide identities of arbitrary holomorphic coefficients; the extensions are explicitly delimited.
AG	Radius-free germ algebra $\mathcal{A}_n$; evaluation and translation; finite jets	sections 9.7 and 9.9	Not every convergent germ is already a CAS object; <i>finite mathematical dependencies need not be effectively enumerable</i> — AG's own warning.
AG	Weierstrass preparation and division operators (9.20)	section 9.10	Exact formulas, not an implemented routine ; an arbitrary choice-based splitting is not executable; semidecidable germ zero-detection may not terminate.
AG	Normal forms, multiplication matrices, perturbed residue, trace identity; length preservation	sections 10.4 and 10.5	Requires an isolated leading complete intersection with n equations in n variables; an overdetermined system is not covered.
AG	Root separation κ_a and certified continuation	section 8.5.2	Conditional on obtaining the exact bounds; the equality case $\delta = 2\kappa_a$ cannot be accepted.

Origin	Source result or convention	Role in this report	What is <i>not</i> inferred
CT	Coefficientwise Hahn integration and Stokes; microscopic rescaling	sections 11.2, 11.4 and 11.6	Not a limit of fine-topological Riemann sums; not an algorithm deciding admissibility of every input map; Jordan separation alone gives no integration operation.
CT	Separating torus with $\det C$; algebraic rational periods	sections 11.3 and 11.5	Imported <i>conditionally</i> under CT's finite-algebra and transformation hypotheses; a pole on the contour is outside the definition.
Published literature	Conway, Gonshor, van den Dries–Ehrlich, Poonen, Perrucci–Roy, van der Hoeven, Berarducci–Mantova, Costin–Ehrlich, Bagayoko–van der Hoeven, Mantova 2026, Richardson, Chen–Fang–van der Hoeven, Bournez–Guilmant, Cattani–Dickenstein–Sturmfels	Foundations, exponential structure, algebraic closure, quantifier elimination, transseries, derivations, integration, zero tests	See section A.1 . No claim that one cited framework equals any other.
Official CAS documentation	Wolfram and Sage reference pages, consulted September 21, 2026	section 15	A statement about the interfaces inspected, not an exhaustive survey; these motivate reuse, not a claim that the facilities already implement surreal semantics.
The three source articles	The countability bound, both halting reductions, the core theorem, the grid lemma in two constructions, the precision rules, the cutoff obstruction, the proposed architectures, and the executed checks	Throughout	No priority claim is made for any of these elementary deductions. The executed checks are not proofs of the general theorems.

A.1 A note on what is not claimed

Collected for the published literature specifically: hyperseries and the Bagayoko–van der Hoeven representation theorem are cited for the mathematical result only, not for a finite universal encoding or an implementation claim, and the countability argument still applies to them. The Hahn-field algebraic-closure input (Poonen) is a theorem about the complete mathematical field, not a terminating algorithm for arbitrary coded series, and the full abstract Puiseux field still contains noncomputable coefficient sequences. Mantova’s 2026 monotonicity and Taylor-approximation results, Berarducci–Mantova’s derivations and germs, Bournez–Guilmant’s stable set-sized fields, and the elementary-extension theorem for a surreal exponential structure are all *semantic theorems for specified classes*, not universal decision algorithms, and a birthday bound is not a finite executable representation or a complexity estimate. Costin–Ehrlich integration is established background for suitable analytic and resurgent classes and is explicitly not identified with the local Taylor-lift interface or with the manuscripts’ coefficientwise contour functional. Richardson’s undecidability results have restrictions that matter and must not be inflated into a blanket claim about every

restricted exponential-constant identity problem. The ISSAC 2026 D-algebraic transseries zero-test holds under its effective-field, transbasis and solution-selection hypotheses and is not a zero-test for arbitrary series-producing programs.

B Minimal backend contracts and component responsibilities

A **coefficient backend** should specify exact construction; zero and sign capabilities; arithmetic; conversion to larger coefficient domains; and whether its ordinary constants admit certified numerical enclosures.

A **monomial (exponent) backend** should specify equality, multiplication (addition of exponents), order, rational-root promotion where supported, and the embedding used to interpret it inside the surreal field — including its coordinate significance, which is part of the descriptor and never inferred from variable names.

A **support backend** should distinguish a *semantic proof of well ordering* from *algorithms* for membership, finite coefficient dependencies, leading-term discovery and cutoff materialization, and it should declare which of those operations are total.

An **analytic backend** should specify the ordinary or surreal domain; branch and extension identifiers; Taylor coefficients; the derivative mode; the regularity category (fine-local, common-domain coherent, radius-free germ, or contour-integrable); and the admissible composition rules.

An **operation result** should preserve its exact origin and its obligations. If the answer is a truncation it must include a remainder descriptor and the error calculus used to obtain it. If the answer is conditional, the condition must remain visible in further transformations. If an unsupported request has a well-defined semantic node, the system may retain that node without pretending to have evaluated or normalized it. The component responsibility table of [section 16.4](#) applies to this appendix in full.

C Notation and semantic conventions

Notation	Meaning
No , SC	The surreal class field and its complexification No [<i>i</i>].
t^γ	The Conway monomial $\omega^{-\gamma}$, <i>not</i> a generic power operation.
$\mathbb{R}_{\text{alg}}$	The real algebraic numbers.
$\mathbb{Q}_{\text{lex}}^r, \mathbb{Z}_{\text{lex}}^d$	Rational or integral exponent vectors ordered by their first differing coordinate — with the coordinate significance declared by the domain descriptor (theorem 5.2).
E_r, F_r, C_r	An ordered rational monomial field, its relative real closure, and the latter’s complexification.
$v(A), \text{lc}(A)$	The least Hahn exponent and its coefficient; $v(0) = +\infty$, used only as a distinguished marker.
$ z $	The surreal modulus $\sqrt{z\bar{z}}$, distinct from the valuation.
$\mathfrak{m}, \mathcal{O}, \text{st}$	Infinitesimals, finite elements, and the standard part on finite elements.
fin	TR’s finite-part projection, removing all negative-exponent terms; additive but <i>not</i> multiplicative.
$\Pi, \Pi \oplus \mathbb{Z}$	The purely infinite additive class, and the ordered subring used for the floor (7.7).

Notation	Meaning
E^*	The monoid of finite sums from a positive well-ordered exponent set E .
$O_v(\beta)$	A remainder with valuation at least β , including zero; the <i>weak</i> inequality unless a strict one is displayed.
$U^\sharp, f^\sharp$	Finite surcomplex points with ordinary standard part in U , and the ordinary analytic Taylor lift.
$\mathcal{H}(U), \mathcal{A}_n$	Common-domain Hahn-coherent coefficient data, and AG’s radius-free Hahn-germ algebra (strictly weaker).
D_z, D_t (or D_T), ∂	Coordinate derivative, formal scale-variable derivative, and a separately selected surreal-field derivation; three different operators.
E_α	SA’s explicitly phase-selected global surcomplex exponential; <i>not</i> an unqualified canonical complex exponential.
U_χ	TR’s circular group-law extension by a character $\chi : (\Pi, +) \rightarrow (\mathbb{T}(\mathbf{No}), \cdot)$.
$\mathcal{H}$ (strip), ψ	TR’s canonical strips: finite imaginary part, and finite real part respectively.
$I_{a,\rho}(P)$	The initial polynomial at a centre and a valuation scale.
$\lambda_P(H), \text{Res}_F$	The top coefficient of H modulo monic P , and AG’s perturbed residue functional.
κ_a	AG’s root-separation constant (8.6).

Per-kernel coordinate conventions, restated.

Kernel	Variable listing	Comparison
RationalHahn	<i>Least</i> -significant-valuation first: $v(t_1) = 1, v(t_2) = \omega, \dots$	Exponent vectors compared after Reverse ; highest-index coordinate first.
SurrealCASCore	<i>Most</i> -significant first; $(q_1, \dots, q_r) \mapsto q_1\omega^{r-1} + \dots + q_r$	Plain lex on $\mathbb{Q}^r$; first coordinate most significant.
HahnRational	<i>Most</i> -significant first; for $\{\mathbf{u}, \mathbf{t}\}$, $v(u) = \{1, 0\}, v(t) = \{0, 1\}$	Plain lex on $\mathbb{Z}^d$; leftmost coordinate most significant; a reversed list is a <i>different parent</i> .

D Reproducibility and archive contents

D.1 Building this report

This article is standalone LaTeX with an internal bibliography. It uses no external figures, data files, or bibliography database. From its own directory:

```
latexmk -pdf -interaction=nonstopmode article.tex
```

Alternatively run `pdflatex` until the references and table of contents stabilize. Standard TeX Live packages and Latin Modern are sufficient.

D.2 Archive contents

Beside this article, the directory carries the executable files of all three prototypes under `code/` and the recorded verification reports under `data/`. **The files in `code/` and `data/` ship verbatim as delivered and are not modified here** — including the defective `RationalHahn.wl` (see

section D.3). The three source manuscripts themselves are not redistributed with this report; the disposition of each source result is recorded in the provenance material of section 1.5 and in the per-result attributions throughout the text.

D.3 Three separate run instructions

RationalHahn — requires a one-character repair first. The archive’s own instruction is

```
wolframscript -file code/RunChecks.wl
```

As delivered this does not work: the package does not parse, and the driver reports `Checks -> 34, Passed -> 12` — the twelve kernel-independent identities — with the kernel’s own `Syntax::sntue` error. Apply the one-character repair of theorem 18.1 to a *copy* of the package (close the `If[ in RHInverse: write ...a[[1]]]; for ...a[[1]]];`), place that copy beside `Checks.wl` and `RunChecks.wl` under the names the driver expects, and the driver then returns `Checks -> 34, Passed -> 34, Failures -> {}` with no warnings. Run it in a fresh kernel or a disposable context: `Checks.wl` uses named global variables for readability, and the package rejects variables that have already evaluated to nonsymbolic values.

SurrealCASCore.

```
wolframscript -file TestCore.wls
python -m pip install -r requirements.txt
python verify_examples.py
```

The harness locates the package relative to its own path, so the working directory need not be the package directory; the Python script regenerates its report beside itself and fails if an assertion fails. Expect 110/110 and 69/69.

HahnRational.

```
wolframscript -file code/test_HahnRational.wl
python code/verify_examples.py
```

The test file loads the package relative to its own path when the package is not already loaded. Expect 88/88 and 323 exact Python checks. The short demo `examples.wl` prints four of the displayed session results.

D.4 Combined statement

One kernel release (15.0.1 of July 2, 2026, recorded on Linux x86-64 by the archives and reported separately on Windows x86-64 at merge time); one date (September 21, 2026); **no local installation exercised by any of the three original runs; counts not additive**, either within an article or across articles; no formal verification of general theorems by these tests; and no Lean files, blueprints, proof-assistant audits or pinned Lean dependencies in the three original packages. These are historical package claims, not a description of the containing repository’s current formalization work. The executed checks are exact finite algebra and formal-series checks, and a successful driver run does not upgrade them into proofs of the infinite constructions.

References

- [1] *Surcomplex Analysis: Infinitesimal Calculus, Hahn-Coherent Holomorphy, and Contour Theory*. Supplied research manuscript, September 21, 2026; file `surcomplex_analysis(2).tex`. **Revision (2)**, supplied to the capability-tiers article, which bundled it unchanged. Identical, after CRLF normalization, to the infinitesimal-calculus-and-contour-theory manuscript behind *Surcomplex Analysis: Holomorphic Functions on No[i]* (`docs/surcomplex/analysis/`) in this repository. Source labels relied upon include `def:strong`, `thm:smallscale`, `def:coherent`, `thm:preparation`, `thm:globallog` and `thm:rigidity`. No refereeing or proof verification is inferred.
- [2] *Surcomplex Analysis: Infinitesimal Calculus, Hahn-Coherent Holomorphy, and Contour Theory*. Supplied research manuscript, September 21, 2026; file `surcomplex_analysis(3).tex`. **Revision (3)**, supplied to the layered-representations and Gaussian-prototype articles, which cited rather than bundled it. Its separate notions of local germs, coherent data, coefficientwise contours and all-scale functions are retained here. No refereeing or proof verification is inferred.
- [3] *Trigonometry on the Surcomplex Plane: Canonical Angles, Infinite Triangles, Infinitesimal Contact, and Hahn-Analytic Oscillation*. Supplied research manuscript, `surcomplex_trigonometry.tex`, September 21, 2026. Byte-identical to the canonical-angles-and-oscillation manuscript behind *Trigonometry on the Surcomplex Plane* (`docs/surcomplex/trigonometry/`) in this repository. Primary basis for the finite-angle, strip, branch, root-count and infinite-phase conventions; source labels relied upon include `thm:polar`, `thm:tangency`, `thm:stripexp`, `thm:polyroots`, `thm:characters` and `thm:infiniteperiods`. Two further archives mentioned inside this manuscript were not supplied and are not treated as reviewed sources.
- [4] *Surcomplex Polynomial Algebra: Order Geometry, Newton Profiles, and Scale-Resolved Stability*. Supplied research manuscript, September 21, 2026; file `surcomplex_polynomial_algebra(2).tex`. The workspaces, initial-polynomial formulas, Newton profiles, Hensel lifting, perturbation bounds and finite residue algebra are used with their stated hypotheses. A fourth, global-divisor manuscript cited inside this file was not supplied and is not used here as an independently inspected source.
- [5] *Hartogs Coherence, Finite Maps, and Residue Duality over the Surcomplex Numbers*. Supplied research manuscript, September 21, 2026; file `surcomplex_research.tex`. In particular the positive-operator calculus, fixed-domain division, coordinate-power perturbations, coupled finite-free systems and finite residue pairings.
- [6] *Infinitesimal Analytic Geometry over the Surcomplex Numbers: Radius-Free Hahn Germs, a Nullstellensatz, and Conservation of Singular Zeros*. Supplied research manuscript, September 21, 2026; file `surcomplex_analytic_geometry.tex`. Especially its Sections 2–4, 6–12 and Appendix B. Its general theorems are not independently machine-verified here.
- [7] *Jordan Separation, Cauchy Theory, and Stokes Formulae on the Surcomplex Plane: A Category-Sensitive Contour Calculus*. Supplied research continuation, September 21, 2026; file `surcomplex_contours(2).tex`. Especially its Sections 2, 4–10 and Appendices A–C. The stated category hypotheses are retained.
- [8] John H. Conway. *On Numbers and Games*. Academic Press, 1976; second edition, A K Peters, 2001.
- [9] Harry Gonshor. *An Introduction to the Theory of Surreal Numbers*. London Mathematical Society Lecture Note Series **110**, Cambridge University Press, 1986. [doi:10.1017/CBO9780511629143](https://doi.org/10.1017/CBO9780511629143).
- [10] Lou van den Dries and Philip Ehrlich. “Fields of surreal numbers and exponentiation.” *Fundamenta Mathematicae* **167** (2001), 173–188. [doi:10.4064/fm167-2-3](https://doi.org/10.4064/fm167-2-3). Erratum, **168** (2001), 295–297. Used for the normal-form and exponential-structure background, not for ordinal-length estimates affected by the erratum.
- [11] Bjorn Poonen. “Maximally complete fields.” *L’Enseignement Mathématique* (2) **39** (1993), 87–106. [Author-hosted text](#). Corollary 4 supplies the algebraic-closure input for divisible Hahn workspaces; Corollary 7 the characteristic-zero Puiseux extension in rank one.

- [12] Daniel Perrucci and Marie-Françoise Roy. “Elementary recursive quantifier elimination based on Thom encoding and sign determination.” *Annals of Pure and Applied Logic* **168** (2017), no. 8, 1588–1604. doi:10.1016/j.apal.2017.03.001; arXiv:1609.02879v2.
- [13] Daniel Richardson. “Some undecidable problems involving elementary functions of a real variable.” *The Journal of Symbolic Logic* **33** (1968), no. 4, 514–520. doi:10.2307/2271358. The restrictions of this theorem matter; it is not used to assert undecidability of every restricted exponential language.
- [14] Joris van der Hoeven. *Transseries and Real Differential Algebra*. Lecture Notes in Mathematics **1888**, Springer, 2006. doi:10.1007/3-540-35590-1; author-hosted text. Especially the chapters on grid-based series, Newton polygons and operations on transseries. None of the delivered prototypes implements that theory.
- [15] Alessandro Berarducci and Vincenzo Mantova. “Surreal numbers, derivations and transseries.” *Journal of the European Mathematical Society* **20** (2018), 339–390. doi:10.4171/JEMS/769; arXiv:1503.00315.
- [16] Alessandro Berarducci and Vincenzo Mantova. “Transseries as germs of surreal functions.” *Transactions of the American Mathematical Society* **371** (2019), 3549–3592. doi:10.1090/tran/7428; arXiv:1703.01995v3.
- [17] Ovidiu Costin and Philip Ehrlich. “Integration on the surreals.” *Advances in Mathematics* **452** (2024), article 109823. doi:10.1016/j.aim.2024.109823; arXiv:2208.14331v5. A distinct constructive extension and integration framework with its own specified function classes; *not* identified here with the supplied manuscripts’ coefficientwise contour functional or with the local Taylor-lift interface.
- [18] Vincent Bagayoko and Joris van der Hoeven. “Surreal numbers as hyperseries.” arXiv:2310.14879v1, October 23, 2023. Cited for its mathematical representation theorem, not for a finite universal encoding or an implementation claim.
- [19] Vincenzo Mantova. “Monotonicity and a Taylor approximation theorem for transseries.” arXiv:2601.07747v2, revised May 9, 2026. Preprint record consulted September 21, 2026. A semantic theorem for specified transserial classes.
- [20] Shaoshi Chen, Hanqian Fang, and Joris van der Hoeven. “A Zero-Test for D-Algebraic Transseries.” In *Proceedings of ISSAC 2026*, ACM, 2026, pp. 105–113. Published July 12, 2026. doi:10.1145/3815436.3815443; arXiv:2602.01188; author-hosted text. Especially Sections 2.4–2.5 and 7, including Theorem 18. **The only positive general decidability result carried here beyond the algebraic core**, and only under its stated hypotheses.
- [21] Olivier Bournez and Quentin Guilmant. “Surreal fields stable under exponential, logarithmic, derivative and anti-derivative functions.” Preprint, 2022. arXiv:2211.08396.
- [22] Eduardo Cattani, Alicia Dickenstein, and Bernd Sturmfels. “Computing multidimensional residues.” In L. González-Vega and T. Recio (eds.), *Algorithms in Algebraic Geometry and Applications*, Progress in Mathematics **143**, Birkhäuser, 1996, pp. 135–164. doi:10.1007/978-3-0348-9104-2_8; arXiv:alg-geom/9404011.
- [23] Wolfram Research. *Root*. Wolfram Language documentation. <https://reference.wolfram.com/language/ref/Root.html>. Consulted September 21, 2026. The consulted documentation includes general-equation neighbourhood representations as well as polynomial indexing.
- [24] Wolfram Research. *AlgebraicNumber*; *Algebraic Number Fields*. Wolfram Language documentation. <https://reference.wolfram.com/language/ref/AlgebraicNumber.html>; <https://reference.wolfram.com/language/tutorial/AlgebraicNumberFields.html>. Consulted September 21, 2026.
- [25] Wolfram Research. *RootReduce*. Wolfram Language documentation. <https://reference.wolfram.com/language/ref/RootReduce.html>. Consulted September 21, 2026.

- [26] Wolfram Research. *SeriesData*. Wolfram Language documentation. <https://reference.wolfram.com/language/ref/SeriesData.html>. Consulted September 21, 2026.
- [27] Wolfram Research. *SeriesCoefficient*. Wolfram Language documentation. <https://reference.wolfram.com/language/ref/SeriesCoefficient.html>. Consulted September 21, 2026.
- [28] Wolfram Research. *Series*. Wolfram Language documentation. <https://reference.wolfram.com/language/ref/Series.html>. Consulted September 21, 2026.
- [29] Wolfram Research. *InverseSeries*. Wolfram Language documentation. <https://reference.wolfram.com/language/ref/InverseSeries.html>. Consulted September 21, 2026.
- [30] Wolfram Research. *Asymptotic*. Wolfram Language documentation. <https://reference.wolfram.com/language/ref/Asymptotic.html>. Consulted September 21, 2026.
- [31] Wolfram Research. *AsymptoticSolve*. Wolfram Language documentation. <https://reference.wolfram.com/language/ref/AsymptoticSolve.html>. Consulted September 21, 2026.
- [32] Wolfram Research. *DifferentialRoot*. Wolfram Language documentation. <https://reference.wolfram.com/language/ref/DifferentialRoot.html>. Consulted September 21, 2026.
- [33] Wolfram Research. *UpValues*. Wolfram Language documentation. <https://reference.wolfram.com/language/ref/UpValues.html>. Consulted September 21, 2026.
- [34] Wolfram Research. *Inactive*. Wolfram Language documentation. <https://reference.wolfram.com/language/ref/Inactive.html>. Consulted September 21, 2026.
- [35] Wolfram Research. *NumericQ*. Wolfram Language documentation. <https://reference.wolfram.com/language/ref/NumericQ.html>. Consulted September 21, 2026.
- [36] Wolfram Research. *PossibleZeroQ*. Wolfram Language documentation. <https://reference.wolfram.com/language/ref/PossibleZeroQ.html>. Consulted September 21, 2026. Documented as a heuristic facility.
- [37] Wolfram Research. *Infinity*. Wolfram Language documentation. <https://reference.wolfram.com/language/ref/Infinity.html>. Consulted September 21, 2026.
- [38] Wolfram Research. *Mathematical Functions: Working with Special Functions*. Wolfram Language documentation. <https://reference.wolfram.com/language/tutorial/MathematicalFunctions>. Consulted September 21, 2026.
- [39] The Sage Developers. *Lazy Series*. SageMath Reference Manual. https://doc.sagemath.org/html/en/reference/power_series/sage/rings/lazy_series.html. Consulted September 21, 2026.
- [40] The Sage Developers. *Asymptotic Ring*. SageMath Reference Manual. https://doc.sagemath.org/html/en/reference/asymptotic/sage/rings/asymptotic/asymptotic_ring.html. Consulted September 21, 2026.